Linux 驱动开发基础



正点原子公司名称 ：广州市星翼电子科技有限公司

原子哥在线教学平台：www.yuanzige.com

开源电子网 / 论坛 ：http://www.openedv.com/forum.php

正点原子淘宝店铺 ：https://openedv.taobao.com

正点原子官方网站 ：www.alientek.com

正点原子 B 站视频 ：https://space.bilibili.com/394620890

电话：020-38271790 传真：020-36773971

请关注正点原子公众号，资料发布更新我们会通知。



扫码关注正点原子公众号

文档更新说明

版本	版本更新说明	负责人	校审	发布日期
V1.0	初稿：	正点原子 linux 团队	正点原子 linux 团队	2023.07.03

目录

前言 6
第一章 VMware 虚拟机安装 8
1.1 安装虚拟机软件 Vmware 9
1.2 创建虚拟机 16
第二章 安装 Ubuntu 操作系统 26
2.1 获取 Ubuntu 系统 27
2.2 安装 Ubuntu 操作系统 27
2.3 VMware tools 安装 39
第三章 Ubuntu 开发环境搭建 43
3.1 Ubuntu 换源 44
3.2 Ubuntu 和 Windows 文件互传 46
3.3 Visual Studio Code 软件的安装和使用 53
3.3.1 Visual Studio Code 的安装和卸载 53
3.3.2 Visual Studio Code 插件的安装 56
3.3.3 vscode 远程 Ubuntu 系统 58
3.3.4 vscode 的使用 67
第四章 Ubuntu 系统入门 73
4.1 Ubuntu 系统初体验 74
4.1.1 Hello Ubuntu 74
4.1.2 系统设置 76
4.1.3 系统关机与锁定 79
4.1.4 中文输入测试 80
4.2 Ubuntu 终端操作 83
4.3 Shell 操作 84
4.3.1 Shell 简介 84
4.3.2 Shell 基本操作 85
4.3.3 常用 Shell 命令 86
4.4 APT 下载工具 94
4.5 Ubuntu 下文本编辑 98
4.5.1 Gedit 编辑器 98
4.5.2 VI/VIM 编辑器 99
4.6 Linux 文件系统 103
4.6.1 Linux 文件系统简介以及类型 103
4.6.2 Linux 文件系统结构 105
4.6.3 文件操作命令 108
4.6.4 文件压缩和解压缩 113
4.6.5 文件查询和搜索 118
4.6.6 文件类型 119
4.7 Linux 用户权限管理 120
4.7.1 Ubuntu 用户系统 120
4.7.2 权限管理 121

4.7.3 权限管理命令 123
4.8 Linux 磁盘管理 125
4.8.1 Linux 磁盘管理基本概念 125
4.8.2 磁盘管理命令 126
第五章 Linux C 编程入门 130
5.1 Hello World！ 131
5.1.1 编写代码 131
5.1.2 编译代码 132
5.2 GCC 编译器 134
5.2.1 gcc 命令 134
5.2.2 编译错误警告 134
5.2.3 编译流程 135
5.3 Makefile 基础 136
5.3.1 何为 Makefile 136
5.3.2 Makefile 的引入 136
5.4 Makefile 语法 140
5.4.1 Makefile 规则格式 140
5.4.2 Makefile 变量 141
5.4.3 Makefile 模式规则 143
5.4.4 Makefile 自动化变量 144
5.4.5 Makefile 伪目标 145
5.4.6 Makefile 条件判断 146
5.4.7 Makefile 函数使用 147

前言

本教程源自正点原子 i.mx6ull 经典 Linux 驱动开发指南，由正点原子左忠凯编写。于 2019 年发布。与 i.mx6ull 阿尔法开发板一起上市，立刻赢得业界好评反响。创下预售 5000 多套阿尔法开发板的记录。同时开源被业界争相模仿！

为了兼初学者入门 Linux，特别将介绍 Linux 基础部分从 i.mx6ull 上的 Linux 驱动开发指南基础部分离到此文档上。如果你已经熟悉 Linux 基础操作，这部分文档你不必再看。

免责声明

本文档所提及的产品规格和使用说明仅供参考，如有内容更新，恕不另行通知；除非有特殊约定，本文档仅作为产品指导，所作陈述均不构成任何形式的担保。本文档版权归广州市星翼电子科技有限公司所有，未经公司的书面许可，任何单位和个人不得以营利为目的进行任何方式的传播。

为了得到最新版本的产品信息，请用户定时访问正点原子资料下载中心或者与淘宝正点原子旗舰店客服联系索取。感谢您的包容与支持。

第一章 VMware 虚拟机安装

如果你在其他文档有安装过 Ubuntu，则不必再看本章。本文档再重复 Ubuntu 安装是为了文档的完整性。

Linux 的开发需要在 Linux 系统下进行，这就要求我们的 PC 主机安装 Linux 系统，本教程我们选择 Ubuntu 这个 Linux 发行版系统。本章讲解如何安装虚拟机，以及如何在虚拟机中安装 Ubuntu 系统，安装完成以后如何做简单的设置。如果已经对于虚拟机以及 Ubuntu 基础操作已经熟悉的话就可以跳过本章。

本章主要讲以下内容

1.1 安装虚拟机软件 Vmware

1.2 创建虚拟机

1.1 安装虚拟机软件 Vmware

不是安装 Ubuntu 吗？怎么要先安装虚拟机呢？虚拟机是个啥？相信大部分第一次安装 Ubuntu 的人都会有这个疑问。我不能直接安装 Ubuntu 吗？能不能不要虚拟机呢？答案是肯定可以的！直接在电脑上安装 Ubuntu 以后你的电脑就是一个真真正正的 Ubuntu 电脑了，你可以再安装一个 Windows 系统，这样你的电脑就是双系统了，在开机的时候可以选择不同的系统启动。但是这样的话会有一个问题，那就是你每次只能选择其中的一个系统启动，要么 Windows 要么 Ubuntu，但是我们再开发的时候很多时候需要在 Windows 和 Ubuntu 下来回切换，Windows 系统下的软件资源要比 Ubuntu 下丰富的多，这个就涉及到两个系统切换问题，显然如果你直接在电脑上安装 Ubuntu 以后就没法做到，因为你每次开机只能在 Windows 和 Ubuntu 下二选一。

如果 Ubuntu 系统能作为 Windows 下的一个软件就好了，我们默认启动 Windows 系统，需要用到 Ubuntu 的话直接打开这个软件就行了。这个当然是可以的！这里就要借助虚拟机了，虚拟机顾名思义就是虚拟出一个机器，然后你就可以在这个机器上安装任何你想要的系统，相当于再克隆出一个你的电脑，这样在主机上运行 Windows 系统，当我们需要用到 Ubuntu 的话就打开安装有 Ubuntu 系统的虚拟机。

虚拟机的实现我们可以借助其他软件，比如 Vmware Workstation，Vmware Workstation 是收费软件，免费的虚拟机软件有 Virtualbox。本文档我们使用 Vmware Workstation 软件来做虚拟机。Vmware Workstation 软件可以在 Wmware 官网下载，下载地址：https://www.vmware.com/products/workstation-pro/workstation-pro-evaluation.html，当前最新的版本是 Vmware Workstation Pro 16，我们下载 Windows 版本的，如图 1.1.1 所示：

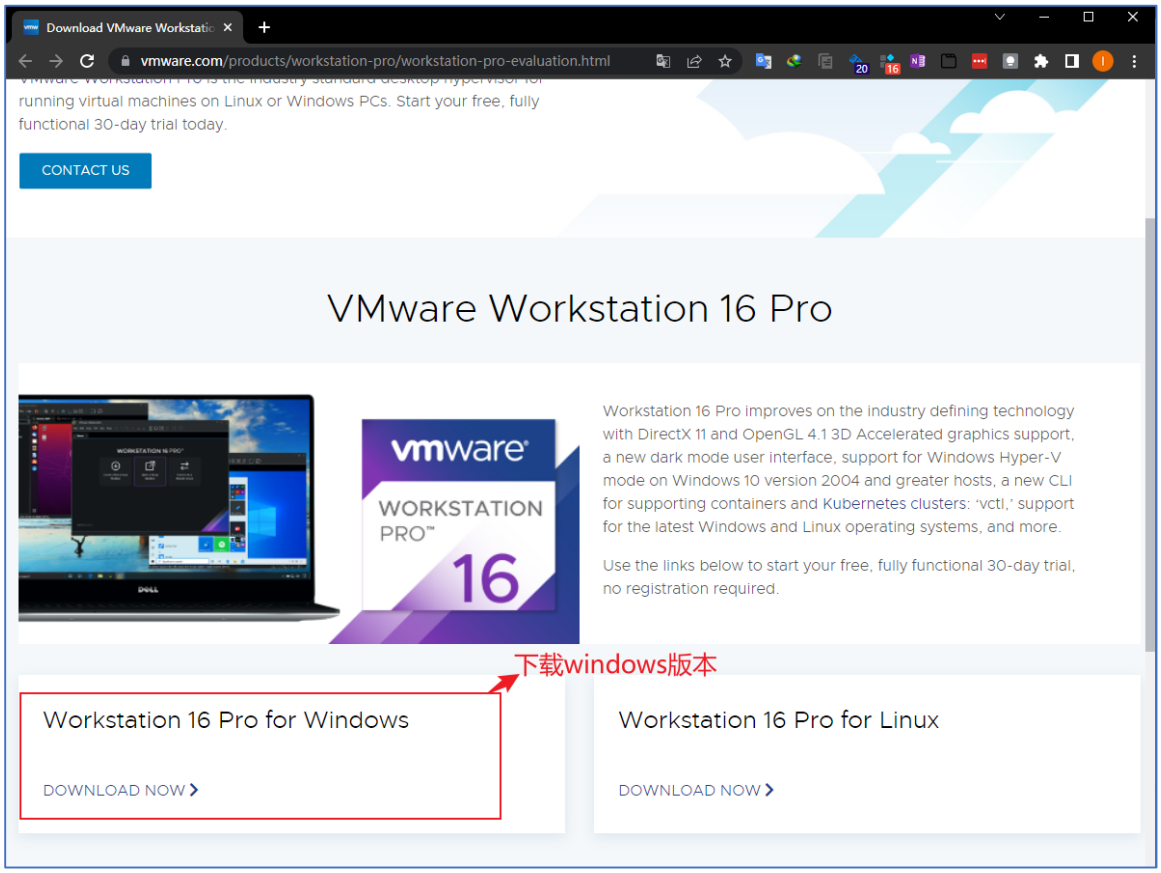


图 1.1.1 Vmware 下载页

我们已经在开发板光盘里面提供了 Vmware Workstation 软件，大家可以直接使用，在光盘目录：开发板光盘 A-基本资料→4、软件→VMware-workstation-full-16.2.x-xxxxxxxx.exe。WMware Workstation 的安装和普通软件安装一样，双击 VMware-workstation-full-16.2.x-xxxxxxxx.exe 进入安装界面(xxxx 代表版本号，截止 2023.05.25 最新是 VM17，但是笔者选择稳定好用的 16，日后版本有更新，安装步骤都是类似的)，如图 1.1.2 所示：

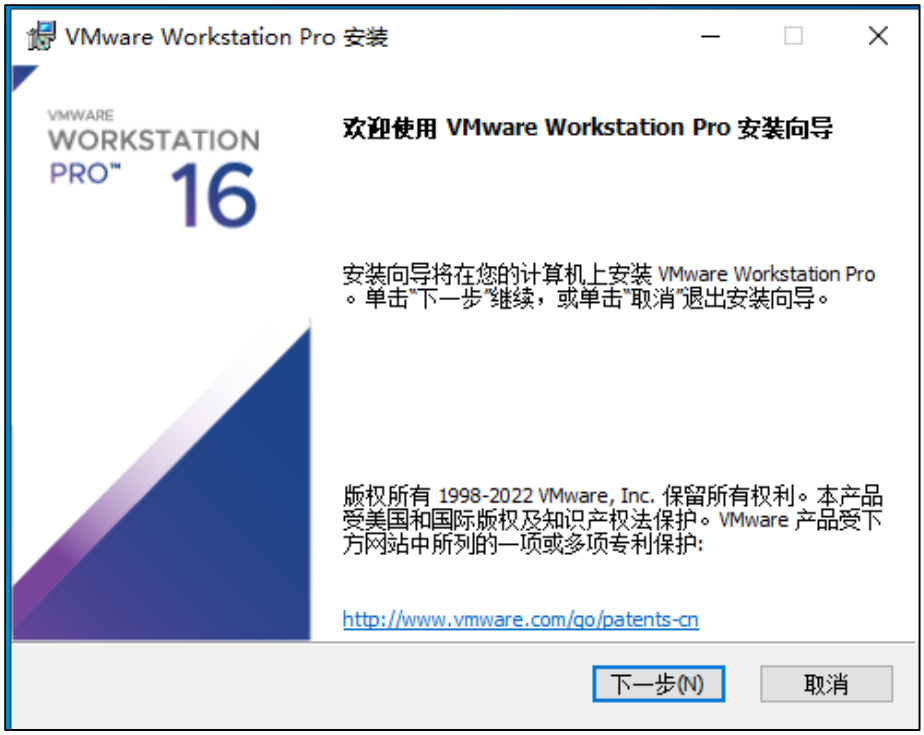


图 1.1.1 VMware 安装界面

勾选接受许可协议条款，点击图 1.1.2 中的“下一步”，进入图 1.1.3 所示步骤：

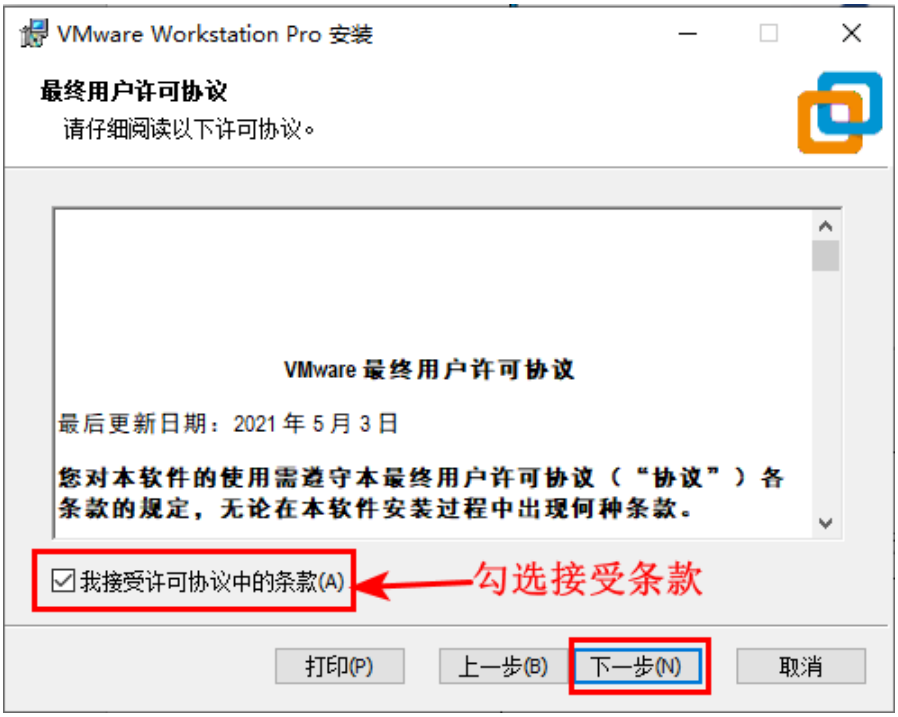


图 1.1.3 VMware 条款

先选择图 1.1.3 中的“我接受许可协议中的条款”，然后在选择“下一步”，进入图 1.1.4 所示步骤：

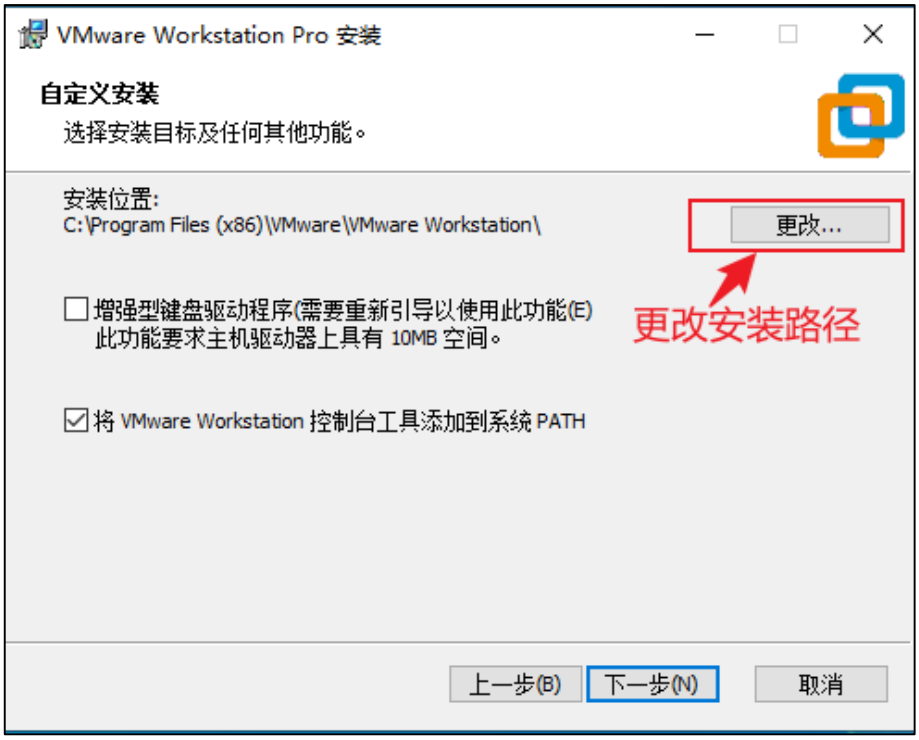


图 1.1.4 选择安装路径

图 1.1.4 中选择软件的安装路径，点击“更改”按钮，然后根据自己的实际需要选择合适路径即可,我的安装路径如图 1.1.5 所示：

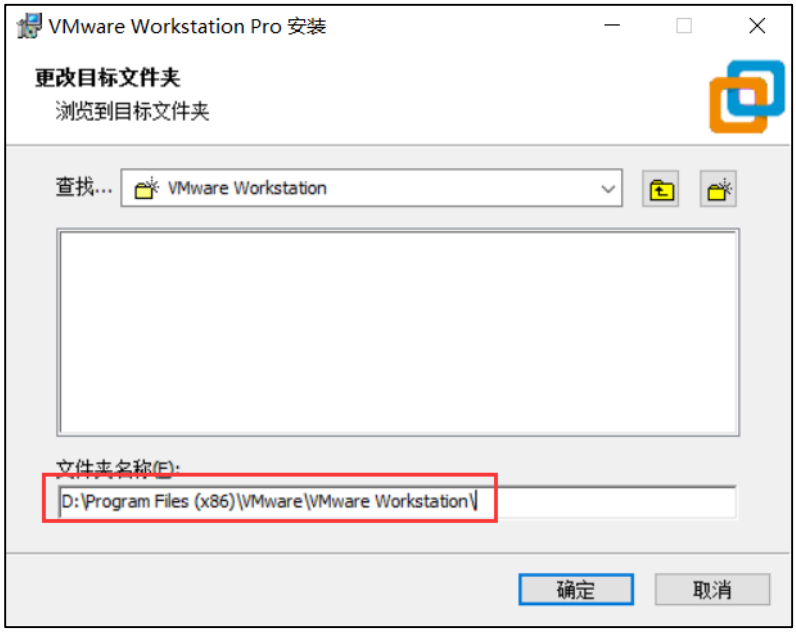


图 1.1.5 安装路径

选择好路径以后点击图 1.1.5 中的“确定”按钮，然后回到图 1.1.4 所示界面，点击图 1.1.4 中的“下一步”，进入图 1.1.6 所示界面：

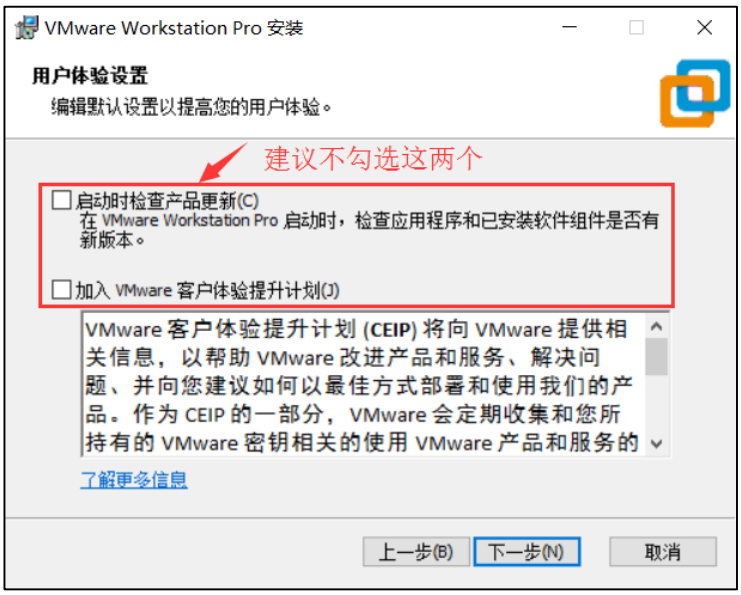


图 1.1.6 检查更新界面

在图 1.1.6 中，会有两个复选框，默认都是选中的，建议不要选中！然后点击图 1.1.6 中的“下一步”按钮，进入图 1.1.7 所示界面：

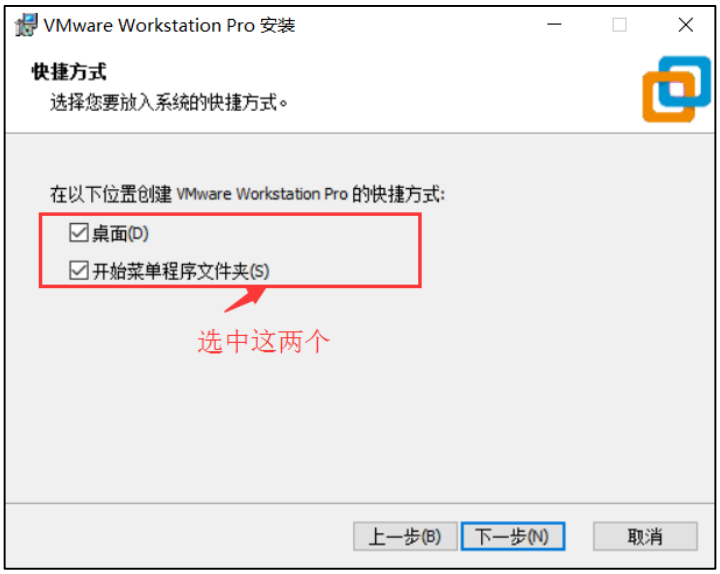


图 1.1.7 快捷方式设置

在图 1.1.7 中有两个选项，我们都选中，这样在安装完成以后就会在开始菜单和桌面上有 VMware 的图标，选中以后点击图 1.1.7 中的“下一步”，进入图 1.1.8 界面：

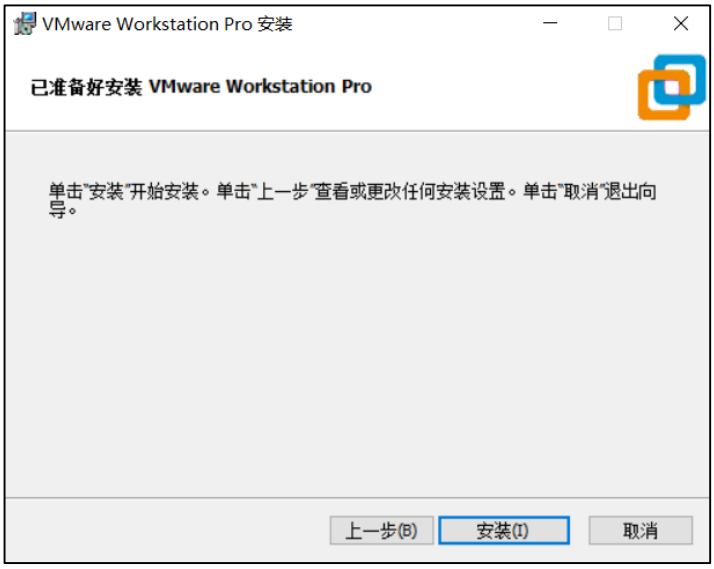


图 1.1.8 安装确定界面

前面几步已经设置好安装参数了，如果不需要修改安装参数的话就点击图 1.1.8 中的“安装”按钮开始安装 VMware，安装过程如图 1.1.9 所示：

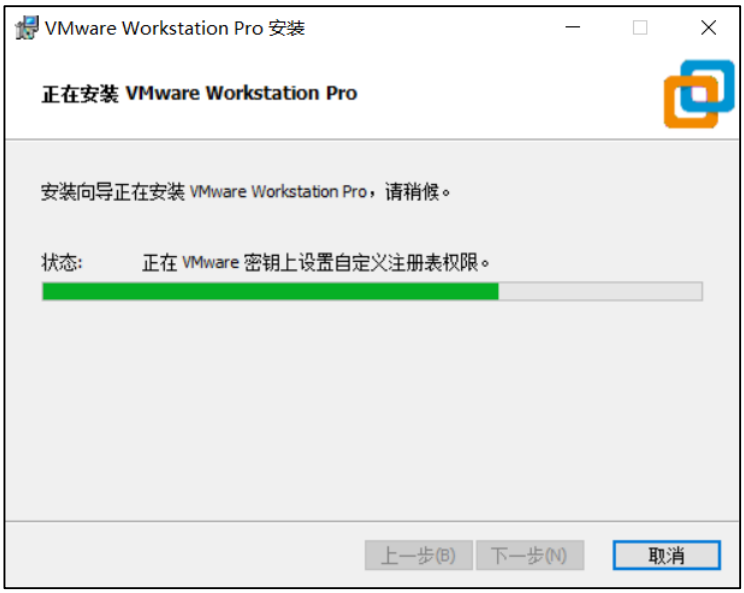


图 1.1.9 安装进行中

图 1.1.9 就是安装过程，耐心等待几分钟，等待安装完成，安装完成以后会有如图 1.1.10 所示提示：

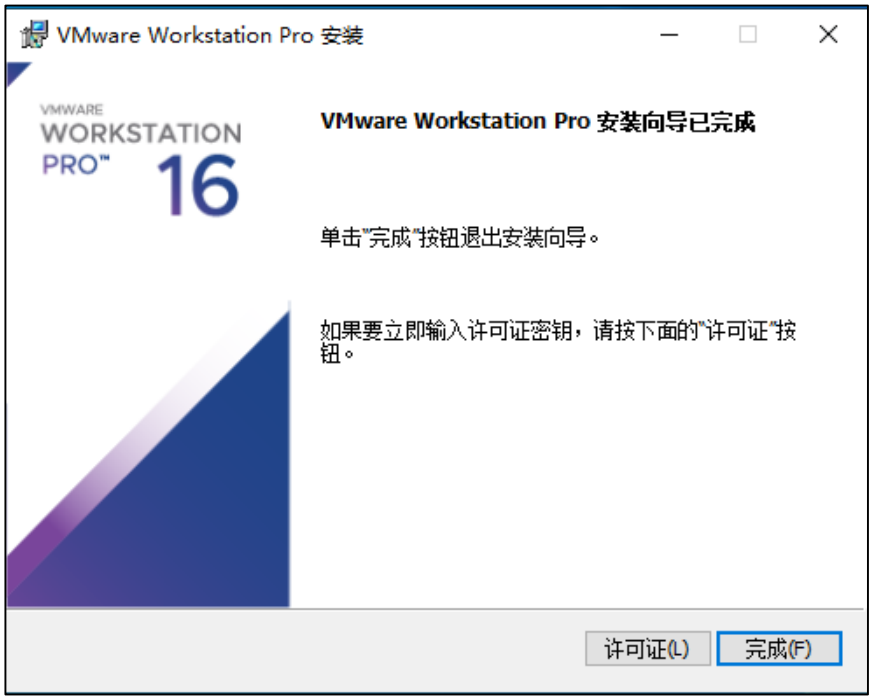


图 1.1.10 安装完成

点击图 1.1.10 中的“完成”按钮，完成 VMware 的安装，安装完成以后就会在桌面上出现 VMware Workstation Pro 的图标，如图 1.1.11 所示：

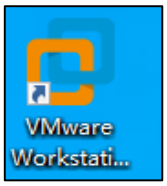


图 1.1.11 VMware 桌面图标

双击图 1.1.11 中的图标打开 VMware 软件，在第一次打开软件的时候会提示你输入许可证密钥，如图 1.1.12 所示：



图 1.1.12 输入许可证密钥

前面说了 VMware 是付费软件，是需要购买的，如果你购买了 VMware 的话就会有一串许可密钥，如果没有购买的话就选择“我希望试用 VMware Workstation 16 30 天”选项，这样你就可以体验 30 天 VMware。输入密钥以后点击“继续按钮”，如果你的密钥正确的话就会提示你购买成功，如图 1.1.13 所示：



图 1.1.13 购买 VMware 成功

点击图 1.1.13 中的“完成”按钮，VMware 软件正式打开，界面如图 1.1.14 所示：

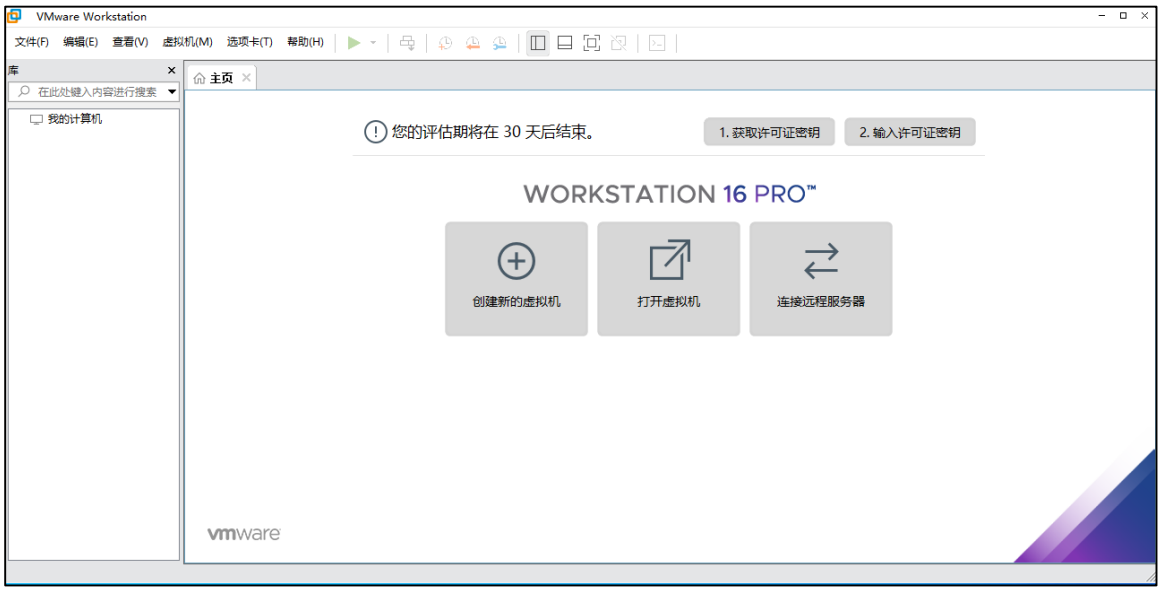


图 1.1.15 VMware Workstation 主界面

至此，虚拟机软件 VMware 安装成功。

1.2 创建虚拟机

安装好 VMware 以后我们就可以在 VMware 上创建一个虚拟机，打开 VMware，选择：文件->新建虚拟机，如图 1.2.1 所示：

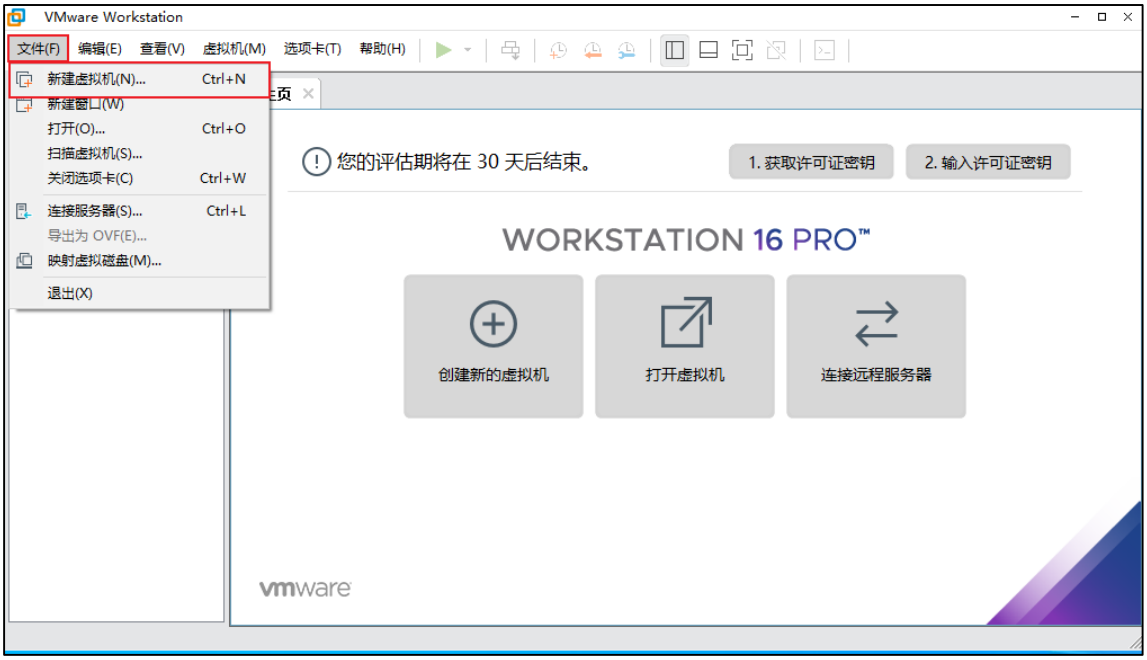


图 1.2.1 新建虚拟机

打开图 1.2.2 所示创建虚拟机向导界面：

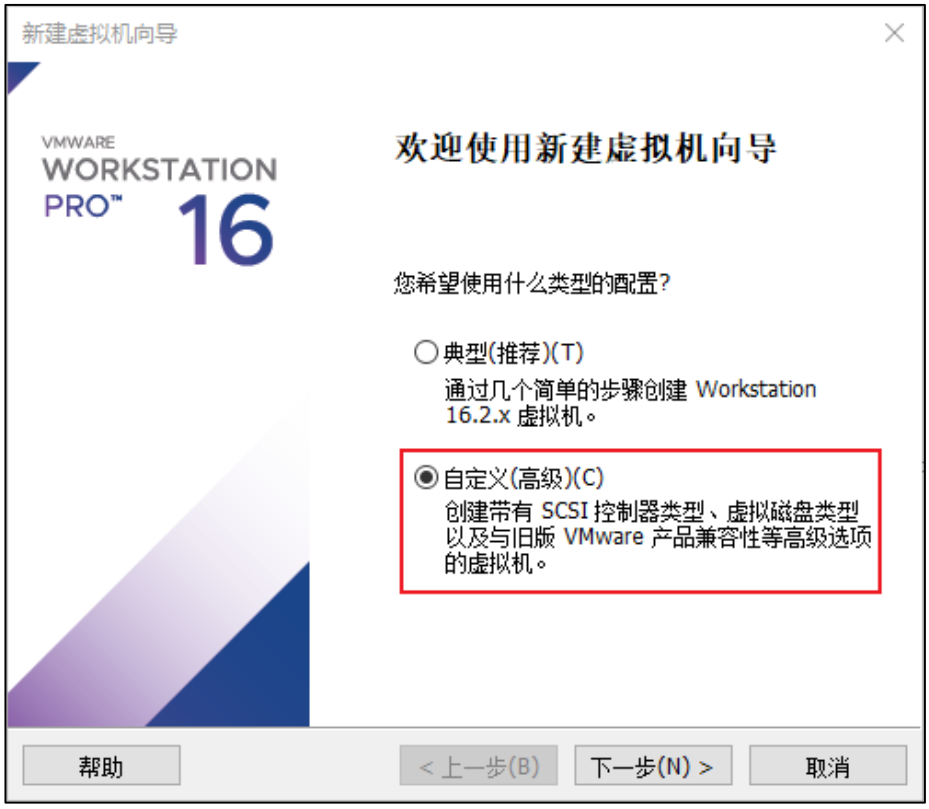


图 1.2.2 创建虚拟机向导

选中图 1.2.2 中的“自定义”选项，然后选择“下一步”，进入图 1.2.3 所示硬件兼容性选择界面：



图 1.2.3 硬件兼容性选择

在图 1.2.3 中我们使用默认值就行了，直接点击“下一步”，进入图 1.2.4 所示的操作系统安装界面：



图 1.2.4 安装客户机操作系统

图 1.2.4 就是选择你新创建的虚拟机要安装什么系统？windows 还是 linux，如果你要现在就安装系统的话需要准备好系统文件，一般是.iso 文件。我们现在不安装系统，因此选择“稍后安装操作系统(S)”这个选项，然后选择“下一步”，进入图 1.2.5 所示界面：



图 1.2.5 客户机操作系统选择

图 1.2.5 中依旧是让你选择你要在虚拟机中装什么系统，图 1.2.5 是和图 1.2.4 配合在一起使用的，在图 1.2.4 中放入系统文件(.iso 文件)，然后在图 1.2.5 中选择你放入的是什么系统，然后 VMware 就会稍后自动安装所设置的系统。在图 1.2.4 中我们没有设置系统文件，因此图 1.2.5 是没用的，不过我们还是在图 1.2.5 中的客户机操作系统一栏选择“Linux”，版本选择 Ubuntu 64 位，然后点击“下一步”，进入图 1.2.6 所示界面：

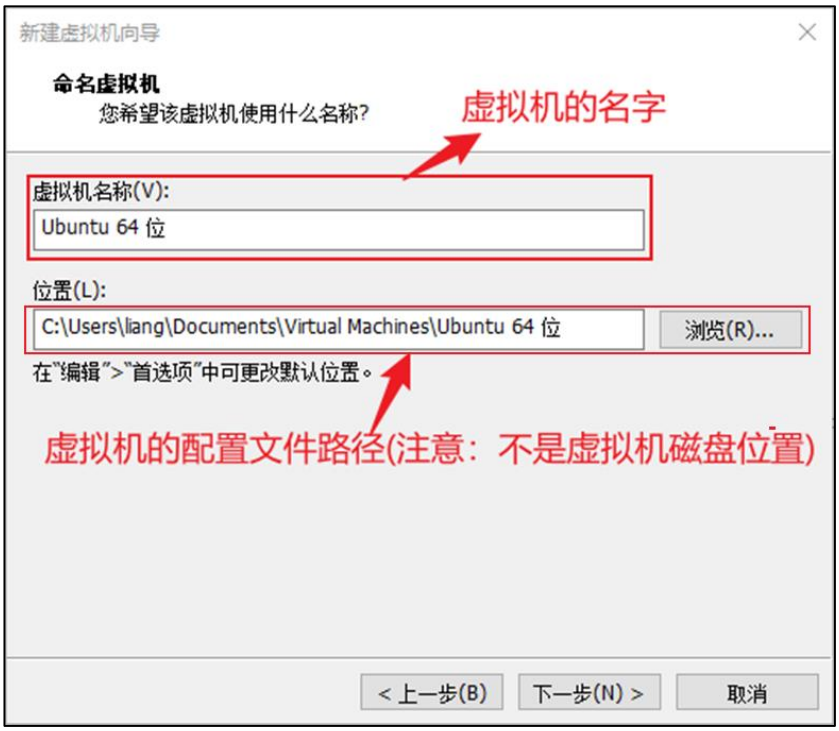


图 1.2.6 命名虚拟机

图 1.2.6 中第一个红色框设置虚拟机名字，第二个红色框里设置虚拟机的配置文件路径，大家可以根据自己的使用习惯给虚拟机命名和设置虚拟机的位置。这里笔者的虚拟机的路径如下图所示：

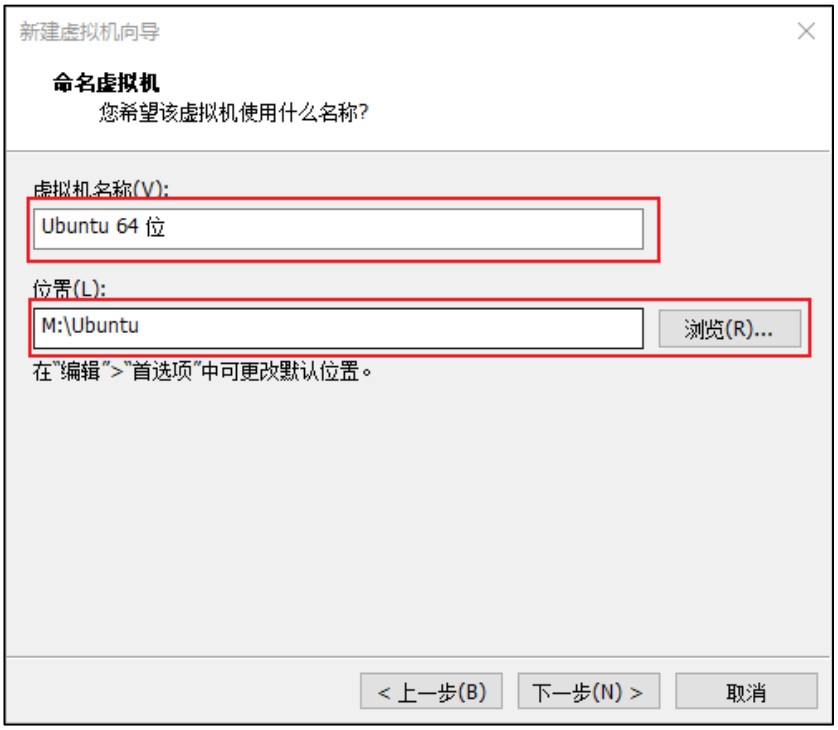


图 1.2.7 设置虚拟机的位置

设置好了，点击“下一步”，进入图 1.2.8 所示的处理器配置选择界面：



图 1.2.8 处理器配置界面

图 1.2.8 中就是配置你的虚拟机所使用的处理器数量，以及每个处理器的内核数量，这个要根据自己实际使用的电脑 CPU 配置来设置。比如笔者的电脑是一个 CPU 的电脑（一般电脑只有一个 CPU），CPU 处理器有 8 个逻辑核，笔者这里分 4 个。VMware 虚拟机配置就如图 1.2.8 所示，大家根据自己的实际电脑 CPU 配置来设置即可，设置好以后点击“下一步”，进入图 1.2.9 所示内存配置界面：

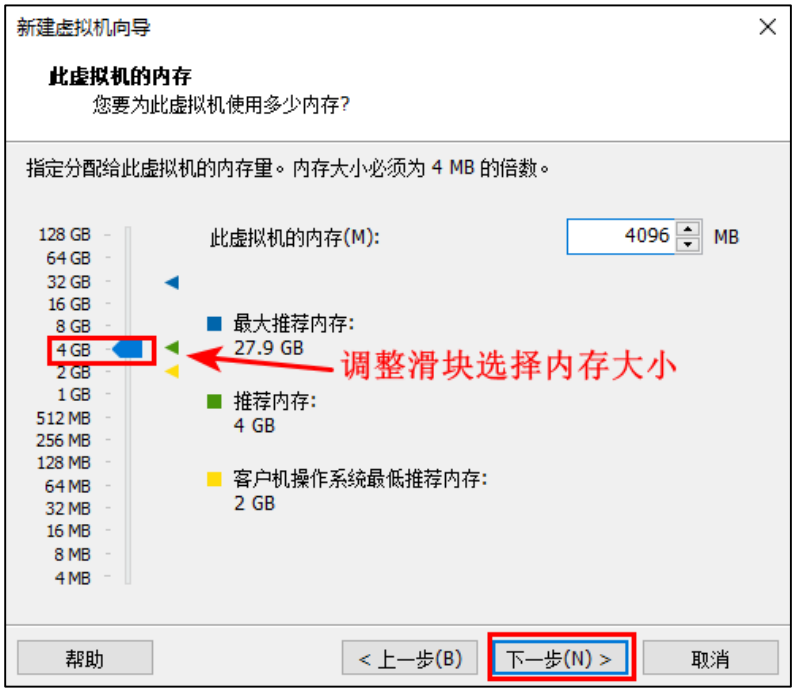


图 1.2.9 内存配置

同样的在图 1.2.9 中根据自己电脑的实际内存配置来设置分给虚拟机的内存大小，比如我的电脑是 32GB 的内存，因此笔者可以给虚拟机最大可分配 27.9GB 的内存。初学者的计算机配置，可能不高，当然是内存越大越好。笔者这里选择分配 4G，内存的大小对编译安卓或者 linux 系统起至关重要的作用。分配的内存根据实际情况可以安装虚拟机后再修改。配置好虚拟机的内存大小以后点击“下一步”，进入图 1.2.10 所示的网络类型选择界面：



图 1.2.10 网络类型选择界面

在图 1.2.10 中我们选择“使用桥接网络”，然后点击“下一步”，进入图 1.2.11 示的选择 I/O 控制器类型界面：



图 1.2.11 I/O 控制器选择

I/O 控制器类型选择默认值就行，也就是“LSI Logic”，然后点击“下一步”，进入磁盘类型选择界面，如图 1.2.12 所示：



图 1.2.12 磁盘类型选择

图 1.2.12 中选择磁盘类型，使用默认值“SCSI”即可，然后点击“下一步”，进入选择磁盘界面，如图 1.2.13 所示：

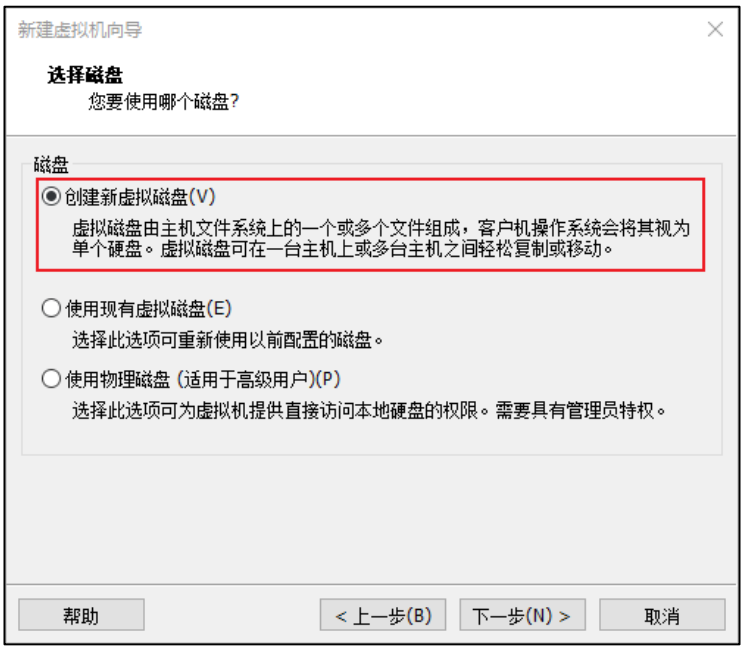


图 1.2.13 磁盘选择

图 1.2.13 中使用默认值，即“创建新虚拟磁盘”，这样我们前面设置好的那个空的磁盘就会被创建为一个新的磁盘，设置要以后点击“下一步”，进入磁盘容量设置界面，如图 1.2.14 所示：



图 1.2.14 磁盘容量设置

图 1.2.14 是用来设置虚拟机的磁盘大小，磁盘大小根据自己的电脑而定，这里笔者设置给 500GB 的空间（实际上不会占 500G 空间，用多少占多少），然后点击“下一步”，进入图 1.2.15 所示界面指定磁盘路径和名字：

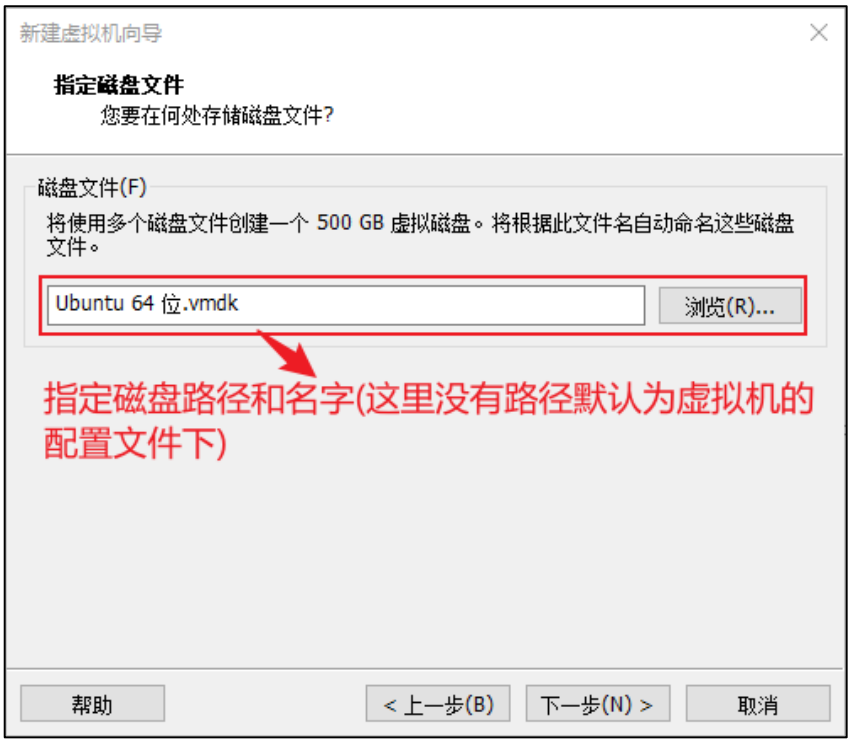


图 1.2.15 指定磁盘文件

图 1.2.15 使用默认设置，不要做任何修改，直接点击“下一步”，进入已准备好创建虚拟机界面，如图 1.2.16 所示：

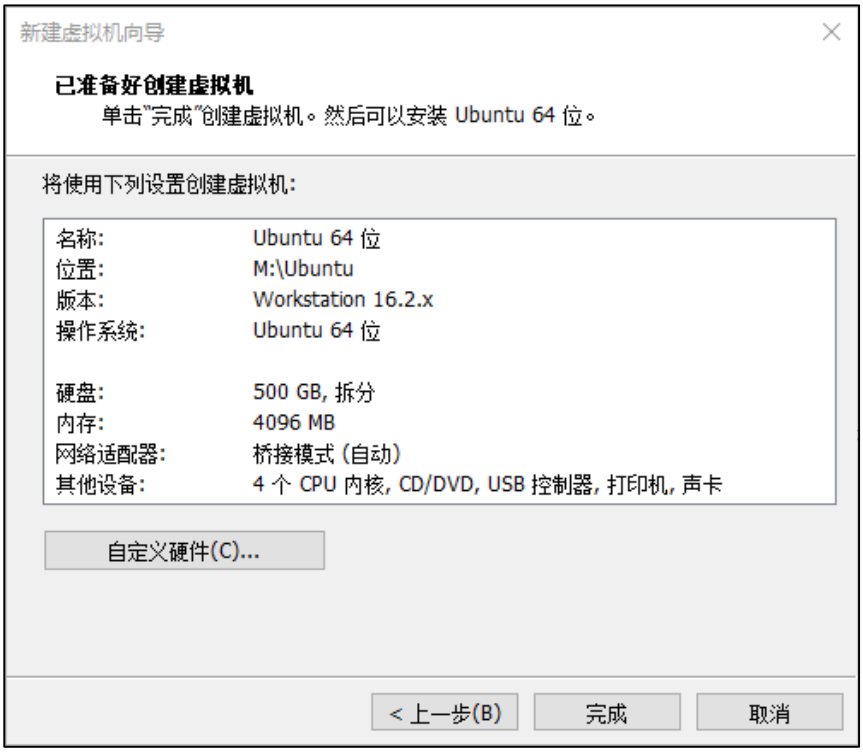


图 1.2.16 准备创建虚拟机

在图 1.2.16 中确认自己的虚拟机配置，如果确认无误就点击“完成”，如果有误的话就返回有误的配置界面做修改，点击“完成”按钮以后就会创建一个虚拟机，如图 1.2.17 所示：

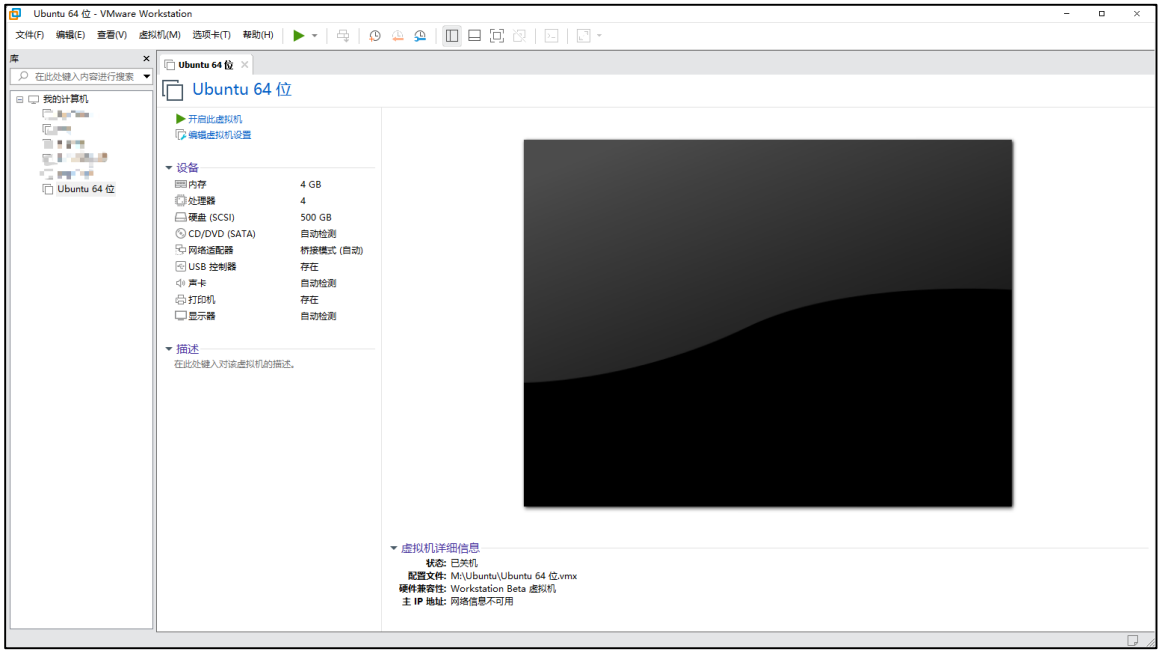


图 1.2.17 新创建的虚拟机

创建虚拟机成功以后就会在右侧的：我的计算机下出现刚刚创建的虚拟机“Ubuntu 64 位”，点击一下就会在右侧打开这个虚拟机的详细信息，如图 1.2.18 所示：



图 1.2.18 新建虚拟机配置信息

在图 1.2.18 中的设备一栏我们可以看到虚拟机详细的配置信息，图 1.2.19 所示示的两个按钮就是虚拟机的开关，



图 1.2.20 虚拟机开关机

图 1.2.20 中的这两个绿色三角按钮都可以打开虚拟机，但是此时虚拟机没有安装任何操作系统，因此没法打开，接下来我们就是要在刚刚新建的这个虚拟机中安装 Ubuntu 操作系统。

第二章 安装 Ubuntu 操作系统

本章主要讲以下内容

2.1 获取 Ubuntu 系统

2.2 安装 Ubuntu 操作系统

2.3 VMware tools 安装

2.1 获取 Ubuntu 系统

前面虚拟机已经创建成功了，相当于硬件已经准备好了，接下来就是要在虚拟机中安装 Ubuntu 系统了，首先肯定是获取到 Ubuntu 的系统镜像，Ubuntu 系统镜像肯定是在 Ubuntu 官网获取，下载地址为：https://www.ubuntu.com/download/desktop，如图 2.1.1 所示：

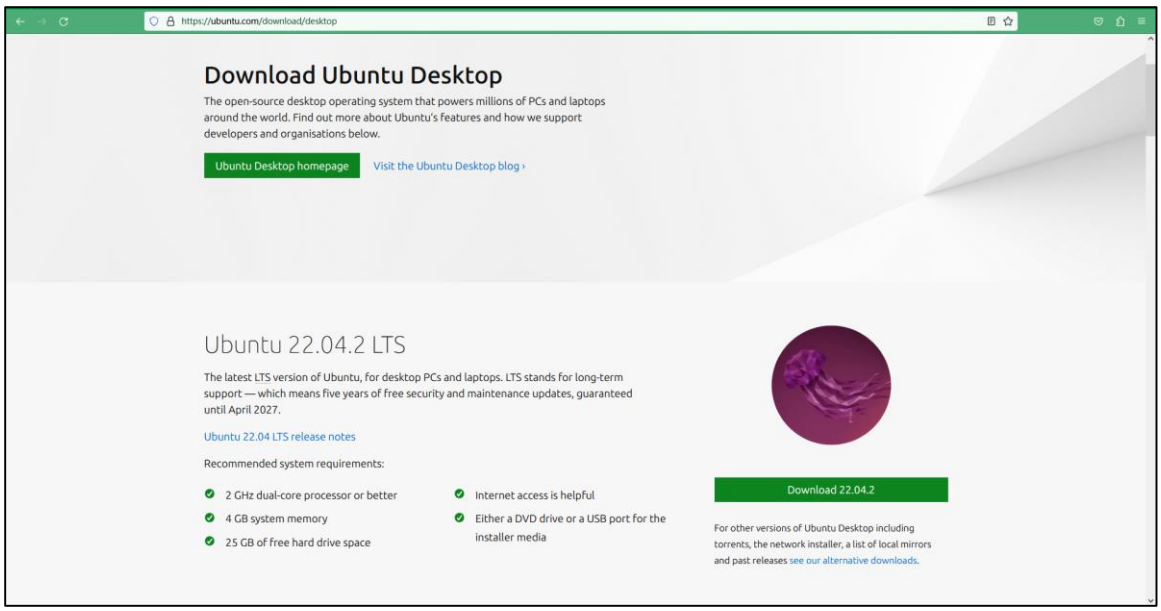


图 2.1.1 Ubuntu 最新版系统下载

从图 2.1.1 中可以看出，截止到 2023.05.25 日最新的 Ubuntu 系统为 22.04.2，此版本是官方长期支持的版本，笔者经过测试发现此版本不适合正点原子许多 Linux 开发板的开发环境。笔者将各个 linux 开发板已经下载下来放到了开发板光盘中，路径为：开发板光盘 A-基础资料→4、软件→ ubuntu-xx.xx.x-desktop-amd64.iso(x 代表未知版本)，正点原子 Linux 开发板较多，请根据开发板光盘 A-基础资料→4、软件→ ubuntu-xx.xx.x-desktop-amd64.iso 确认要安装的 Ubuntu 版本，请用户必须根据提供的 ISO 镜像来安装你的 Ubuntu，请不要随意选择 Ubuntu 开发，因为有些开发板提供的 SDK 都需要在特定版本的 Ubuntu 下开发，因为芯片厂商有特定说明用版本的 Ubuntu，或者>=某个版本的 Ubuntu。

2.2 安装 Ubuntu 操作系统

Ubuntu 系统获取到以后就可以安装了，打开 VMware 软件，选择：虚拟机->设置，如图 2.2.1 所示：

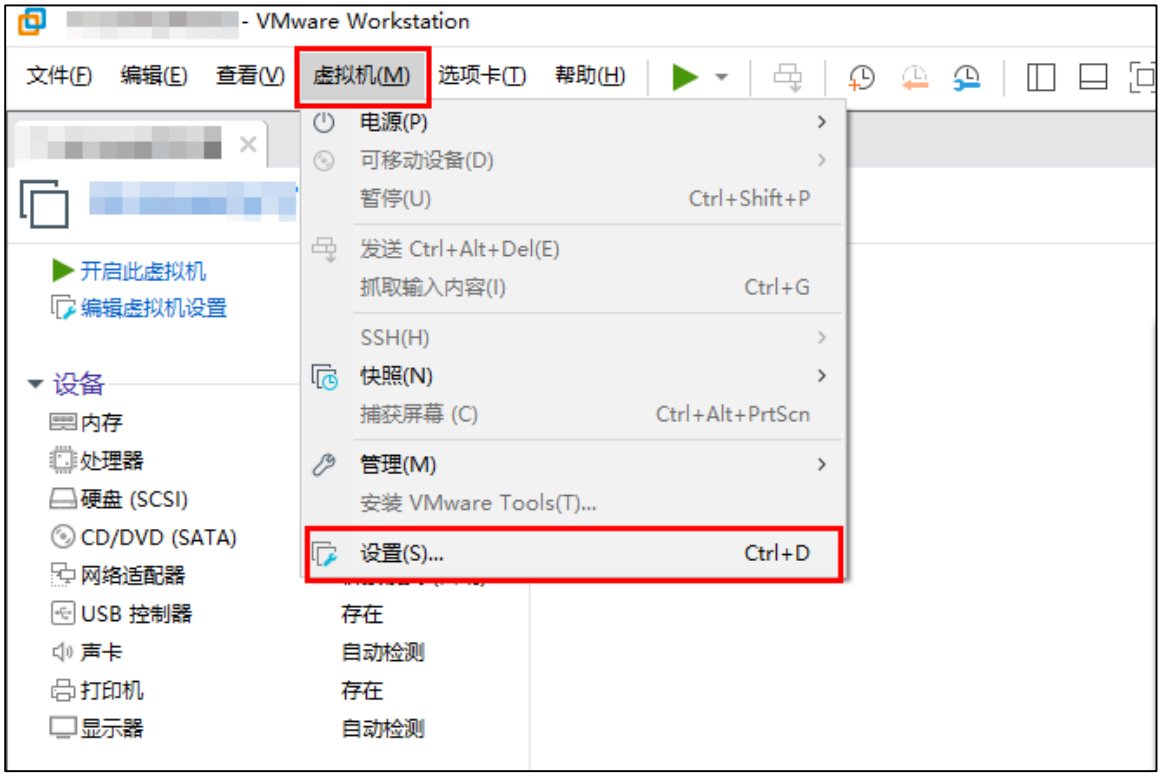


图 2.2.1 打开虚拟机设置对话框

打开以后的虚拟机设置对话框如图 2.2.2 所示：

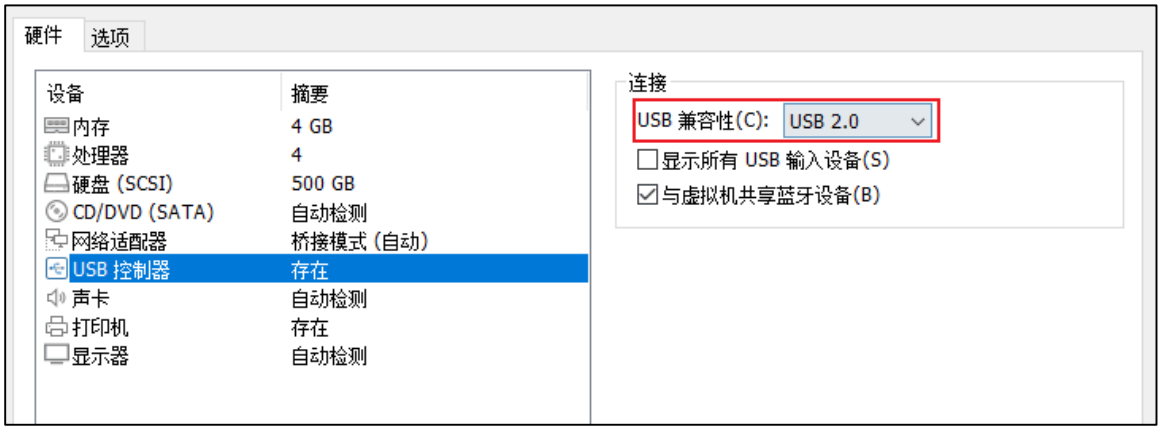


图 2.2.2 虚拟机对话框

首先设置“USB 控制器”选项，默认 USB 控制器的 USB 兼容性为 USB2.0，这样当你使用 USB3.0 的设备的时候 Ubuntu 可能识别不出来，因此我们需要调整 USB 兼容性为 USB3.0，如图 2.2.3 所示：



图 2.2.3 USB 兼容性设置

设置要 USB 兼容性以后就开始安装 Ubuntu 系统了，选中虚拟机设置对话框中的“CD/DVD(SATA)”选项，然后在右侧选中“使用 ISO 映像文件”，如图 2.2.4 所示：

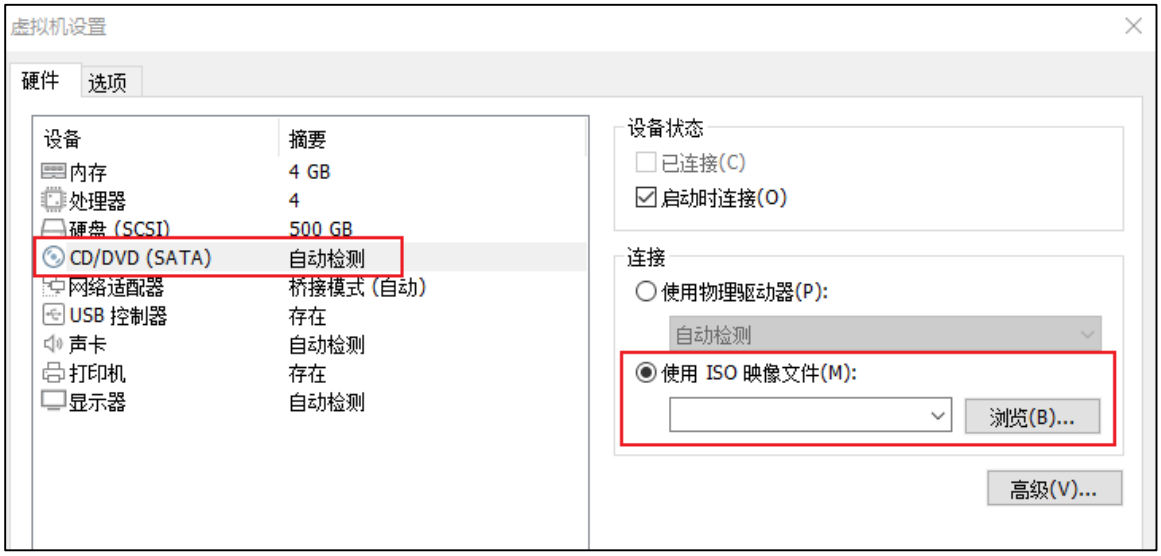


图 2.2.4 系统镜像设置

在图 2.2.4 中的“使用 ISO 映像文件”里面添加我们开发板光盘 A-基础资料→4、软件→ubuntu-xx.xx.x-desktop-amd64.iso，点击“浏览”按钮，选择 Ubuntu 系统镜像，完成以后如图 2.2.5 所示：

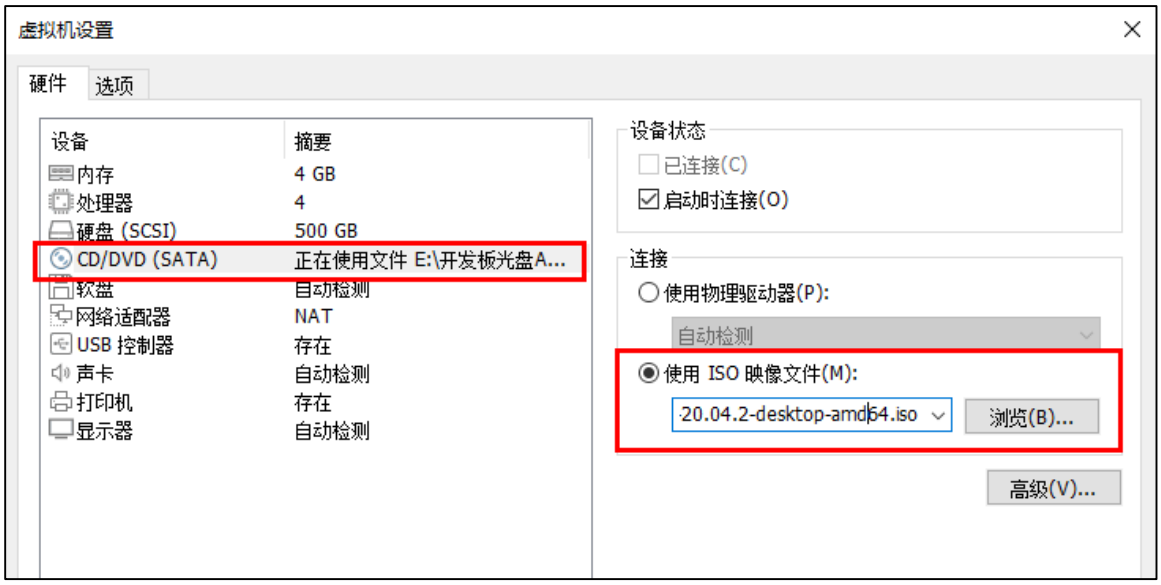


图 2.2.5 Ubuntu 镜像选择

设置好以后点击“确定”按钮退出，退出以后就可以打开虚拟机了，虚拟机就会自动的安装 Ubuntu 系统，如图 2.2.6 所示：

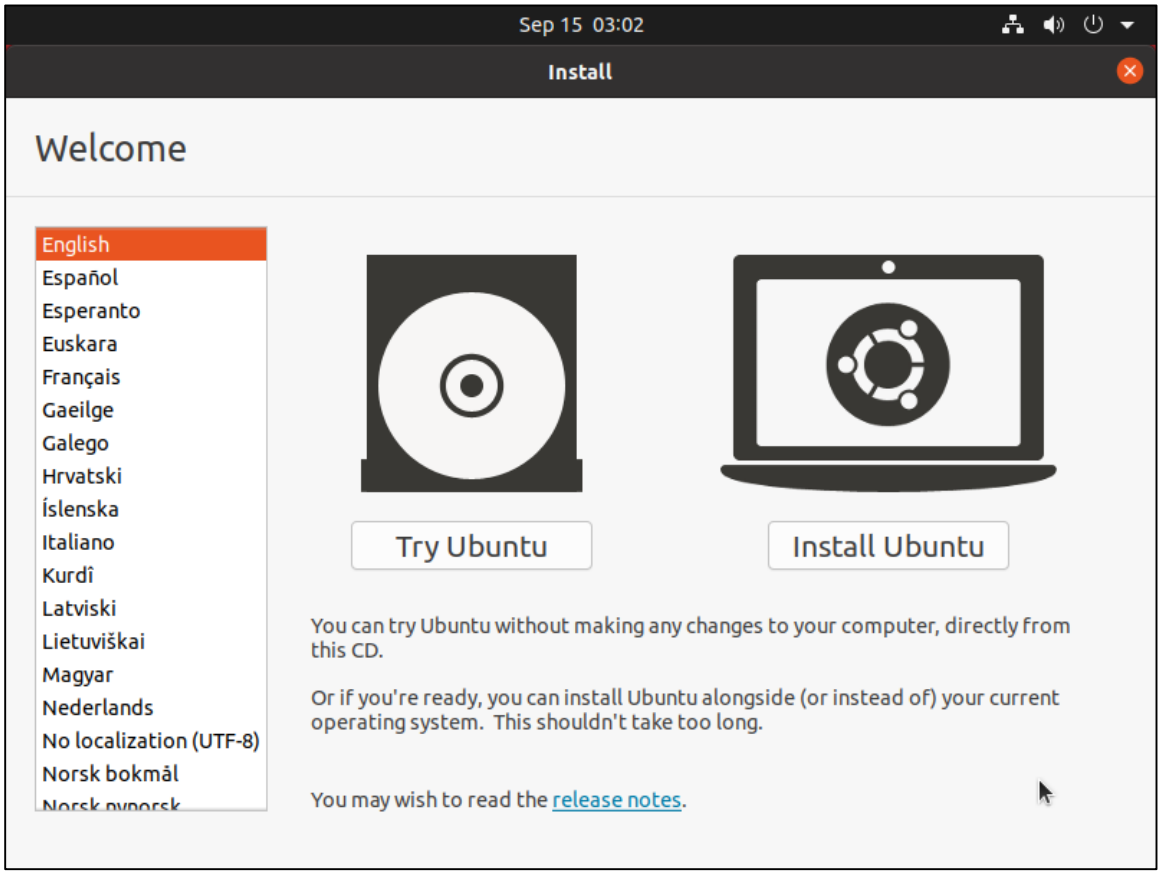


图 2.2.6 Ubuntu 安装开始

Ubuntu 开始安装以后首先是语言选择，如图 2.2.7 所示：



图 2.2.7 语言选择与安装

Ubuntu 默认语言是英文，毫无疑问，我们要选择“中文(简体)”，选择好以后点击右侧的“安装 Ubuntu”按钮，进入安装过程。安装一开始会有 7 个配置步骤，第一配置如图 2.2.8 所示，让你选择键盘布局：

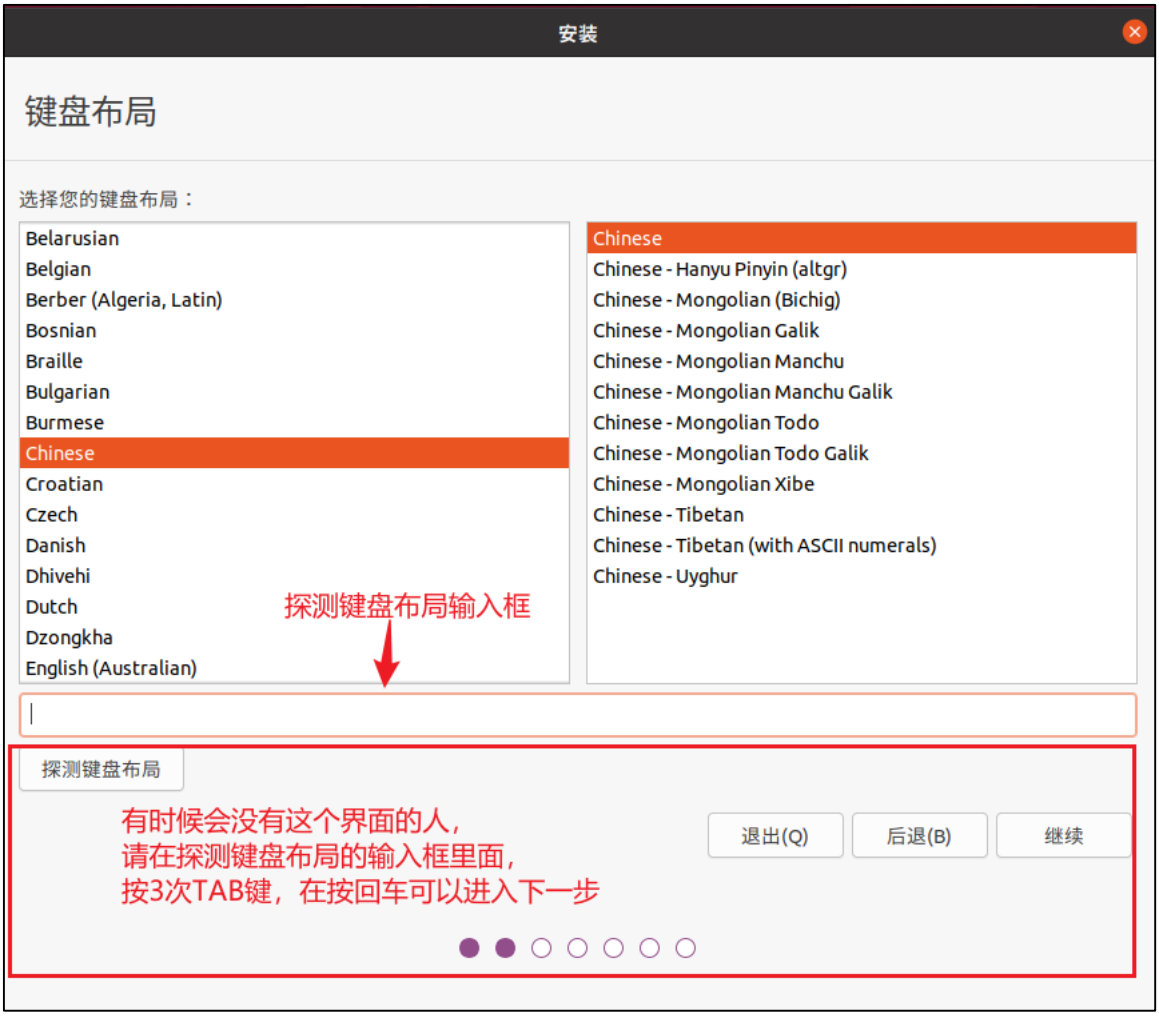


图 2.2.8 键盘布局选择

图 2.2.8 中键盘布局选择“chinese”，然后点击“继续”。接下来就是“更新和其他软件”配置界面，让你选择先安装哪些应用，是否在安装 Ubuntu 时下载更新，以及是否为图形或者无线硬件安装其它第三方软件，配置如图 2.2.9 所示：



图 2..2.9 是否安装是下载更新

直接点击图 2.2.9 中的“继续”按钮，弹出安装类型，使用默认的“清除整个磁盘并安装 Ubuntu”，如图 2.2.10 所示：



图 2.2.10 安装类型选择

设置好安装类型以后点击“现在安装”按钮，会弹出“将改动写入磁盘吗？”对话框，点击“继续”即可，下一步会让你输入你在哪个位置，输入自己所在的城市即可，比如笔者在广州就输入“guangzhou”，如图 2.2.11 所示：

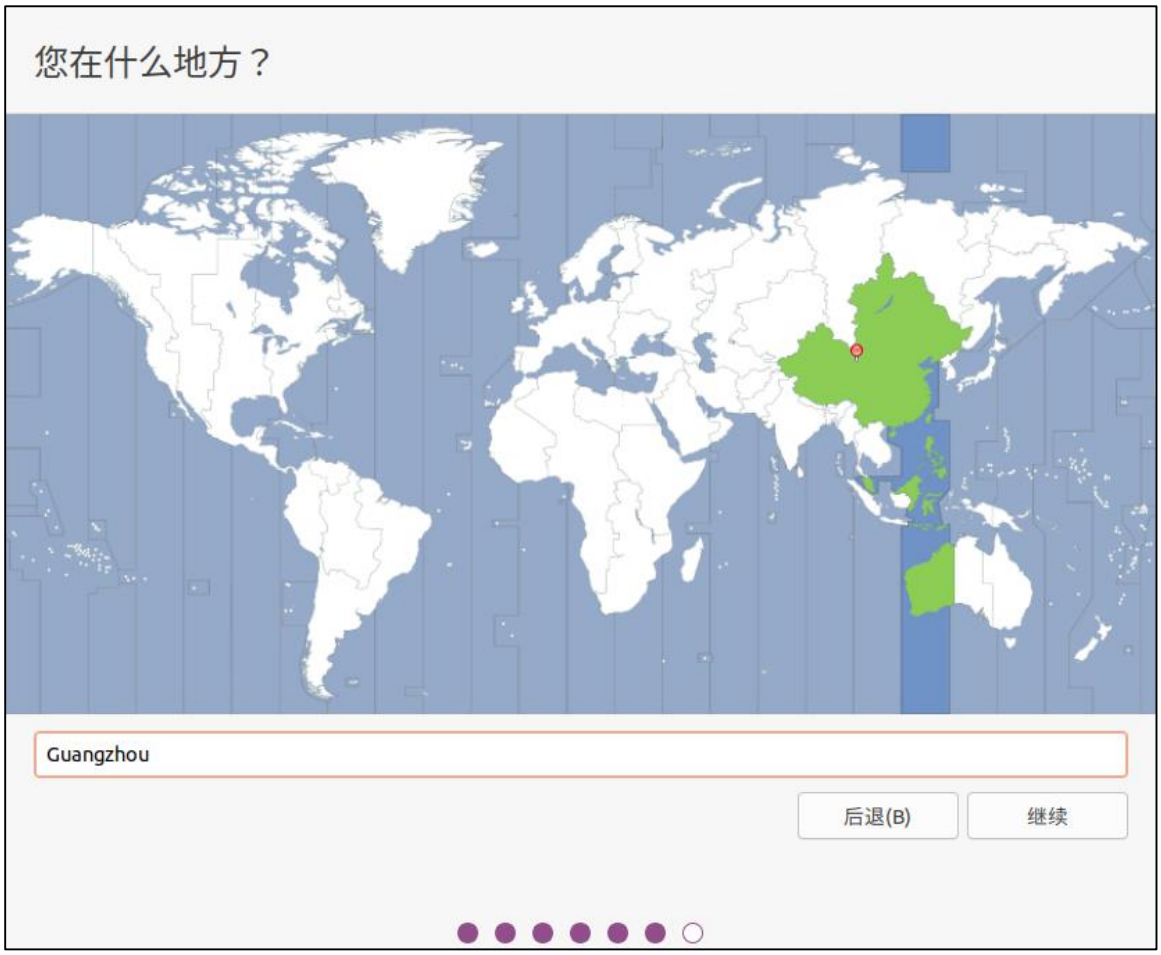


图 2.2.11 输入所在位置

输入地址以后点击“继续”按钮，进入下一步设置用户名和密码，自己设置自己的用户名和密码，比如我的设置如图 2.2.12 所示：

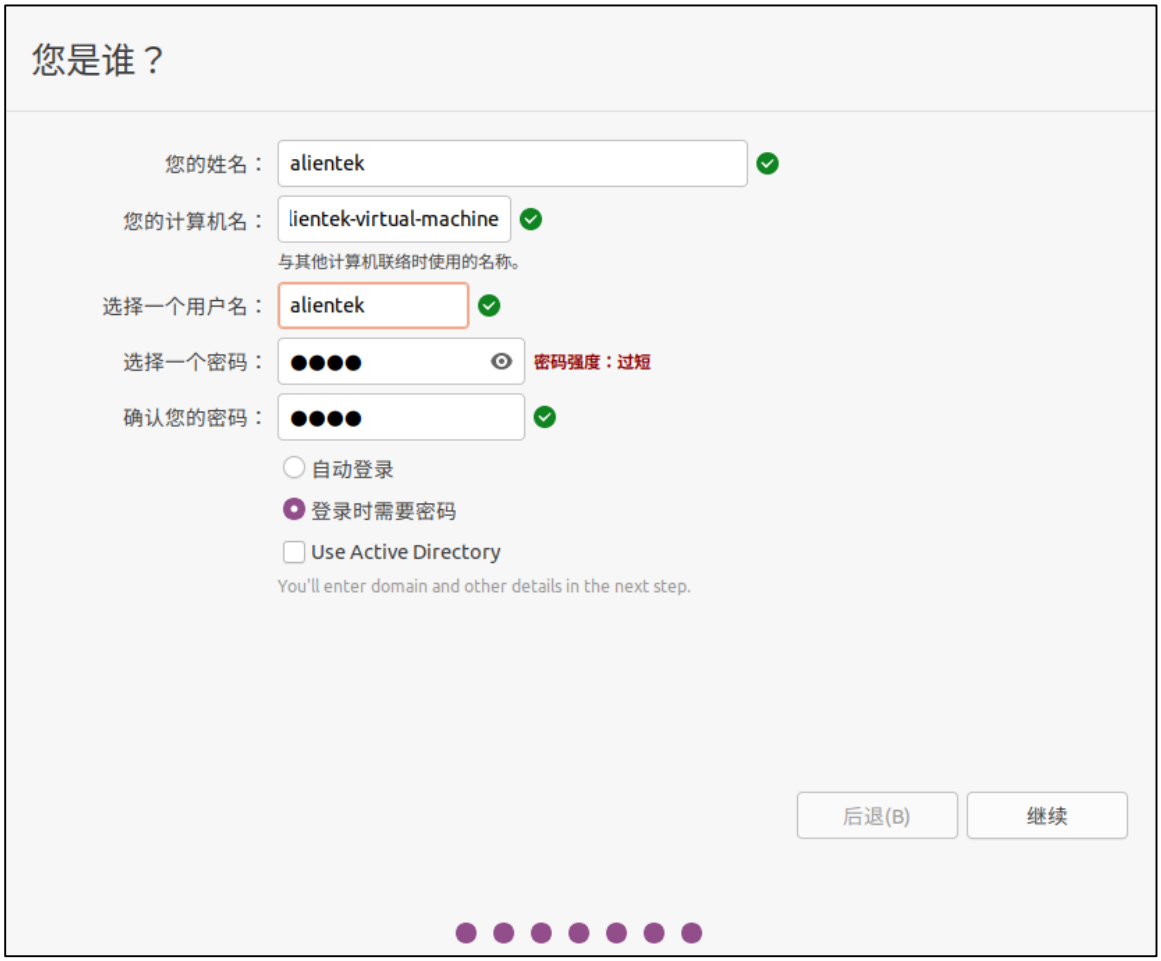


图 2.2.12 设置用户名和密码

设置好用户名和密码(密码强度提示可以不用管)以后点击“继续”按钮，系统就会开始正式安装，如图 2.2.13 所示：

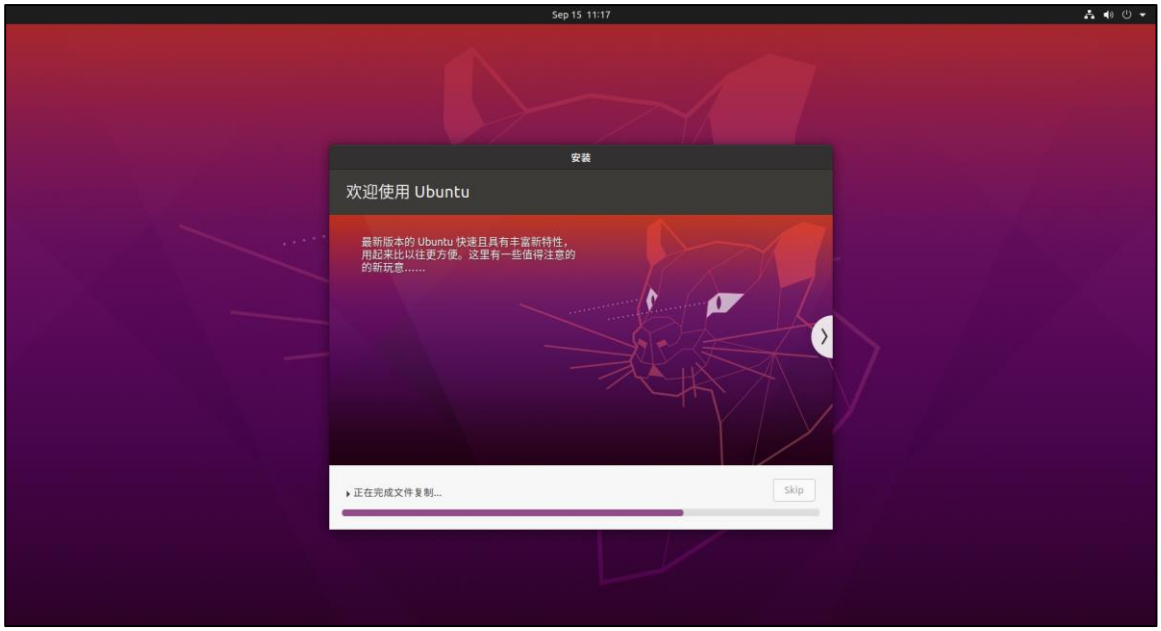


图 2.2.13 系统安装中

等待系统安装完成，安装过程中会下载一些文件，所以一定要保证电脑能够正常上网，如果不能正常上网的话可以点击右侧的“skip”按钮来跳过下载文件这个步骤，对于系统的安装没有任何影响，安装完成以后提示重启系统，如图 2.2.14 所示：

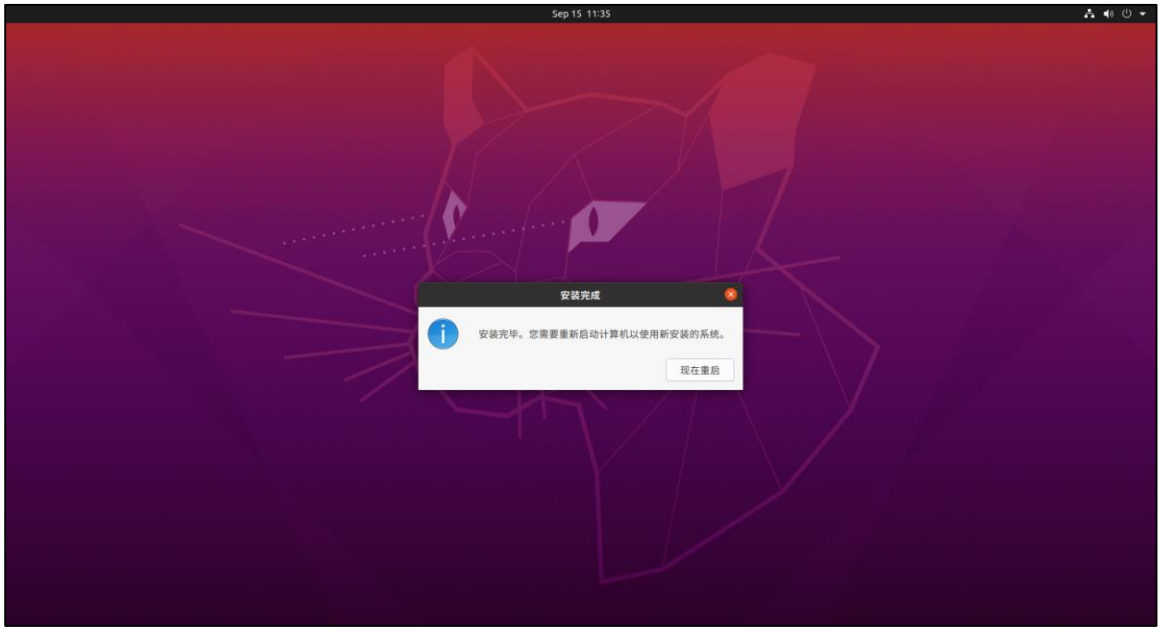


图 2.2.14 安装完成，重启系统

重启的时候需要弹出虚拟机里面的系统镜像！关闭 Ubuntu 操作系统，打开 VMware 的虚拟机设置界面，然后选中“CD/DVD(SATA)”，右侧的“连接”选择“使用物理驱动器”，如图 1.3.2.15 所示。

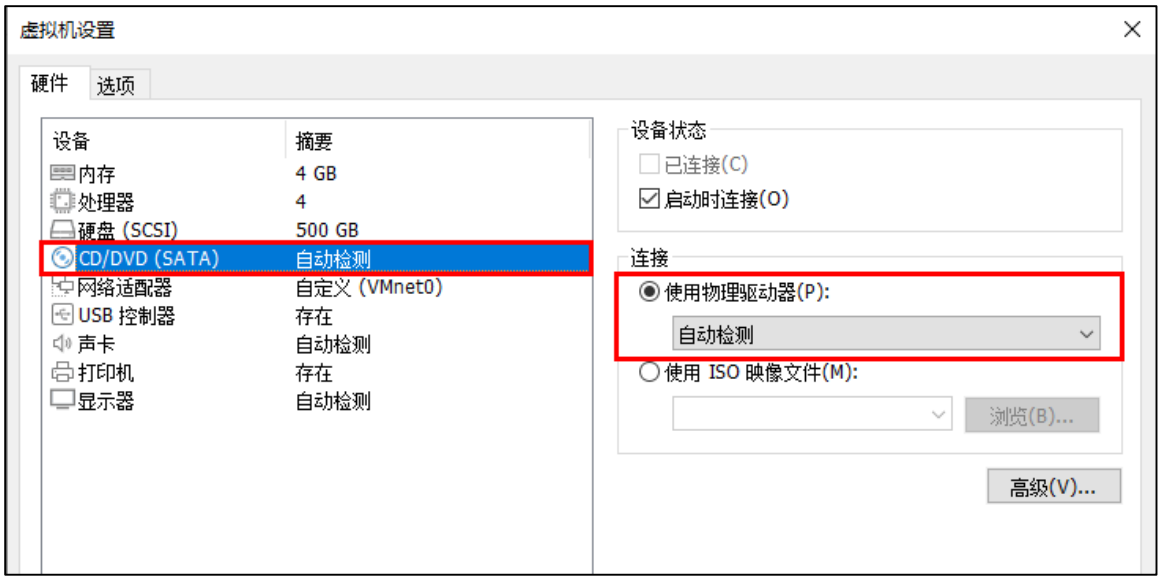


图 2.2.15 弹出 Ubuntu 系统镜像

设置好以后点击“确定”按钮，然后重新打开虚拟机，启动 Ubuntu，系统启动以后就会提示输入密码，如图 2.2.16 所示：

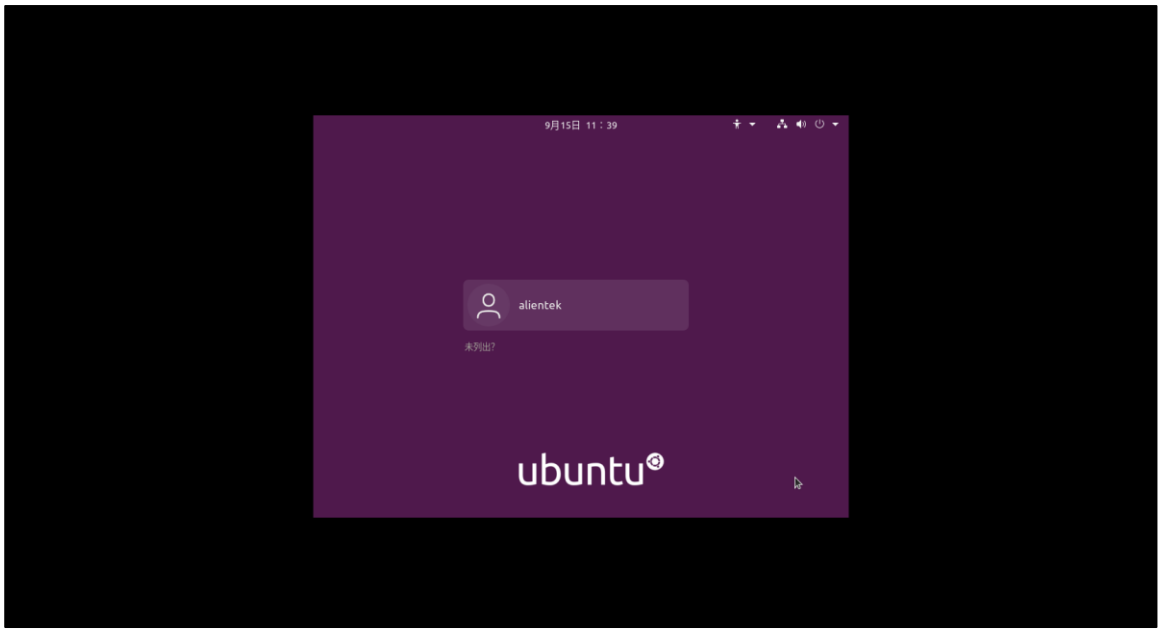


图 2.2.16 系统登录界面

在图 2.2.16 中输入密码，点击键盘上的回车就会进入系统主界面，系统界面如图 2.2.17 所示：



图 2.2.17 系统桌面

图 2.2.17 就是第一次进入系统的系统桌面，第一次进入系统会有“在线账户的界面”，也就是给 Ubuntu 系统的简易引导指南，点击图 2.2.17 右上角的“前进(N)”来一步步观看指南。最终的系统主界面如图 2.2.18 所示：

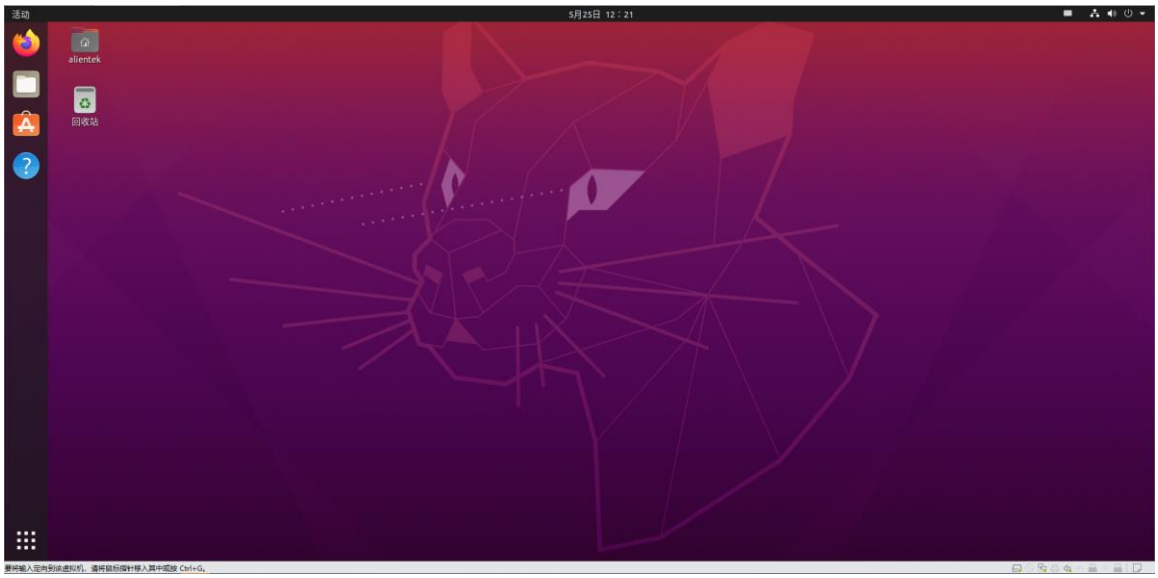


图 2.2.18 Ubuntu 系统主界面

2.3 VMware tools 安装

注意：新版本的 VMware 会自动安装的 VMware tools，如何测试 VMware tools 呢？在 Windows 系统里复制一段话，能粘贴到 Ubuntu 系统里终端里，说明 VMware tools 已经安装了。没有安装的请参考本小节。

打开虚拟机中的 Ubuntu 系统，然后点击：Vmware 的虚拟机(M)->安装 VMware Tools(T)……，如图 2.3.1 所示：



图 2.3.1 安装“VMware Tools”

点击图 2.3.1 中的“安装 WMware Tools”，此时就会在 Ubuntu 系统中自动下载 VMware Tools 工具，下载完成以后会放到 Ubuntu 桌面上，如图 2.3.2 所示：

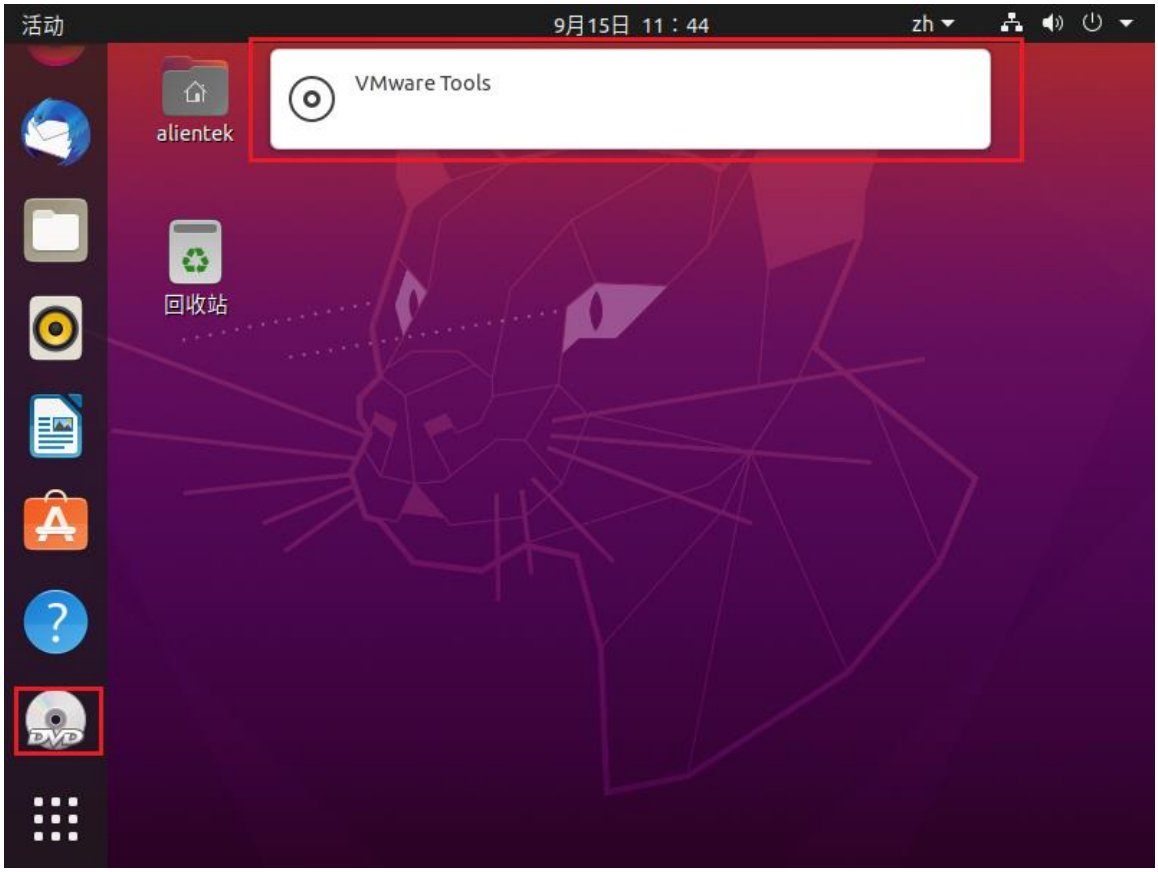


图 2.3.2 下载得到的 VMware Tools

双击图 2.3.2 中的光盘磁盘，打开以后如图 2.3.3 所示：

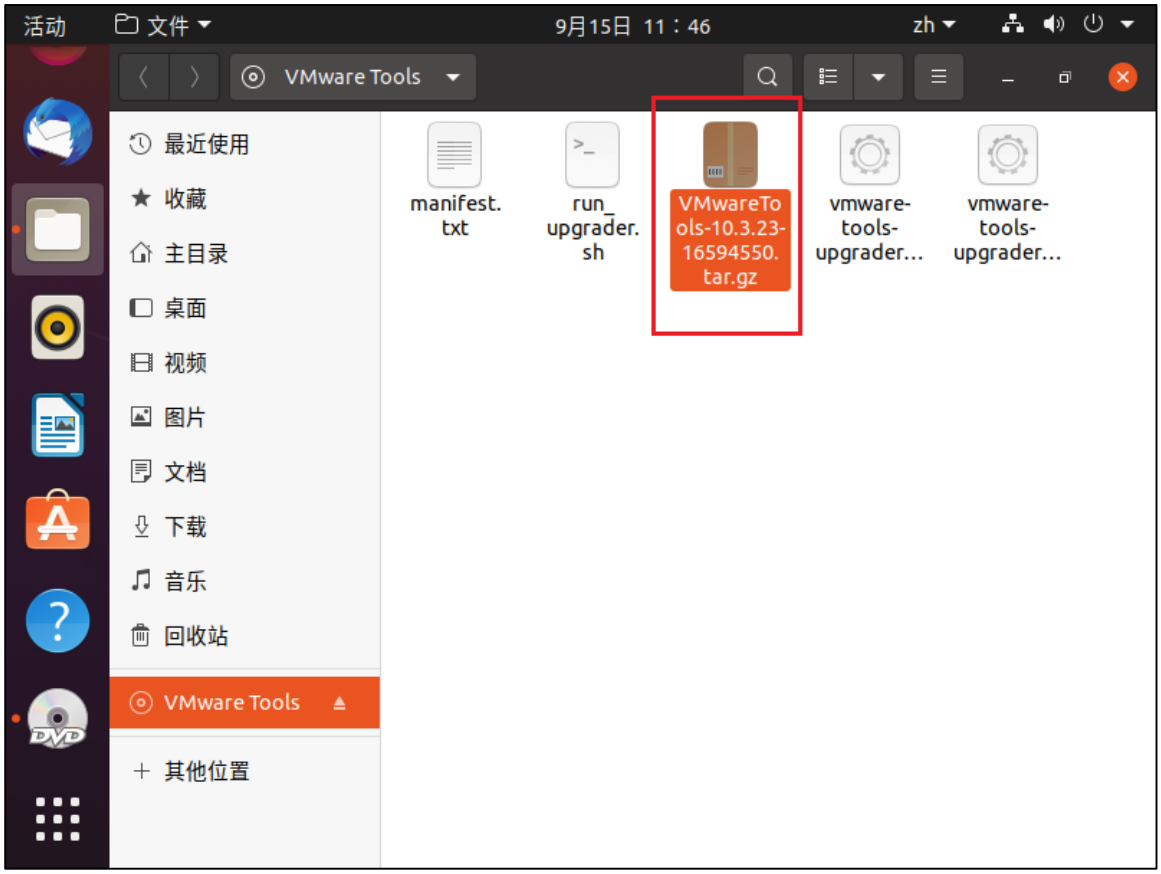


图 2.3.3 VMware Tools 安装包文件

图 2.3.3 中的 VMwareTools-10.3.23-16594550.tar.gz 就是我们要的安装包，将其解压到桌面上，如图 2.3.4 所示：

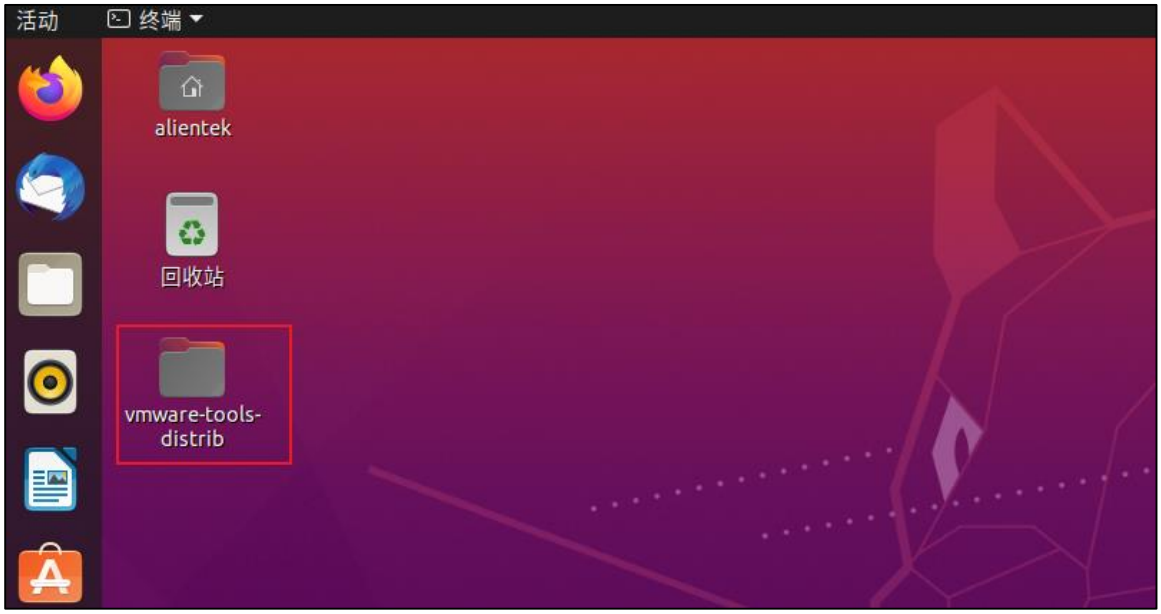


图 2.3.4 解压后的 VMwareTools

图 2.3.4 就是解压出来的 VMware Tools 安装压缩包，用终端进入此目录下，运行以下命令：

```
sudo ./vmware-install.pl                    #执行安装软件
```

安装的过程中会弹出一系列的操作提示，如果遇到选择“是”或“否”的这样的问题，输入“yes”表示是，输入“no”或者直接按回车表示“否”。如果是哪些安装路径的问题直接按回车键，使用默认路径即可，其他类型的问题大家根据实际提示做选择。安装完成以后如图 2.3.5 所示：

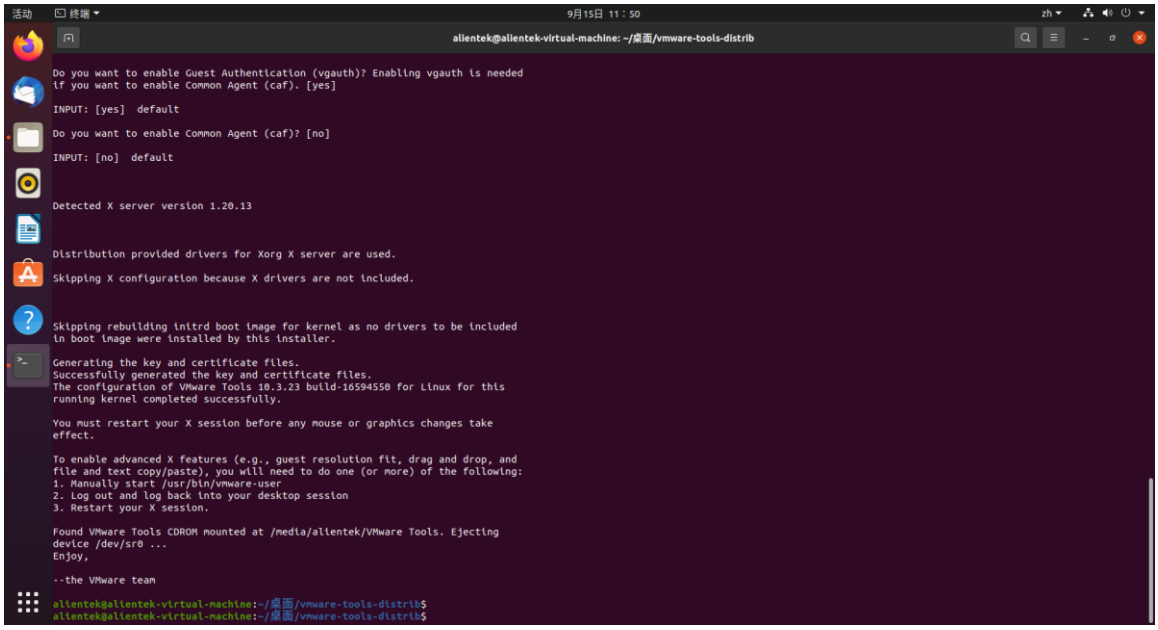


图 2.3.5 VMware Tools 安装完成

VMware Tools 安装完成以后重启 Ubuntu，重启以后就可以直接在虚拟机 Ubuntu 系统和主机 Windows 下进行文字、文件等的复制粘贴。如果还是不能复制的请运行以下命令：

```
sudo apt-get autoremove open-vm-tools#卸载已有的工具
sudo apt-get install open-vm-tools        #安装工具 open-vm-tools
sudo apt-get install open-vm-tools-desktop    #安装 open-vm-tools-desktop
```

接着重启系统即可。

第三章 Ubuntu 开发环境搭建

本章主要讲以下内容

3.1 Ubuntu 换源

3.2 Ubuntu 和 Windows 文件互传

3.3 Visual Studio Code 的安装和使用

3.1 Ubuntu 换源

在上章节里面我们已经安装好 Ubuntu，此时的 Ubuntu 还是不能做开发的，因为还有很多环境和软件没有安装，所以要先安装环境，这里笔者已经把所有坑都填完了。跟着下面一步一步走就行了。

- 先设置 Ubuntu 的源

国内的环境下使用 Ubuntu 官方的默认源下载很慢，有一些包不能安装，所以我们要设置合适的源。Ubuntu 官方更换源有一个很智能的操作，可以根据自己的网络位置设配合适的源，设置如下步骤：

打开设置，在左边设置栏里面找到“关于”如下图所示：

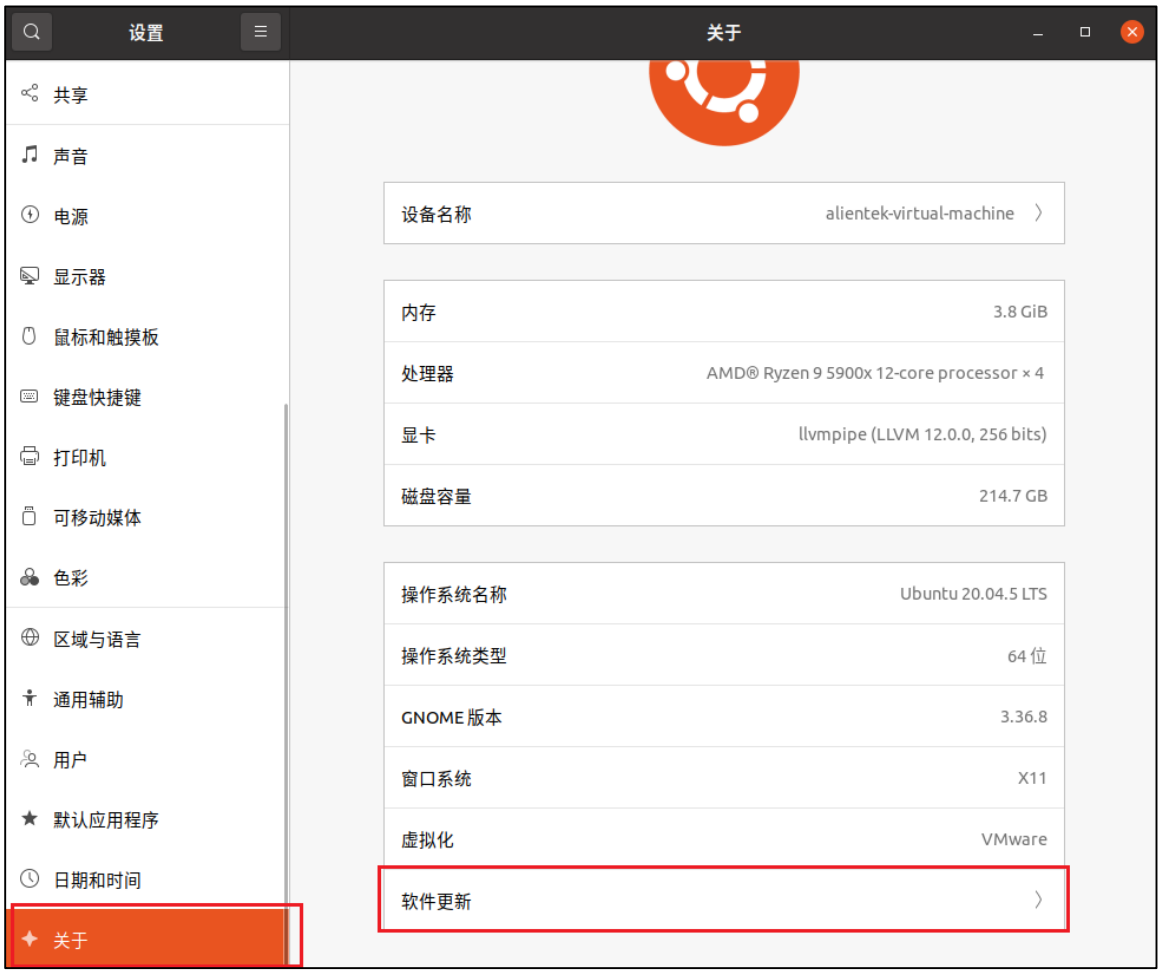


图 3.1.1 关于设置

我们找到“关于”设置后，右边栏最后面有一个软件更新设置，接下来点击“软件设置”，弹出如下界面：



图 3.1.2 软件更新图

图 3.1.2 可以看出是下载软件是在“位于中国的服务器”，此时的源链接还是不能配置开发环境，展开红色框里面，选择“其它”如下图所示：

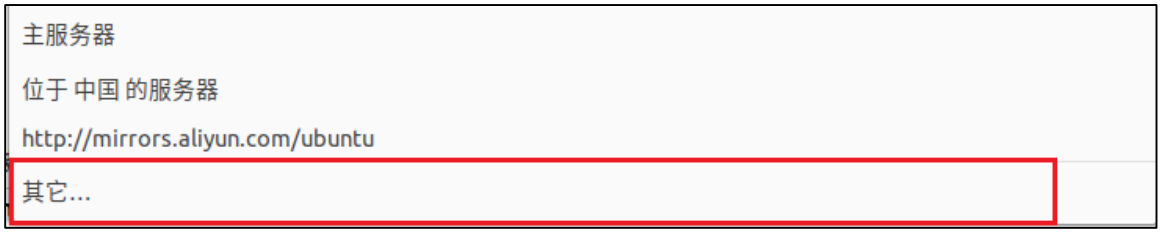


图 3.1.3 源服务器选择

点击成功就会出现如下图所示：

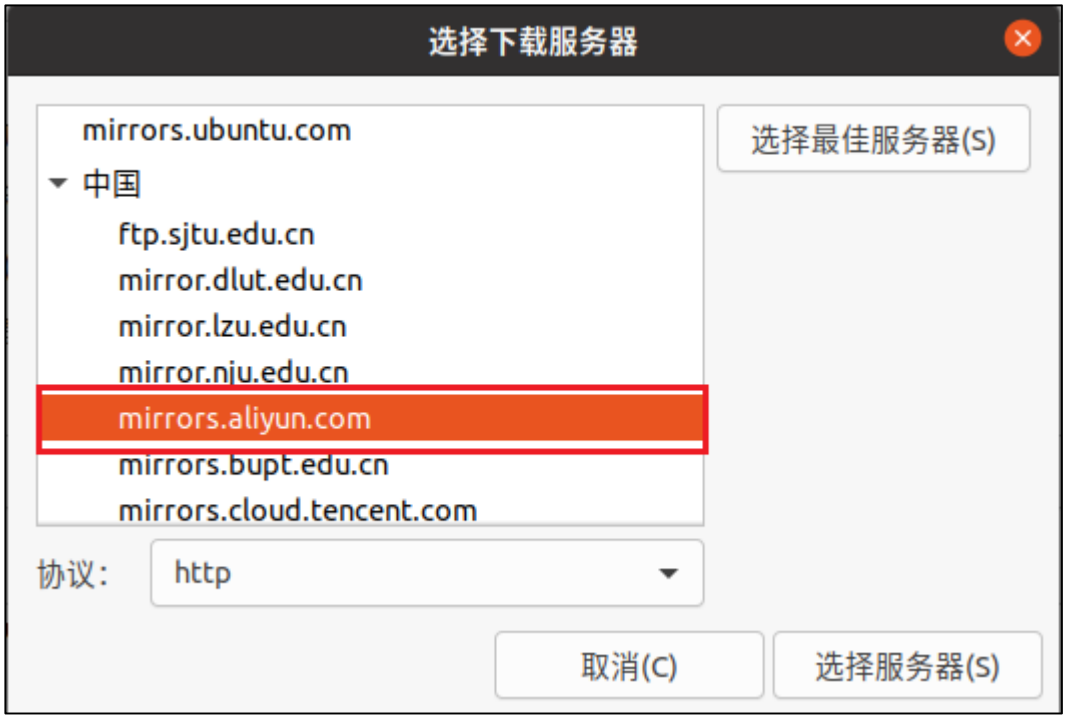


图 3.1.4 选择下载服务器

从图 3.1.4 中可以看出来，有很多服务器的选择，在国内最好选择阿里云源最合适。选择 mirrosr.aliyun.com 为阿里云源。我们也可以根据自己的网络匹配最合适的源点击“选择最佳服务器”就可以根据网络合适的源。输入密码就可以直接更新源。命令如下：

```
sudo apt update          #更新软件源列表
```

3.2 Ubuntu 和 Windows 文件互传

在开发的过程中会频繁的在 Windows 和 Ubuntu 下进行文件传输，比如在 Windwos 下进行代码编写，然后将编写好的代码拿到 Ubuntu 下进行编译。Windows 和 Ubuntu 下的文件互传我们需要使用 FTP 服务(安装 vmware tools 后可以直接拷贝文件，这种方法不推荐使用)，设置方法如下：

1、开启 Ubuntu 下的 FTP 服务

打开 Ubuntu 的终端窗口，然后执行如下命令来安装 FTP 服务：

```
sudo apt-get install vsftpd
```

等待软件自动安装，安装完成以后使用如下 VI 命令打开/etc/vsftpd.conf，命令如下：

```
sudo vi /etc/vsftpd.conf
```

打开以后 vsftpd.conf 文件以后找到如下两行：

```
local_enable=YES
write_enable=YES
```

确保上面两行前面没有“#”，有的话就取消掉，完成以后如图 2.5.1 所示：

```
27 # Uncomment this to allow local users to log in.
28 local_enable=YES
29 #
30 # Uncomment this to enable any form of FTP write command.
31 write_enable=YES
```

图 3.2.1 vsftpd.conf 修改

修改完 vsftpd.conf 以后保存退出，使用如下命令重启 FTP 服务：

```
sudo /etc/init.d/vsftpd restart
```

2、Windows 下 FTP 客户端安装

Windows 下 FTP 客户端我们使用 FileZilla，这是个免费的 FTP 客户端软件，可以在 FileZilla 官网下载，下载地址如下：https://www.filezilla.cn/download/client，下载界面如图 2.5.2 所示：

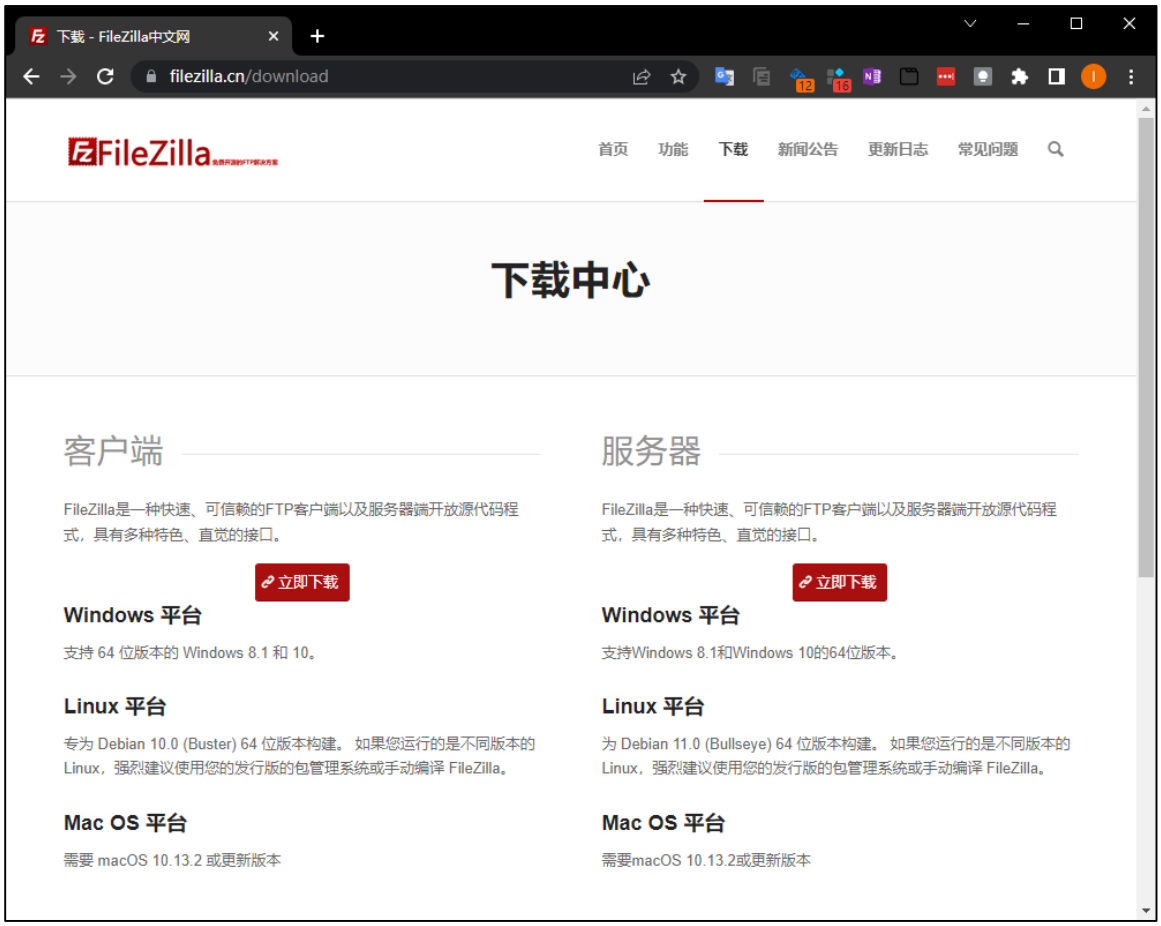


图 3.2.2 FileZilla 软件下载

如果是 32 位电脑就选择 32 位版本（32 位电脑基本无市场了），64 位电脑就选择 64 位版本，我们已经下载好了 64 位版本的 FileZilla 并放到开发板光盘中了，路径为: 开发板光盘 A-基础资料→4、软件→ FileZilla_3.60.1_win64-setup.exe（请自行确认提供的基础资料下的 FileZilla 版本），双击安装即可。安装完成以后找到安装目录，找到图标，然后发送图标快捷方式到桌面，完成以后如图 3.2.3 所示：



图 3.2.3 FileZilla 图标

打开 FileZilla 软件，界面如图 3.2.4 所示：

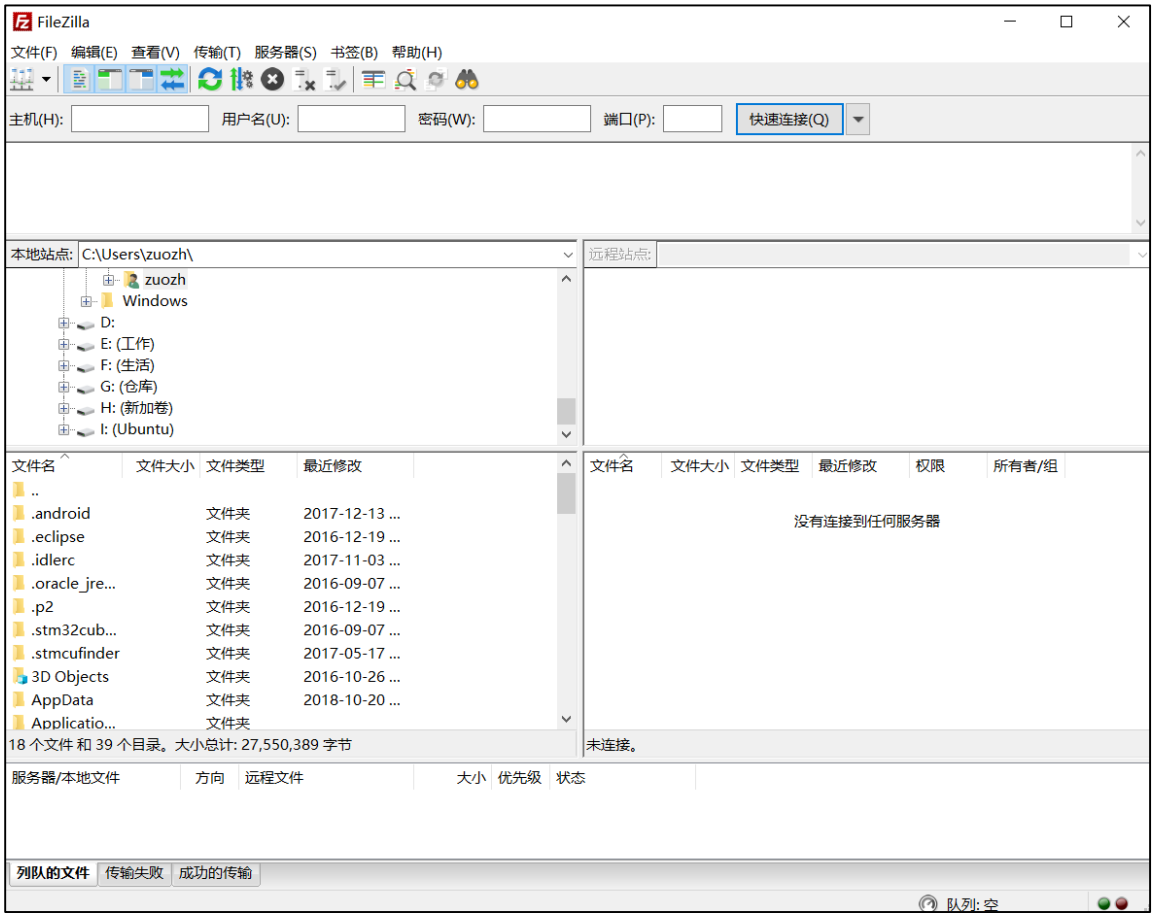


图 3.2.4 FileZilla 软件界面

3、FileZilla 软件设置

Ubuntu 作为 FTP 服务器，FileZilla 作为 FTP 客户端，客户端肯定要连接到服务器上，打开站点管理器，点击：文件->站点管理器，打开以后如图 3.2.5 所示：



图 3.2.5 站点管理器

点击图 3.2.5 中的“新站点(N)”按钮来创建站点，新建站点以后就会在“我的站点”下出现新建的这个站点，站点的名称可以自行修改，比如我将新的站点命名为“Ubuntu”如图 3.2.6 所示：

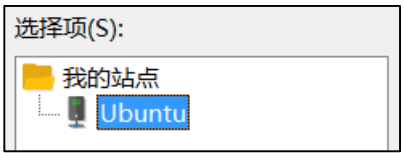


图 3.2.6 新建站点

选中新创建的“Ubuntu”站点，然后对站点的“常规”进行设置，设置如图 3.2.7 所示：



图 3.2.7 站点设置

按照图 3.2.7 中设置好以后，点击“连接”按钮，第一次连接可能会弹出提示是否保存密码的对话框，点击确定即可。连接成功以后如图 3.2.8 所示：

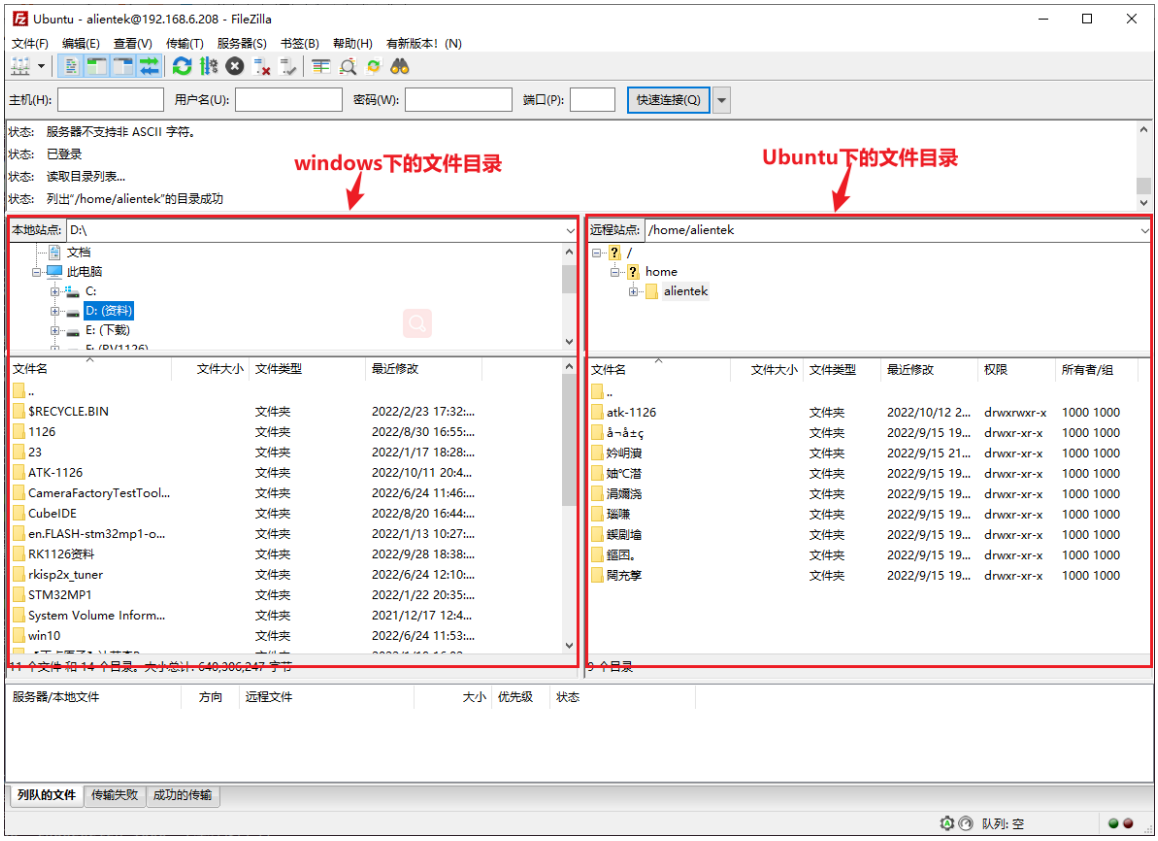


图 3.2.8 连接成功

连接成功以后如图 3.2.8 所示，其中左边就是 Windows 文件目录，右边是 Ubuntu 文件目录，默认进入用户根目录下（比如我电脑的“/home/alientek”）。但是注意观察在图 3.2.8 中 Ubuntu 文件目录下的中文目录都是乱码的，这是因为编码方式没有选对，先断开连接，点击：服务器(S)->断开连接，然后打开站点管理器，选中要设置的站点“Ubuntu”，选择“字符集”，设置如图 3.2.9 所示：



图 3.2.9 设置字符集

按照图 3.2.9 设置好字符集以后重新连接到 FTP 服务器上，重新链接到 FTP 服务器以后 Ubuntu 下的文件目录中文显示就正常了，如图 3.2.10 所示：

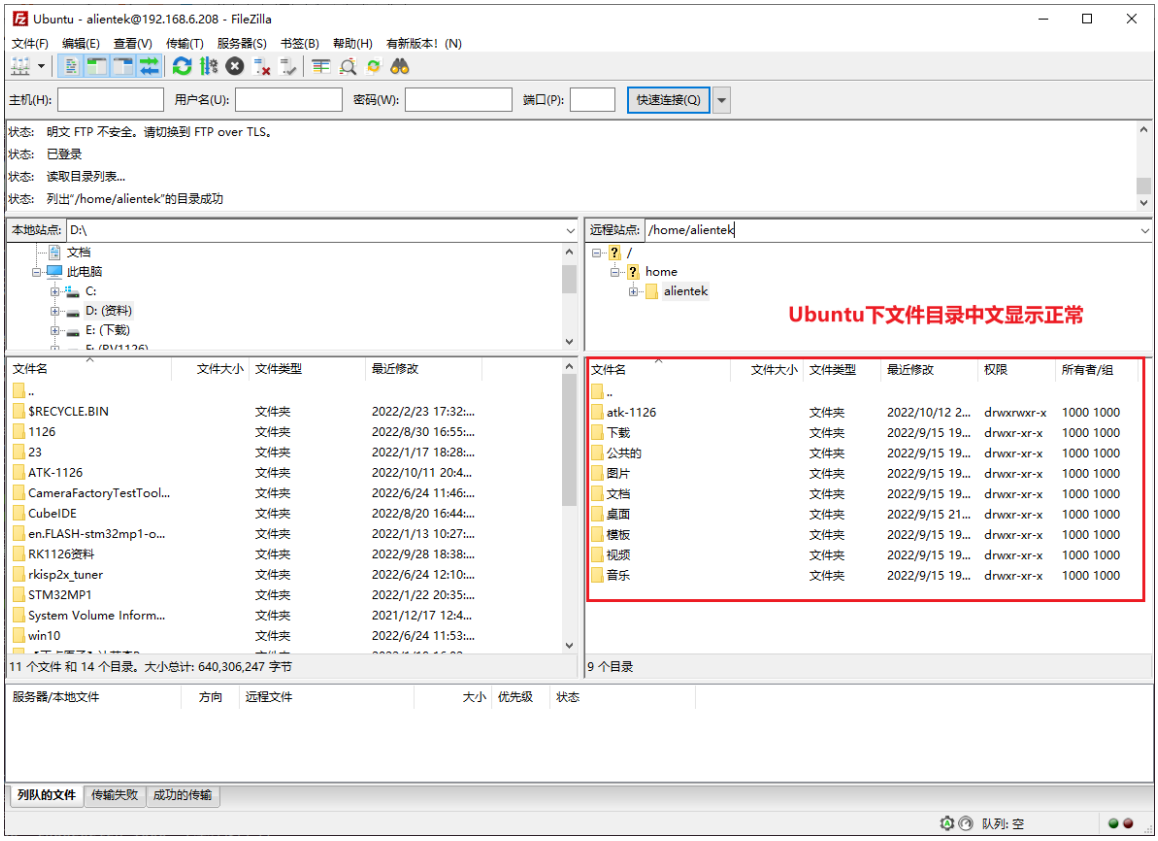


图 3.2.10 Ubuntu 下文件目录中文显示正常

如果要将 Windows 下的文件或文件夹拷贝到 Ubuntu 中，只需要在图 3.2.10 中左侧的 Windows 区域选中要拷贝的文件或者文件夹，然后直接拖到右侧的 Ubuntu 中指定的目录即可。将 Ubuntu 中的文件或者文件夹拷贝到 Windows 中也是直接拖放。

3.3 Visual Studio Code 软件的安装和使用

3.3.1 Visual Studio Code 的安装和卸载

Visual Stuio Code 是一个编辑器，可以用来编写代码，Visual Studio Sode 本教程以后就简称为 VSCode，VSCode 是微软出的一款免费编辑器。VSCode 有 Windows、Linux 和 macOS 三个版本的，是一个跨平台的编辑器。VSCode 下载地址是：https://code.visualstudio.com/Download，下载界面如图 3.3.1 所示：

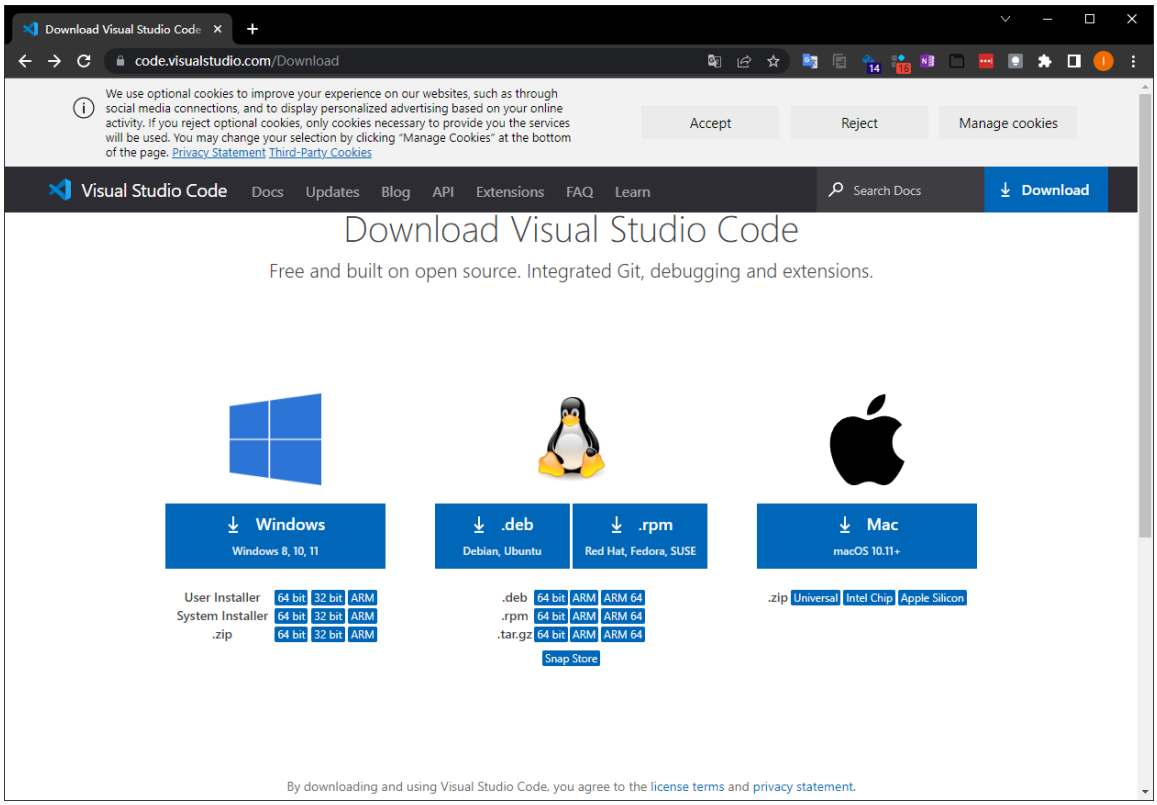


图 3.3.1.1 VSCode 下载界面

在图 3.3.1.1 中下载自己想要的版本，本教程需要 Windows 和 Linux 这两个版本，所以下载这两个即可，我们已经下载好并放入了开发板光盘中，路径为：开发板光盘 A-基础资料→4、软件→Visual Studio Code。

1、Windows 版本安装

Windows 版本的安装和容易，和其他 Windows 一样，Visual Studio Code 文件夹双击.exe 安装包，然后一路“下一步”即可，安装完成以后在桌面上就会有 VSCode 的图标，如图 3.3.1.2 所示：



图 3.3.1.2 VSCode 图标

双击图 3.3.1.2 打开 VSCode，默认界面如图 3.3.1.3 所示：

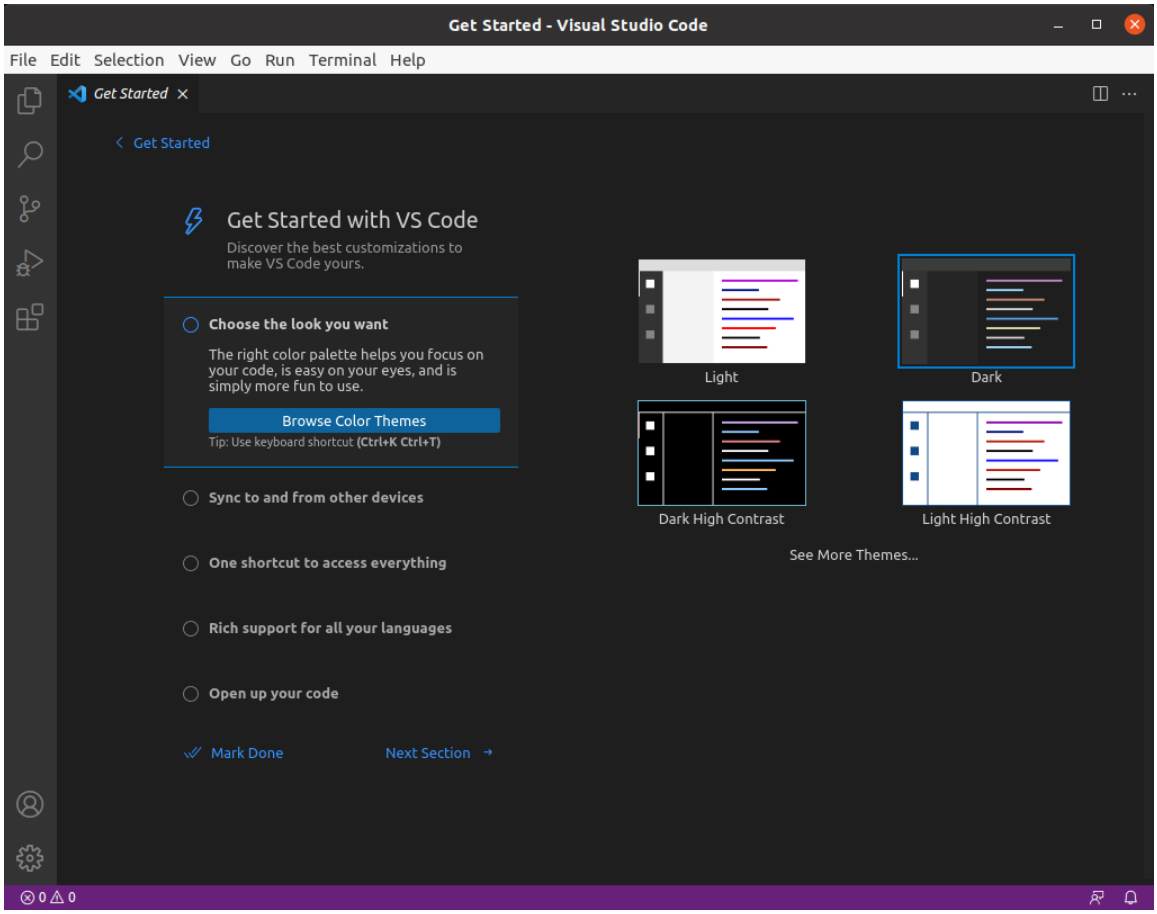


图 3.3.1.3 VSCode 默认界面

2、Linux 版本安装

我们有时候也需要在 Ubuntu 下阅读代码，所以还需要在 Ubuntu 下安装 VSCode。将开发板光盘 A-基础资料→4、软件→Visual Studio Code/code_1.72.1-1665423861_amd64.deb 软件包拷贝到 Ubuntu 系统中，然后使用如下命令安装：注意，这里的 code_1.72.1-1665423861_amd64.deb 可能会更新迭代，请根据各个开发光盘下提供的 VSCode 版本为准。

```
sudo dpkg -i code_1.72.1-1665423861_amd64.deb
```

等待安装完成，如图 3.3.1.4 所示：

```
alientek@alientek-virtual-machine:~$ sudo dpkg -i code_1.72.1-1665423861_amd64.deb
[sudo] alientek 的密码:
正在选中未选择的软件包 code。
(正在读取数据库 ... 系统当前共安装有 247655 个文件和目录。)
准备解压 code_1.72.1-1665423861_amd64.deb  ...
正在解压 code (1.72.1-1665423861) ...
正在设置 code (1.72.1-1665423861) ...
正在处理用于 gnome-menus (3.36.0-1ubuntu1) 的触发器 ...
正在处理用于 desktop-file-utils (0.24-1ubuntu3) 的触发器 ...
正在处理用于 mime-support (3.64ubuntu1) 的触发器 ...
正在处理用于 shared-mime-info (1.15-1) 的触发器 ...
alientek@alientek-virtual-machine:~$
```

图 3.3.1.4 VSCode 安装过程

安装完成以后在终端下运行“code”命令即可打开，如下所示：

```
code
```

结果如下图所示：

```
alientek@alientek-virtual-machine:~$
alientek@alientek-virtual-machine:~$ code
alientek@alientek-virtual-machine:~$
```

运行成功后会弹出如下所示:

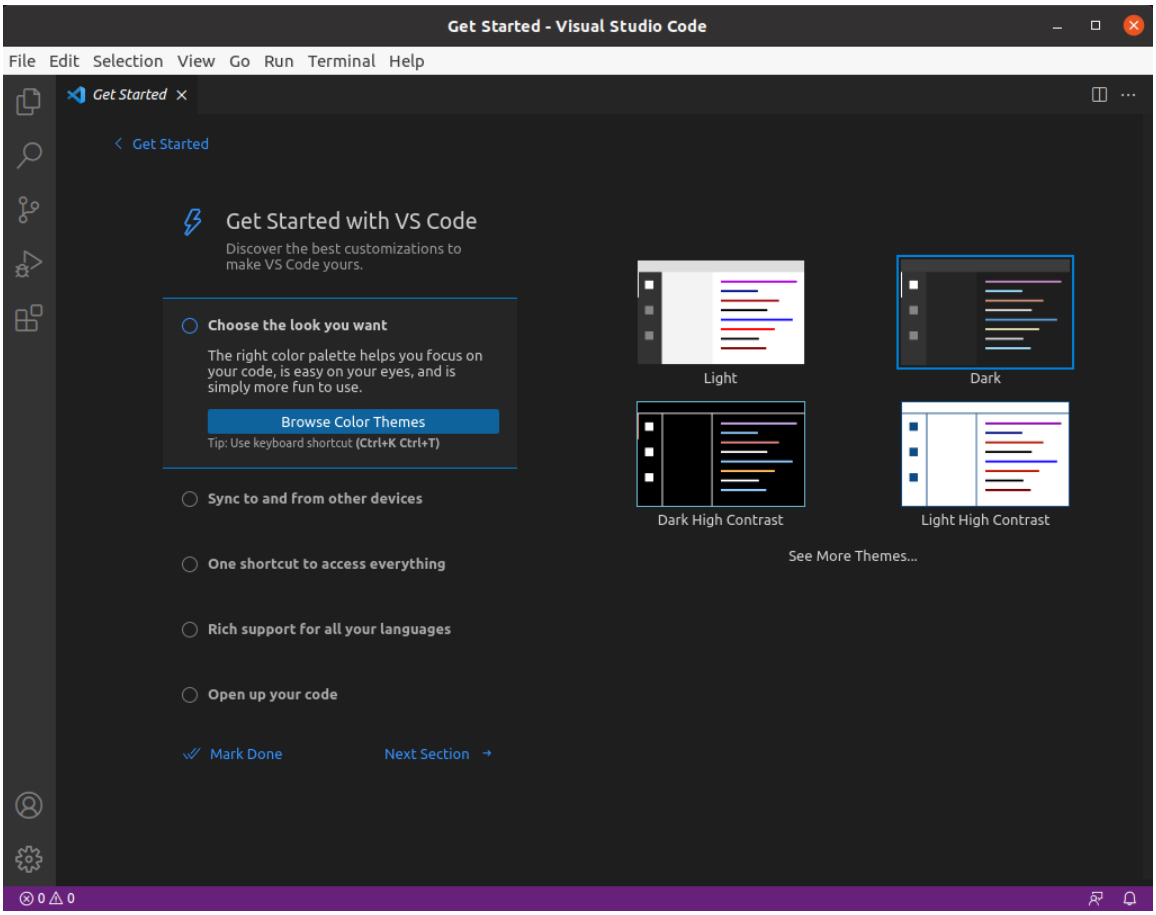


图 4.5.1.9 Linux 下的 VSCode

可以看出 Linux 下的 VSCode 和 Windows 下的基本是一样的，所以使用方法也是一样的。Windows 下卸载大家都会，使用电脑管家或者 360 都可以进行软件卸载。这里就不多说了。在 Ubuntu 下卸载只需要执行下面的指令，彻底清除，包括配置。

```
sudo apt-get --purge remove code
```

只是卸载，保留配置，下次需要用时再安装。

```
sudo apt-get remove code
```

3.3.2 Visual Studio Code 插件的安装

VSCode 支持多种语言，比如 C/C++、Python、C#等等，本教程我们主要用来编写 C/C++程序的，所以需要安装 C/C++的扩展包，扩展包安装很简单，如图 3.3.2.1 所示：

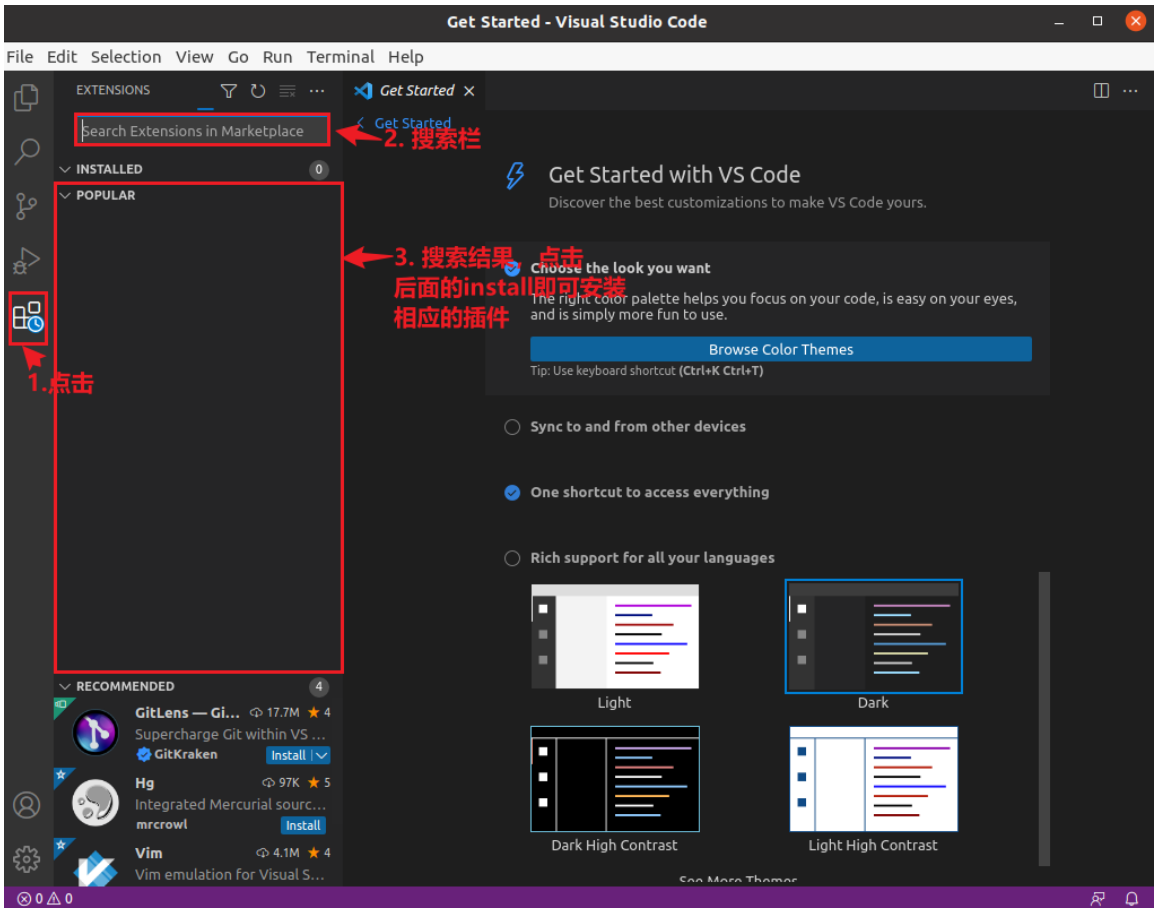


图 3.3.2.1 VSCode 插件安装

我们需要按照的插件有下面几个：

1)、C/C++，这个肯定是必须的。

2)、C/C++ Snippets，即 C/C++重用代码块。

3)、C/C++ Advanced Lint,即 C/C++静态检测 。

4)、Code Runner，即代码运行。

5)、Include AutoComplete，即自动头文件包含。

6)、Rainbow Brackets，彩虹花括号，有助于阅读代码。

7)、One Dark Pro，VSCode 的主题。

8)、GBKtoUTF8，将 GBK 转换为 UTF8。

9)、ARM，即支持 ARM 汇编语法高亮显示。

10)、Chinese(Simplified)，即中文环境。

11)、vscode-icons，VSCode 图标插件，主要是资源管理器下各个文件夹的图标。

12)、compareit，比较插件，可以用于比较两个文件的差异。

13)、DeviceTree，设备树语法插件。

14)、TabNine，一款 AI 自动补全插件，强烈推荐，谁用谁知道！

15)、Remote-SSH，可以远程连接到别的 vscode 上的软件。

完成以后重新打开 VSCode，如果要查看已经安装好的插件，可以按照图 3.3.2.2 所示方法查看：

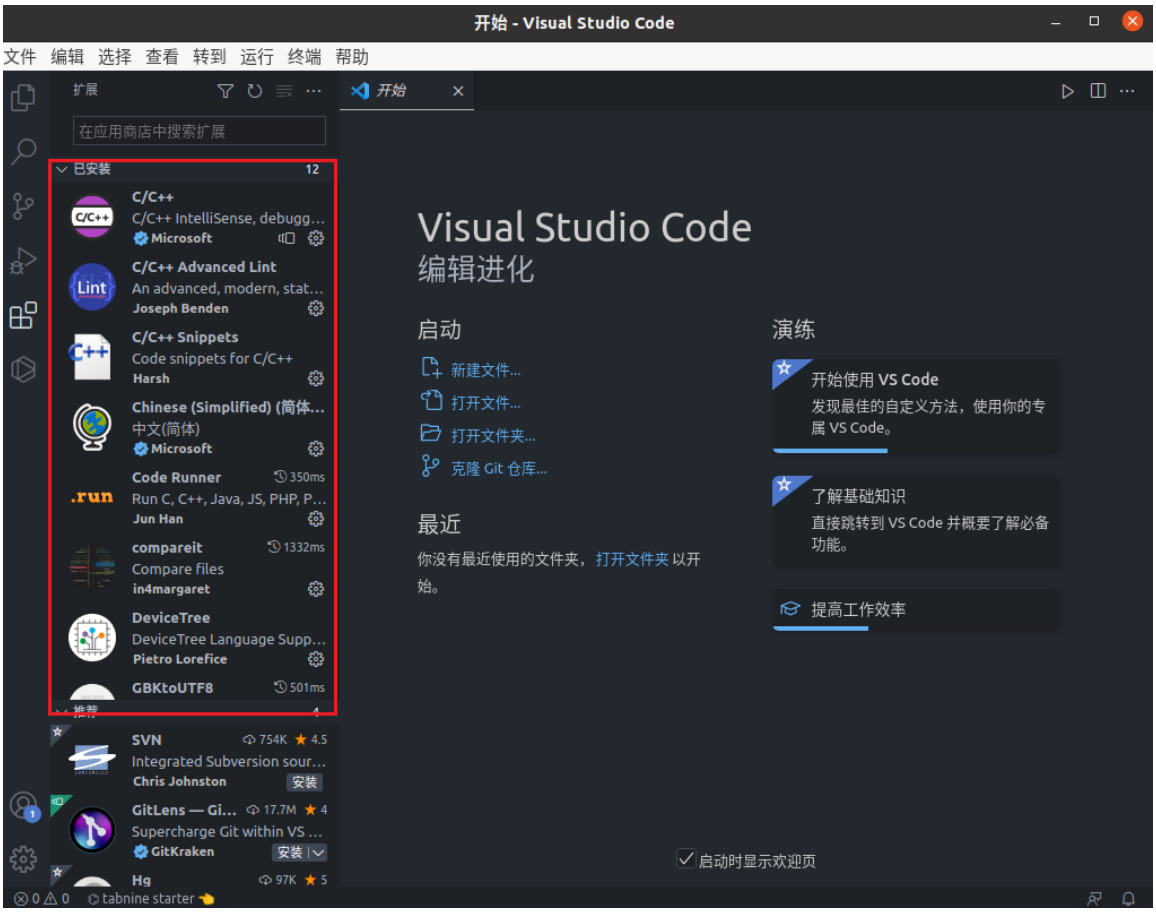


图 3.3.2.2 显示已安装的插件

3.3.3 vscode 远程 Ubuntu 系统

在开发的时候需要切换到 Ubuntu 系统，工作写文档的时候就切换回 Windows 系统，这样是比较很麻烦的，我们可以使用 vscode 的远程功能插件“Remote-SSH”，此插件可以进行远程开发 Ubuntu 下的代码。使用此插件前有本地和远程 Ubuntu 主机可以相互 ping 通(我们安装的是虚拟机，使用桥接网络即可实现)。同时在 Ubuntu 下检查 sshd 守护进程是否开启（指令为 `ps -aux | grep sshd`），如果没有开启，请在 Ubuntu 下输入指令安装 `sudo apt-get install openssh-server`。首先打开 Windows 下的 vscode。点击左下角绿色图标打开远程窗口，如下图所示：

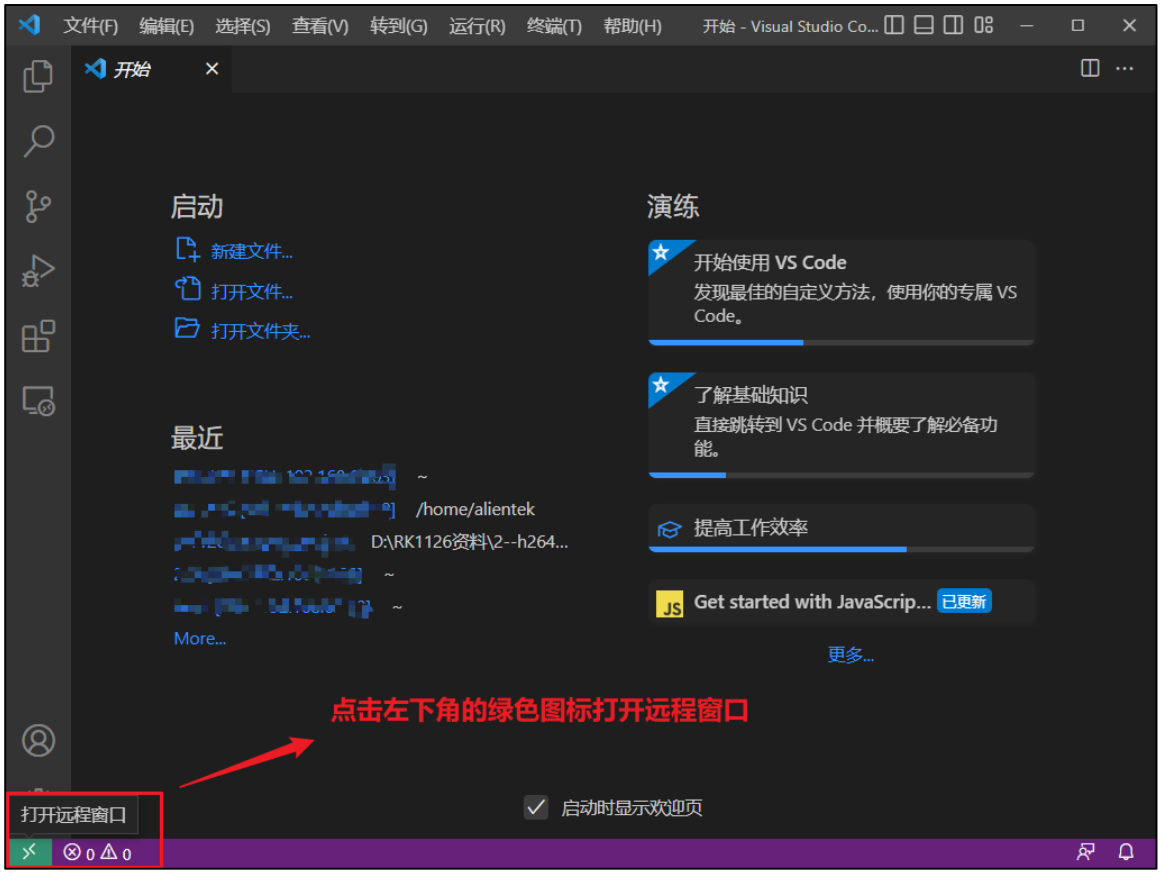


图 3.3.3.1 打开远程窗口

打开远程窗口，就会出现如下图所示：

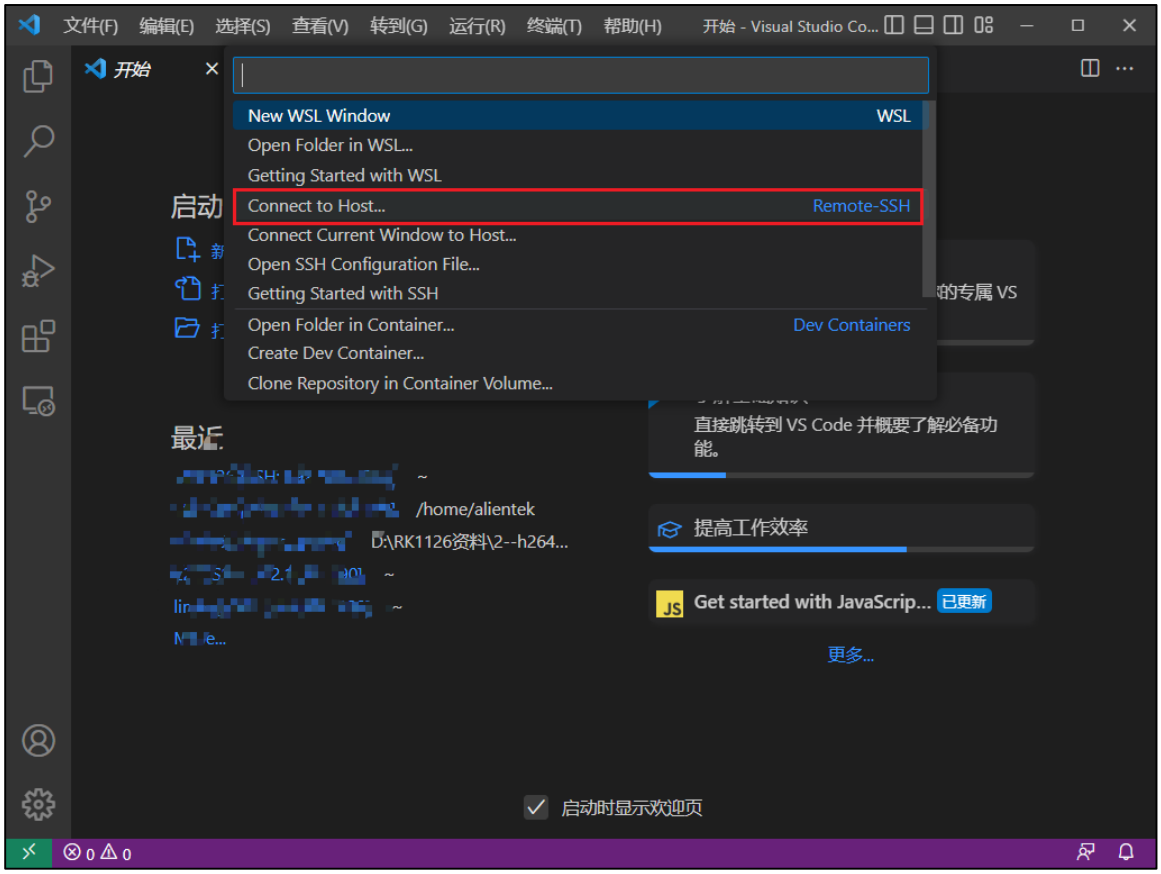


图 3.3.3.2 远程功能的选择

选择图 3.3.3.2 中的红色框“Connect to Host”功能，如下图所示：

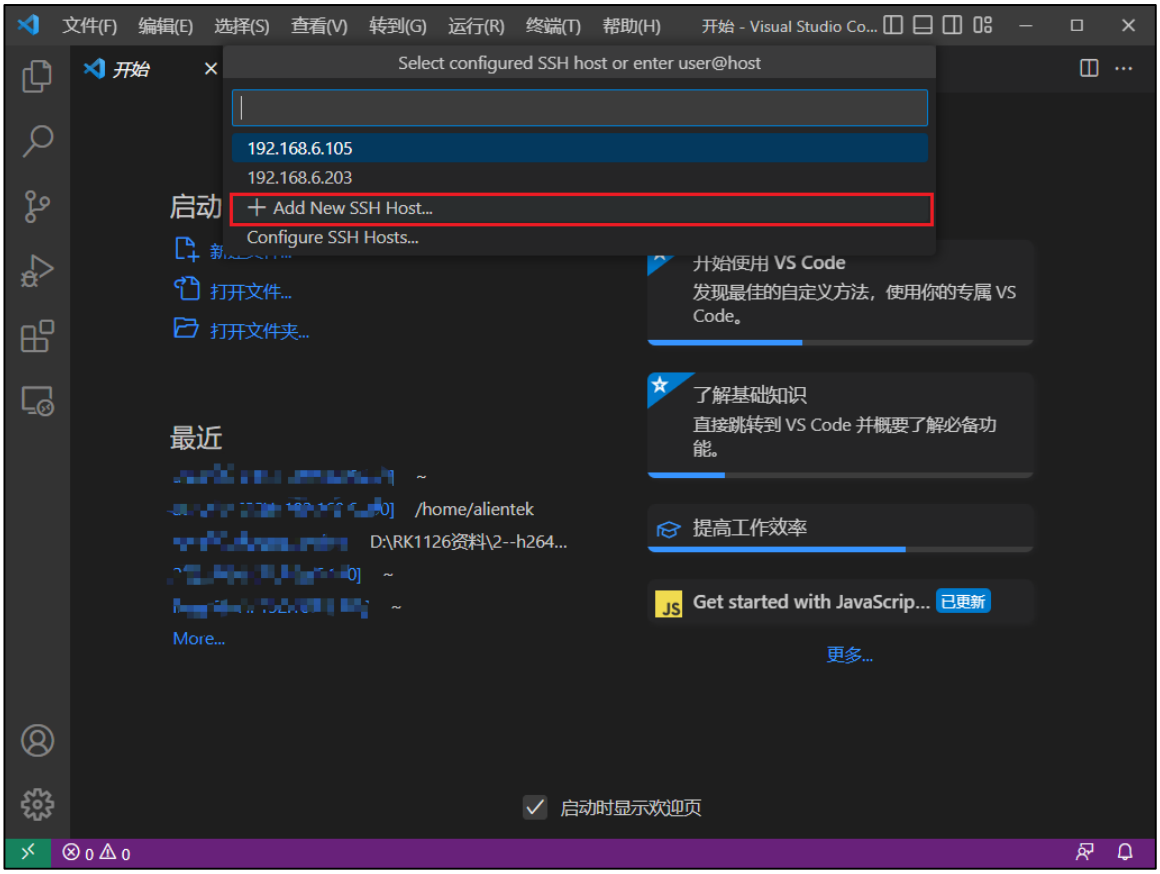


图 3.3.3.3 添加 SSH 的配置

选择图 3.3.3.3 中的红色框“Add New SSH Host”功能。如下图所示：

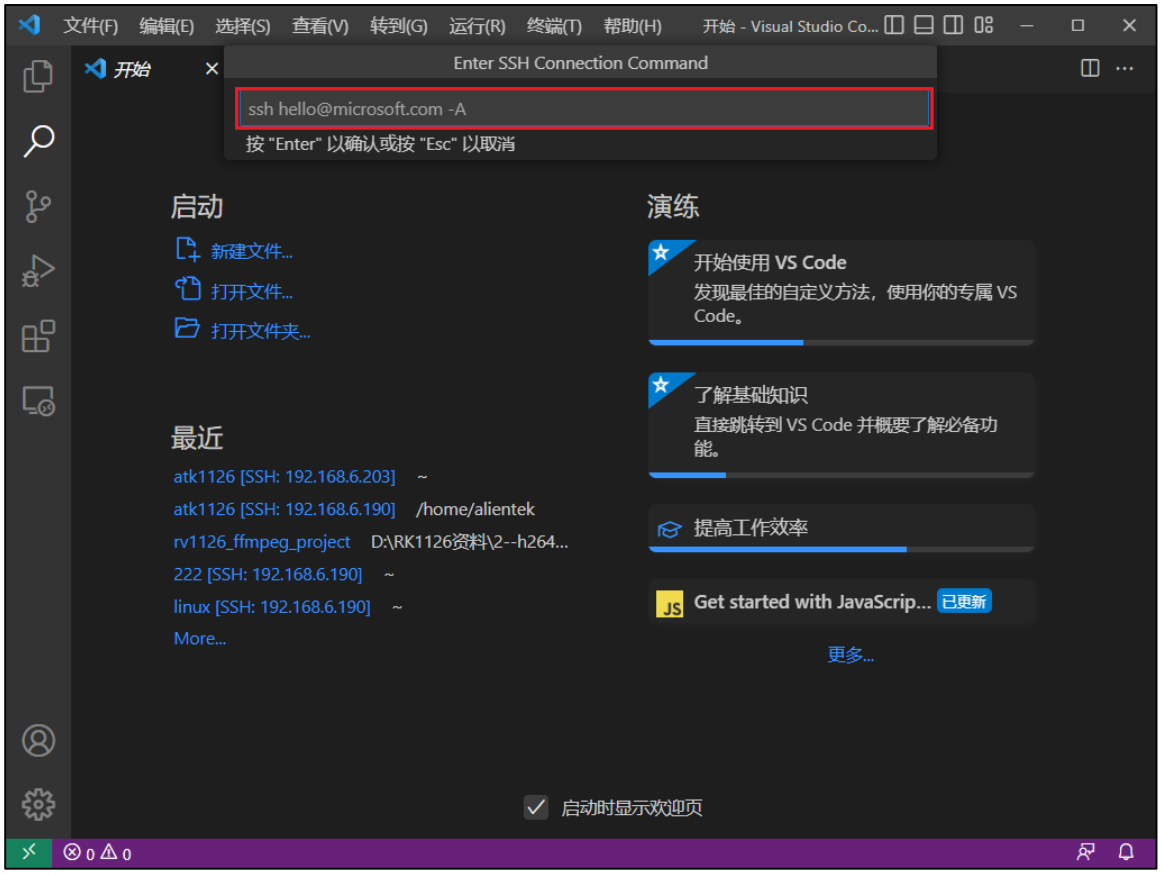


图 3.3.3.4 添加新的远程连接

根据图中的红色框提示信息输入远程电脑的用户名和 IP 地址，这边笔者的用户名和 IP 地址分别为：alientek 和 192.168.6.208。输入如下命令即可连接：

```
ssh alientek@192.168.6.208 -A
```

输入连接命令后，按回车键，如下图所示：

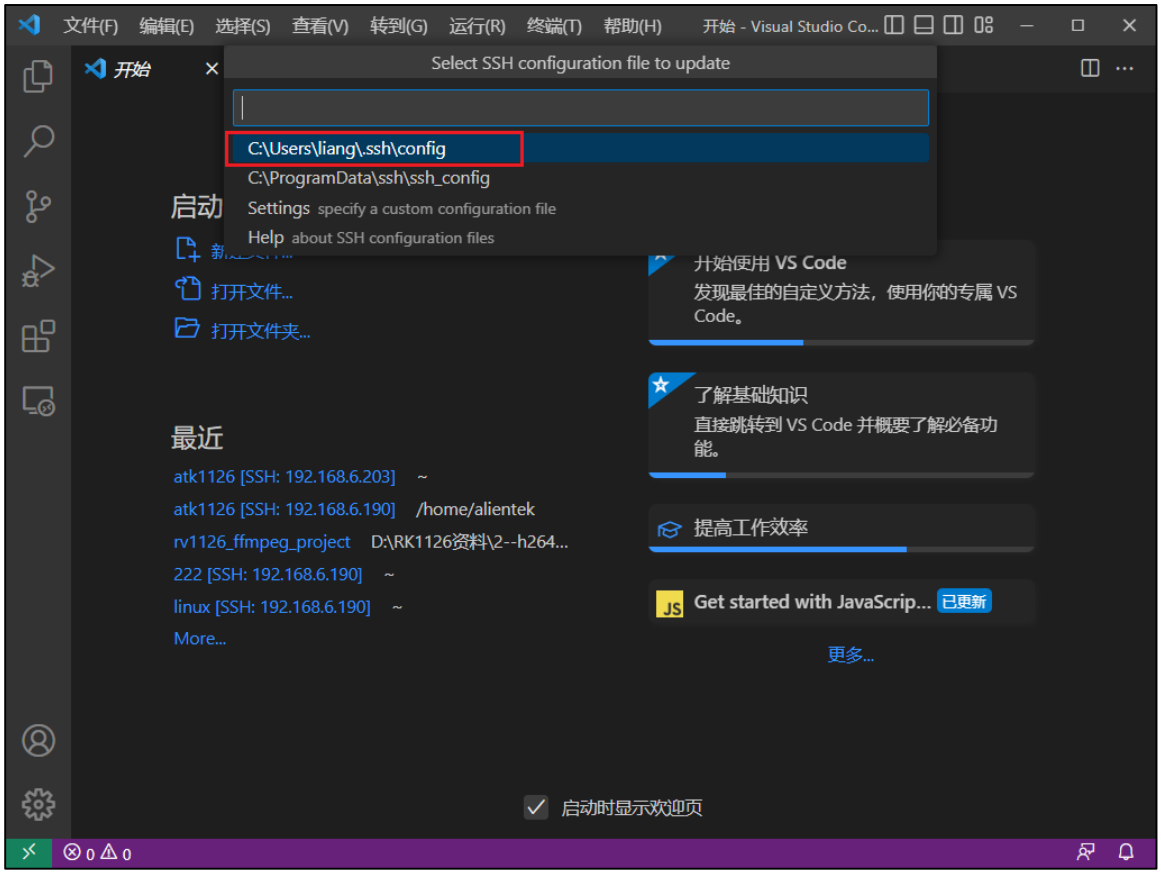


图 3.3.3.5 选择保存配置文件

图 3.3.3.5 中，主要是要保存刚刚输入的配置到那个文件下，通常选择红色框“C:\Users\PC用户名\.ssh\config”的路径文件。配置完成后，就会弹出如下图所示的小框：

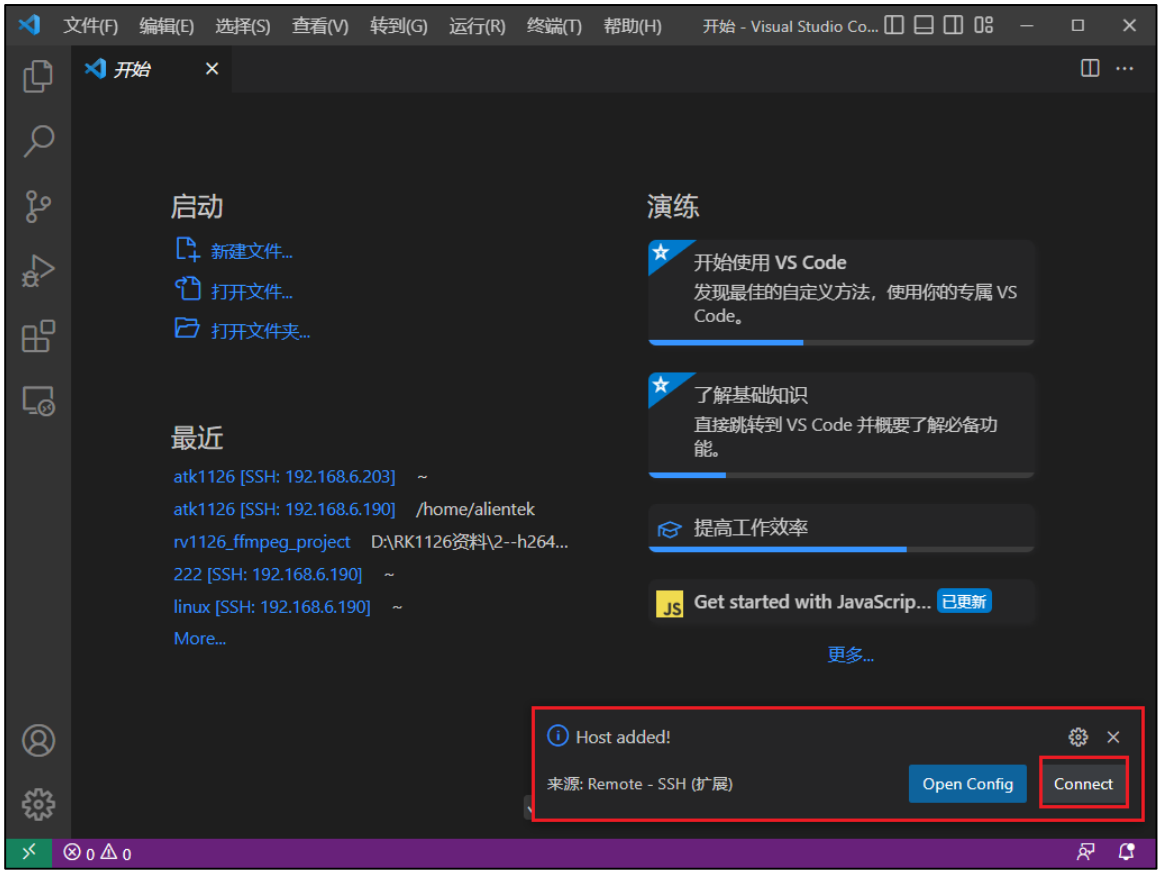


图 3.3.3.6 连接远程端的 vscode

点击图 3.3.3..6 中的右下角“Connect”，即可进入连接状态，如下图所示：

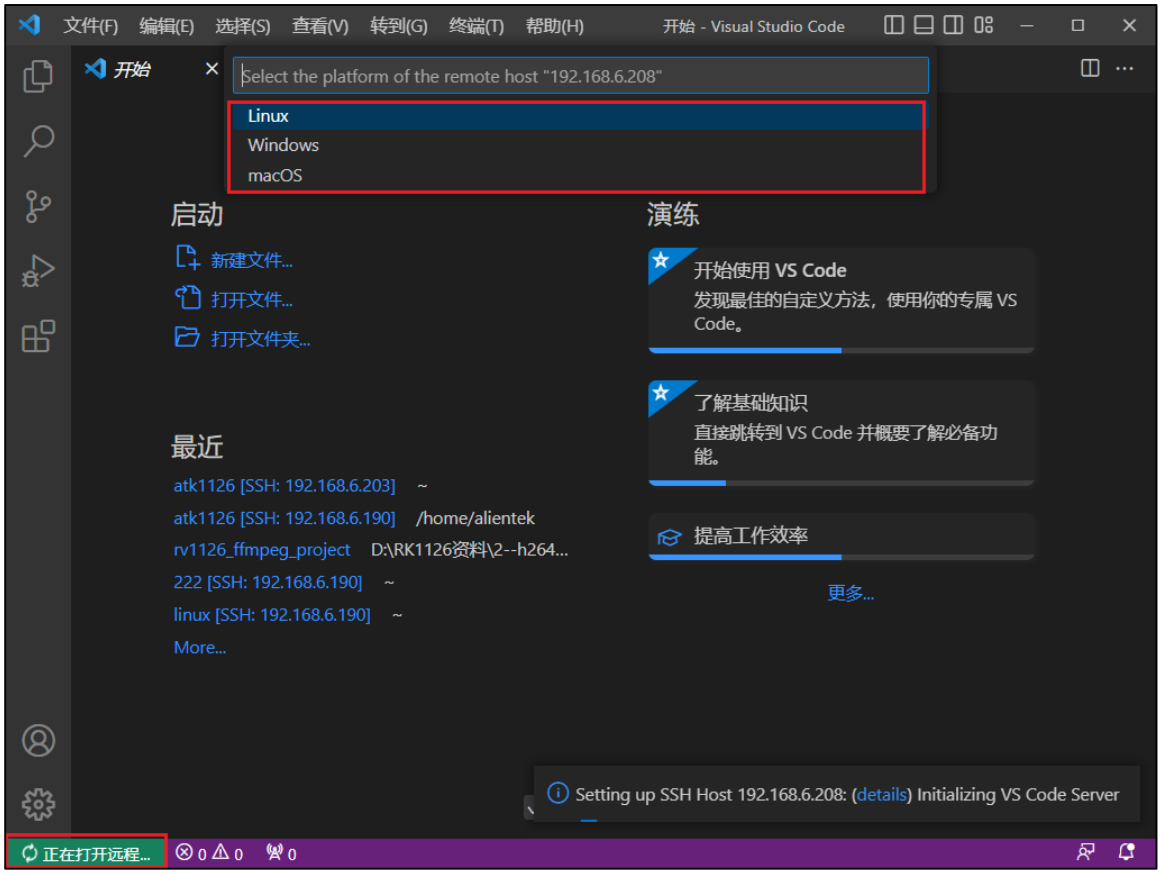


图 3.3.3.7 选择远程电脑的系统

图 3.3.3.7 中，开始进行远程连接配置，首先要选择远程电脑的系统是什么，这边我们连接的是 Ubuntu，所以选择“Linux”。左下角开始显示“正在打开远程”，选择完，就会出现如下图所示：

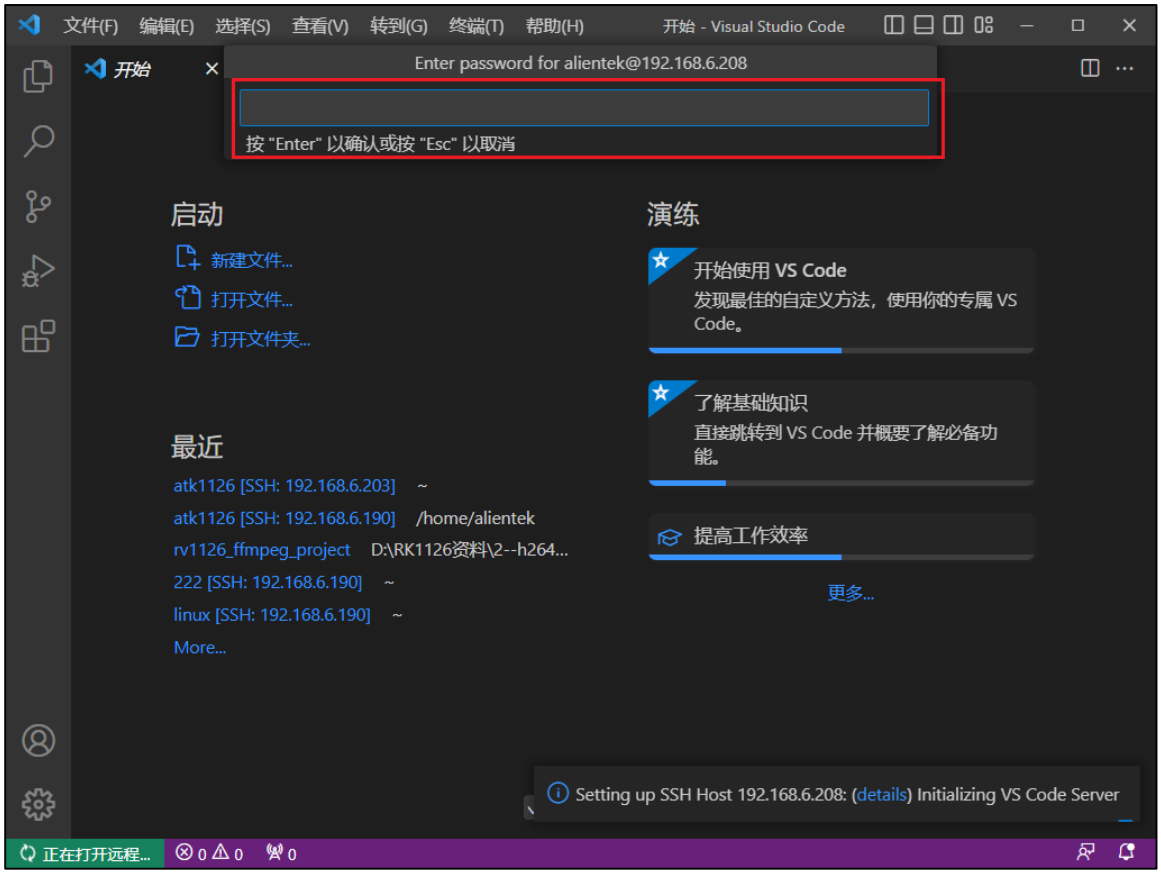


图 3.3.3.8 输入远程登录密码

按照图 3.3.3.8 中，输入远程端的电脑密码，按回车键，就能弹出新的 vscode 窗口，显示连接成功，如下图所示：

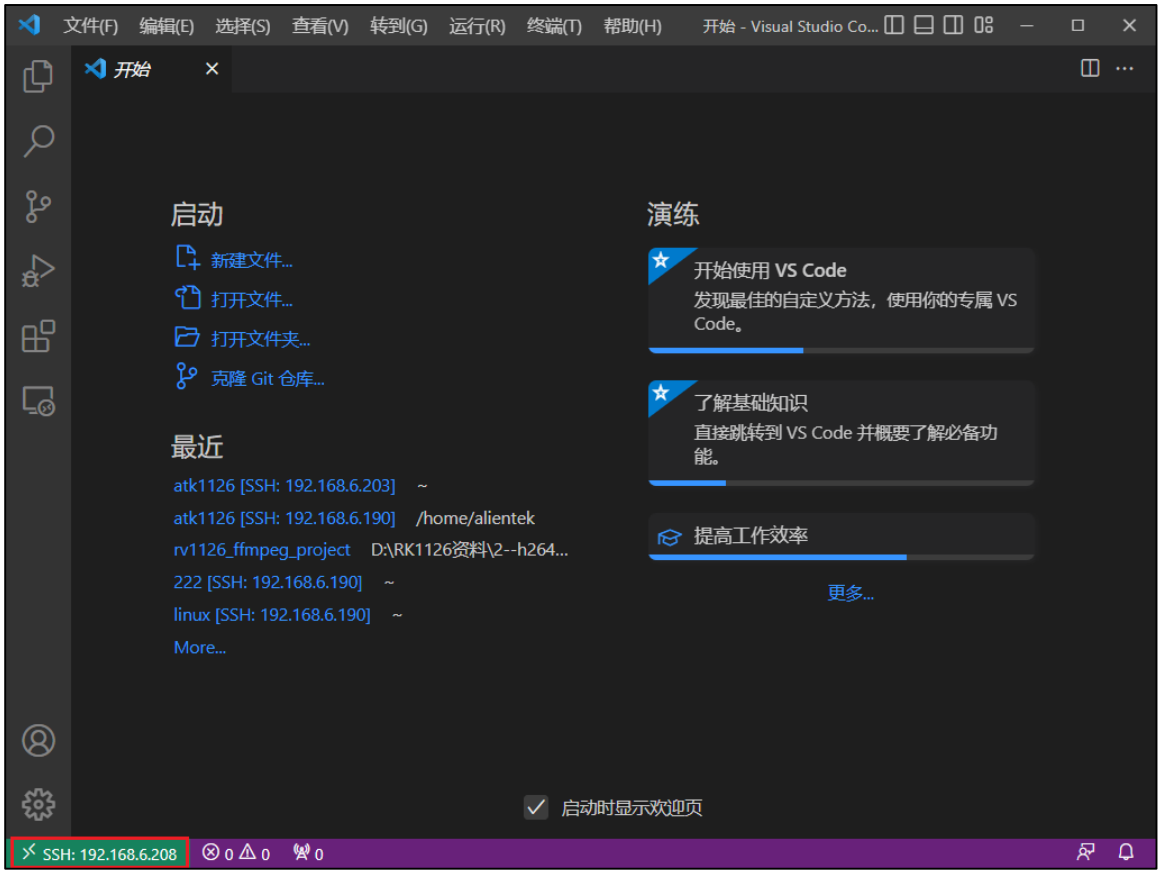


图 3.3.3.9 远程到 192.168.6.208 Ubuntu 系统下的 vscode

图 3.3.3.9 中的左下角里面，已经远程 Ubuntu 系统了。

3.3.4 vscode 的使用

本小节主要是教大家如何通过 vscode 远程的方式，阅读及 Ubuntu 下的代码。首先我们在 Ubuntu 的家目录下下创建一个 test 文件夹，test 文件夹下创建了一个 main.c 文件，如下图。

```
alientek@ubuntu:~/test$ vi main.c
alientek@ubuntu:~/test$ ls
main.c
alientek@ubuntu:~/test$
```

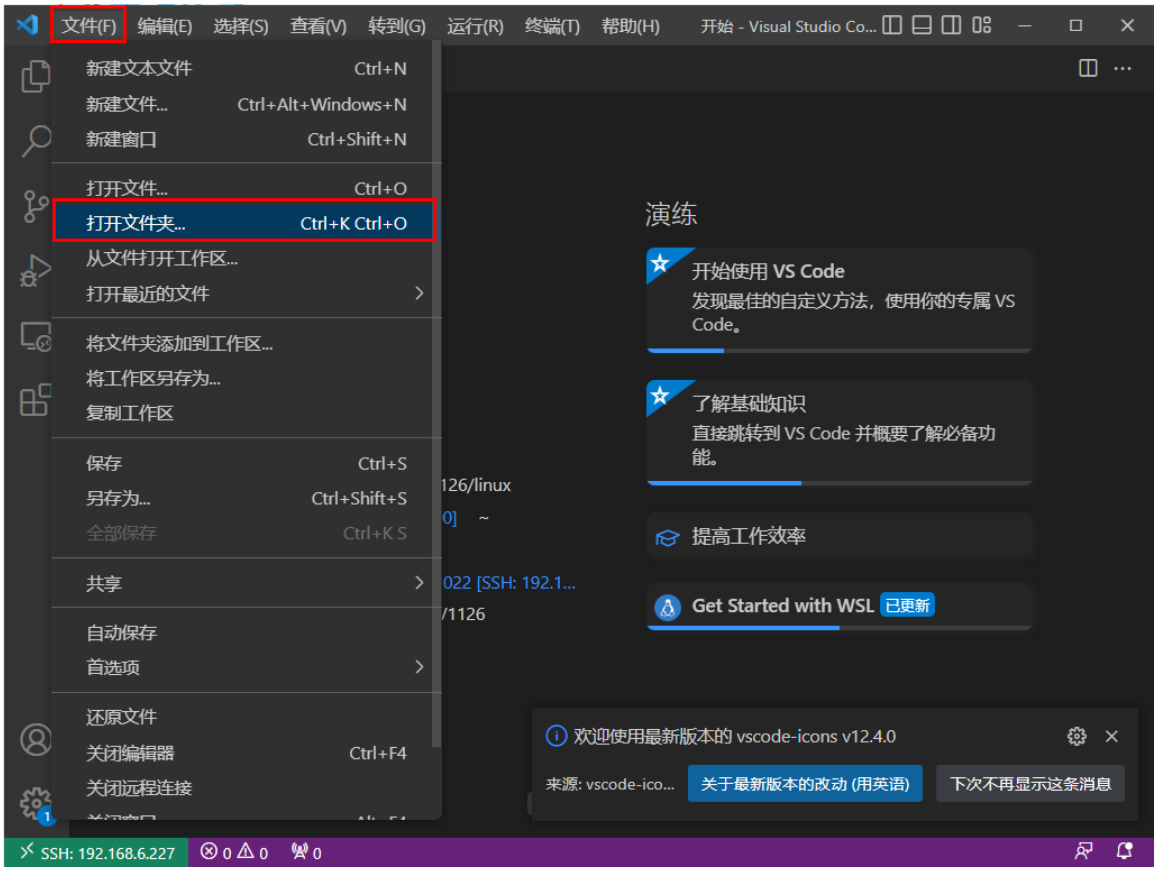


图 3.3.4.1 打开文件夹

点击图 3.3.4.1 中的“打开文件夹”如下图所示：

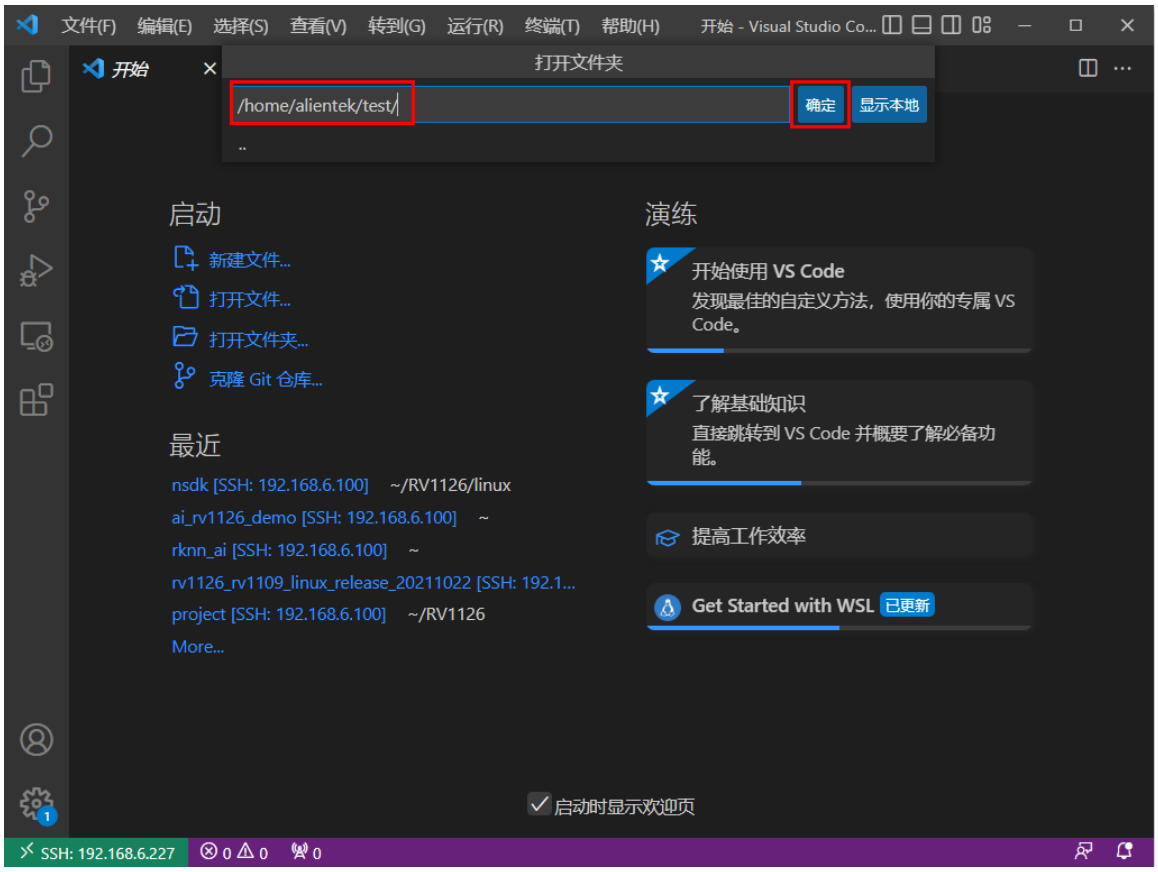


图 3.3.4.2 选择 SDK 包的文件夹

定位到 Ubuntu 下的 test 文件夹“/home/alientek/test”，红色部分为自己 Ubuntu 的用户名。所以我们在输入框中输入此路径，点击“确认”，然后就要我们输入密码，输入密码按回车键，进入如下图所示：

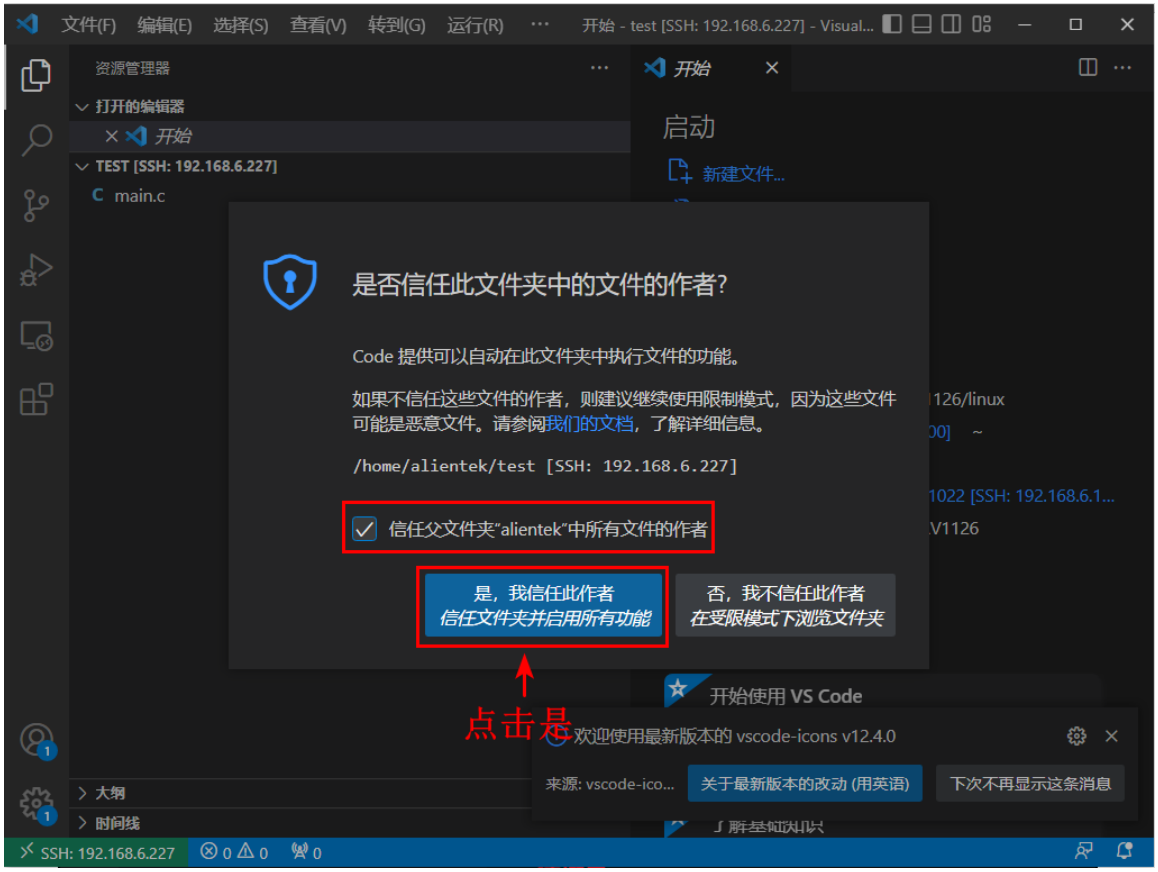


图 3.3.4.3 信任文件夹

图 3.3.4.3 中我们选择是信任文件夹，就会有如下图所示：

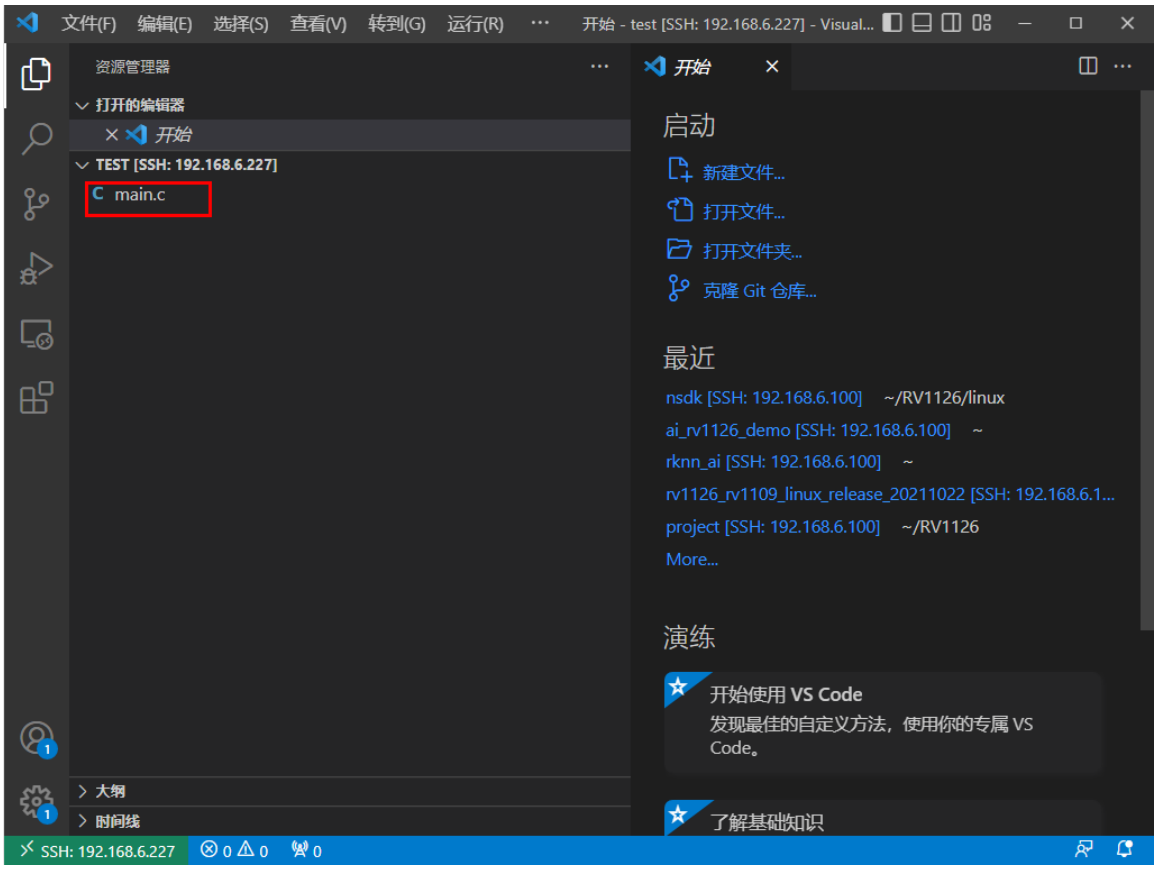


图 3.3.4.4 远程浏览 Ubuntu 系统下的 SDK 代码

此时我们就可以使用 Windows 系统下的 vscode 阅读 Ubuntu 下的代码了，不用切换系统，还能打开终端进行源码编译编译。

打开终端方法如下，连接的终端与 Ubuntu 下的终端使用方法一样。

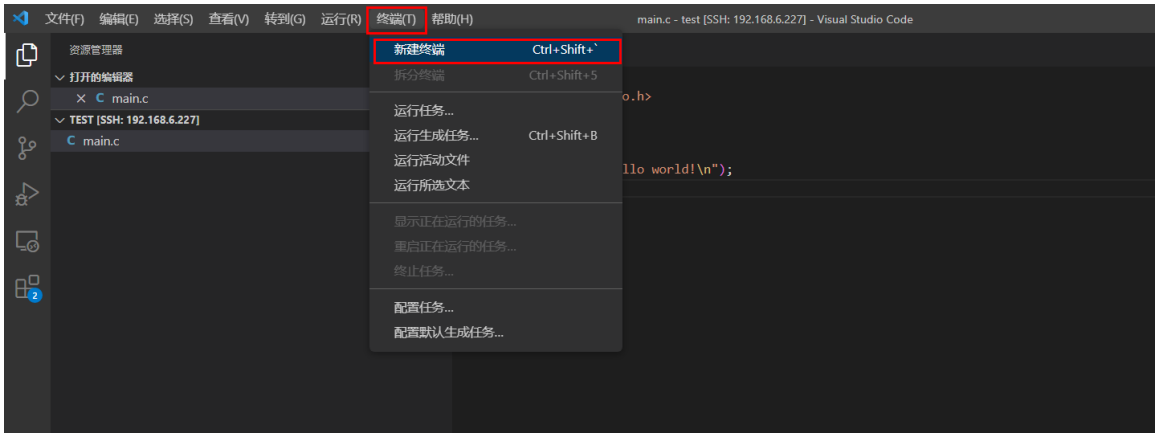


图 3.3.4.5 打开终端

打开的终端如下图 3.3.4.6。

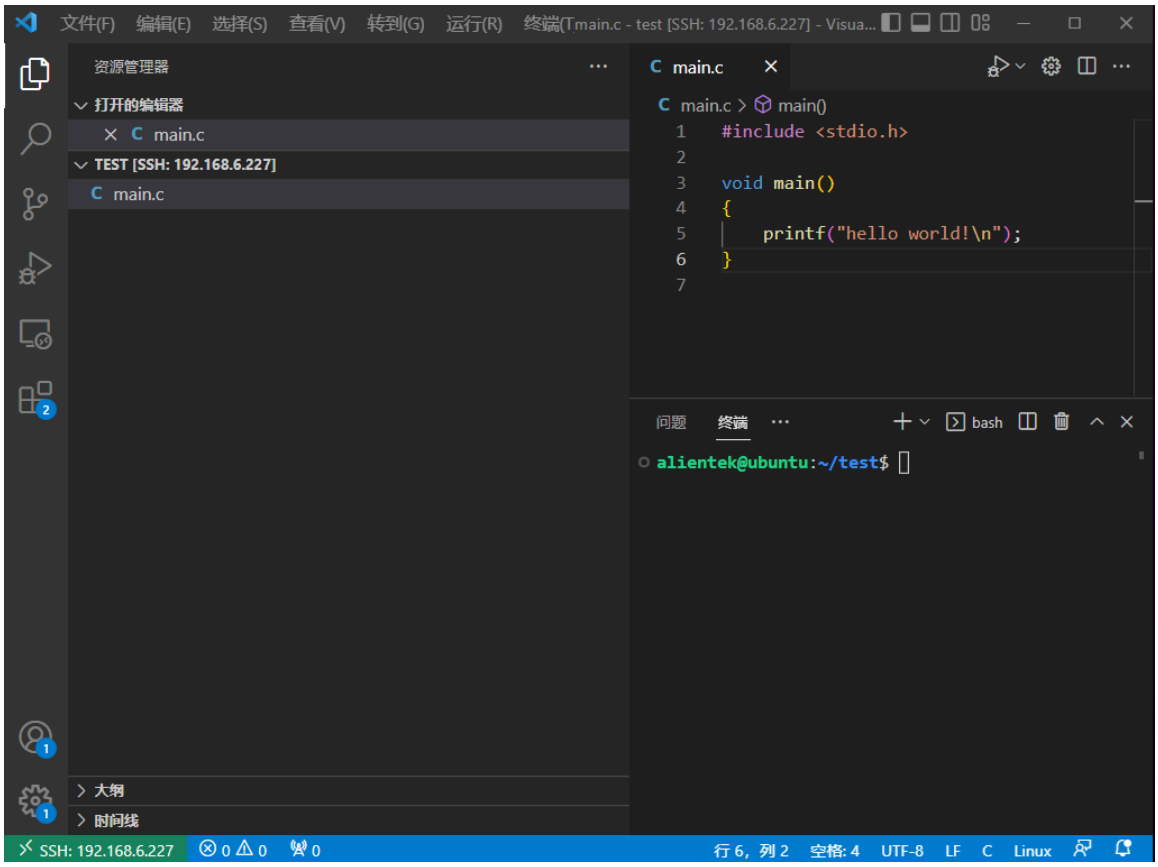


图 3.3.4.6 打开的终端

现在我们就可以使用 gcc 来编译代码了。如果你的 Ubuntu 还没有安装 gcc，请输入 sudo apt-get install gcc。执行下面的指令编译代码与运行代码。

```
gcc main.c -o main
./main
```

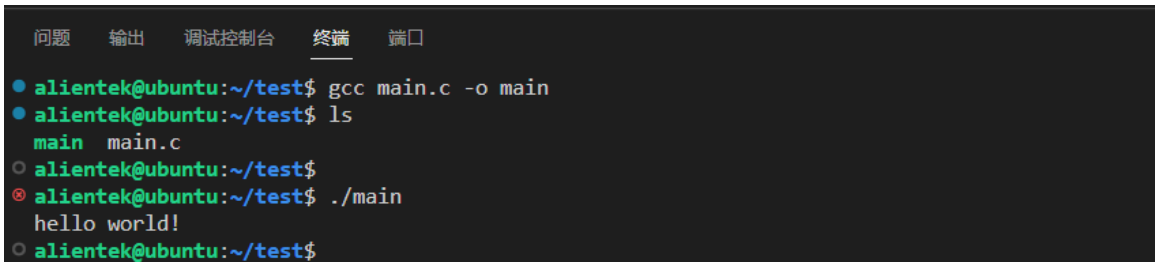


图 3.3.4.7 编译及运行代码

第四章 Ubuntu 系统入门

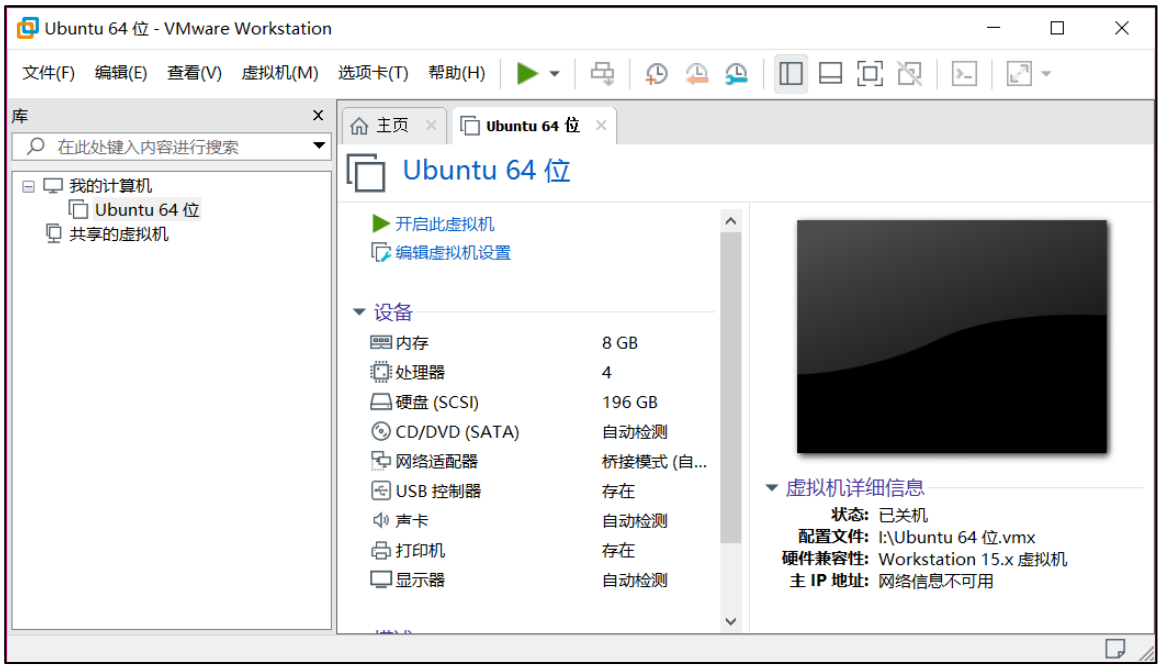
在第二章我们已经安装好虚拟机，并且在虚拟机中安装好了 Ubuntu 操作系统了，本章我们就来学习 Ubuntu 系统的基本使用，通过本章的学习为我们以后的开发做准备。Ubuntu 系统是和 Windows 一样的大型桌面操作系统，因此功能非常强大，不是一章就能介绍完的，因此本章叫做《Ubuntu 系统入门》。本章的主要目的是教会读者掌握后续嵌入式开发所需的 Ubuntu 基本技能，比如系统的基本设置、常用的 shell 命令、vim 编辑器的基本操作等等，如果想详细的学习 Ubuntu 操作系统请参考其它更为详实的书籍，本章参考了《Ubuntu Linux 从入门到精通》，这本书不厚，很适合用来作 Ubuntu 入门。

4.1 Ubuntu 系统初体验

4.1.1 Hello Ubuntu

上一章我们已经安装好了 Ubuntu 操作系统，我们再来回顾一下如何开机，注意截图不是第一章所安装的 Ubuntu，这部分是正点原子左忠凯使用 Ubuntu16 做的教程，为了方便，笔者不再重新截图，步骤基本一样：

1、打开 VMware 虚拟机软件，打开以后如图 4.1.1.1 所示：



、

图 4.1.1.1 WMware 主界面

2、打开 VMware 上的开机按钮，打开方式如图 4.1.1.2 所示：

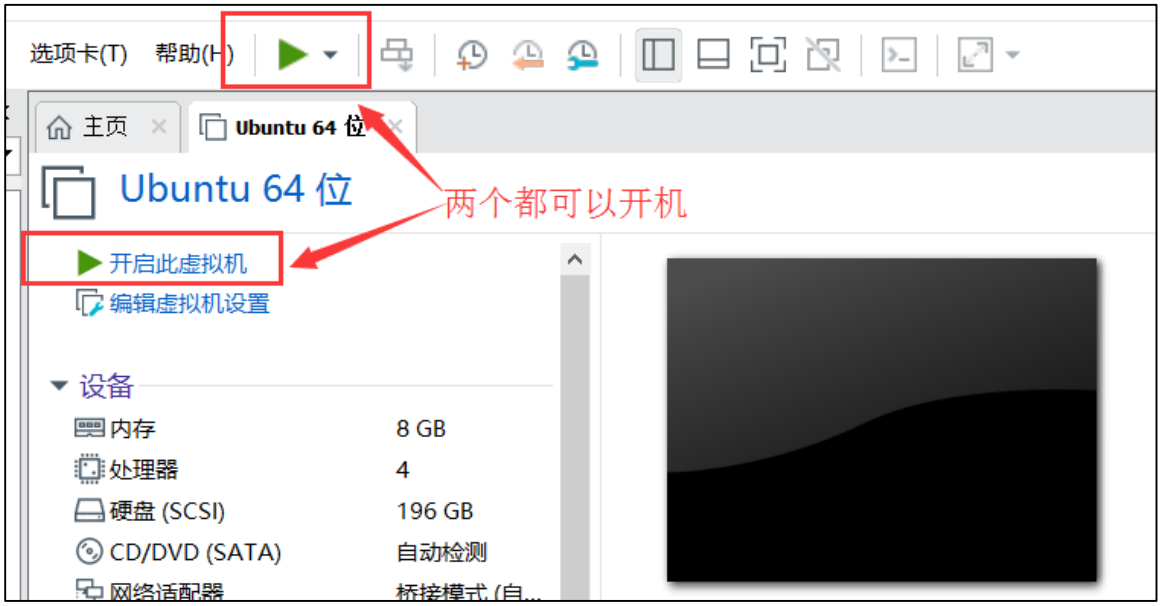


图 4.1.1.2 VMware 开机按钮

3、点击图 4.1.1.2 中两个开机按钮中的任意一个就会打开 Ubuntu 操作系统，首先进入图 4.1.1.3 所示的登陆界面，输入密码即可进入系统。

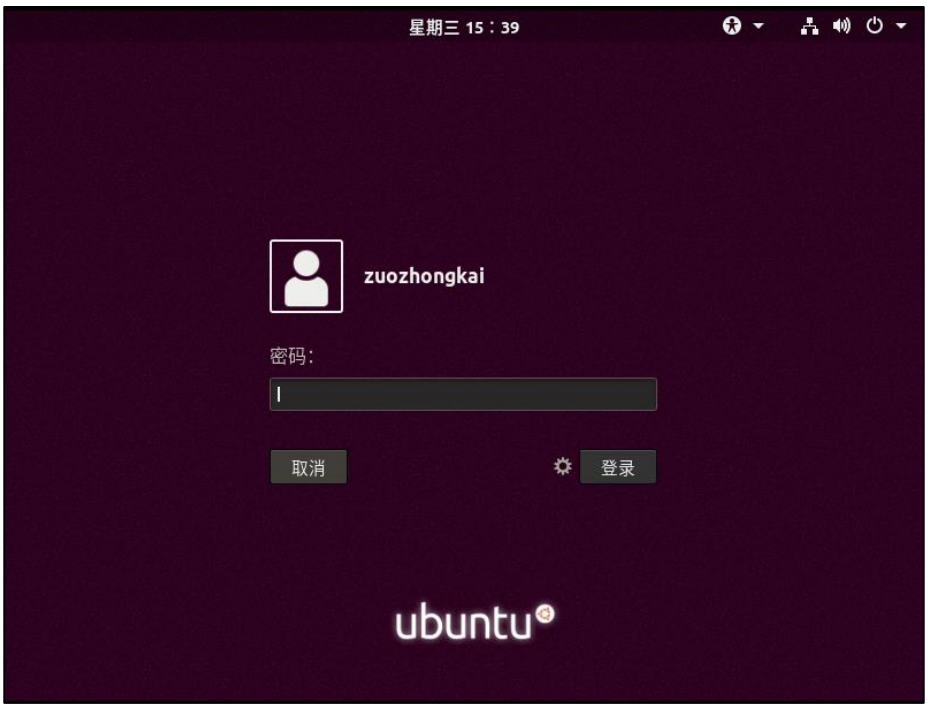


图 4.1.1.3 Ubuntu 登陆界面

在登陆界面输入密码，进入系统主界面，如图 2.1.1.4 所示：

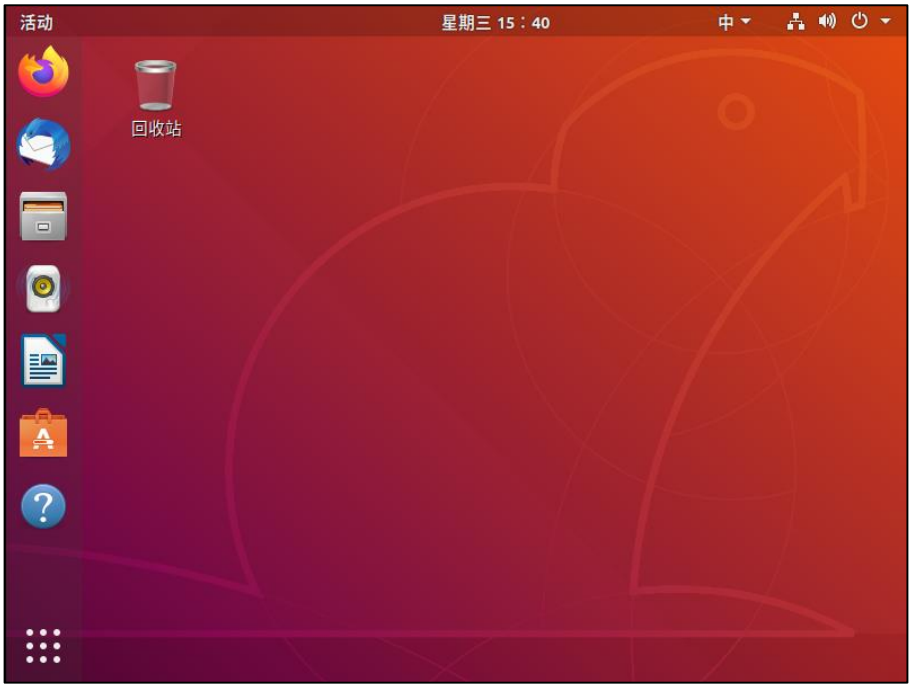


图 4.1.1.4 Ubuntu 主界面

进入主界面以后大家就可以看到和 Windows 基本一样，左侧有一列 APP，第一个是 firefox 浏览器，可以用来上网，比如我们登陆百度网站，如图 4.1.1.5 所示：

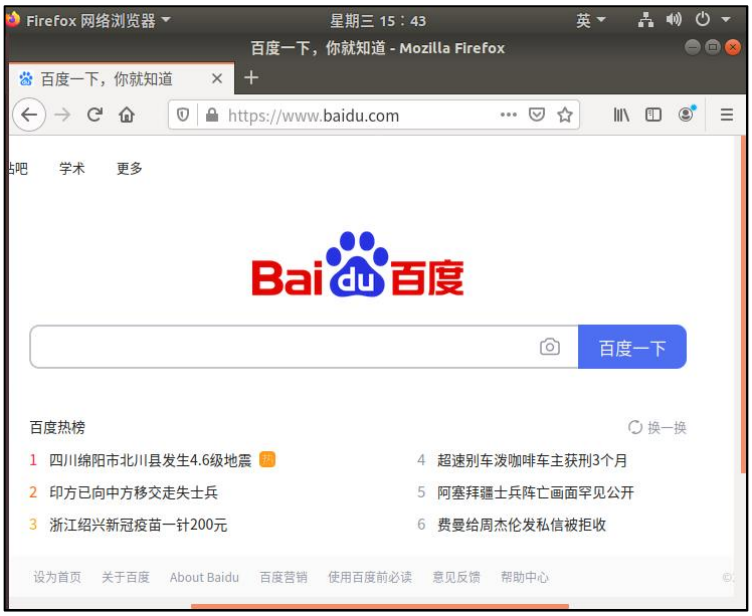


图 4.1.1.5 firefox 浏览器

第二个是 Thunderbird 邮件/新闻，第三个是文件浏览器，打开以后可以浏览 Ubuntu 系统中的文件，打开以后如图 4.1.1.6 所示：

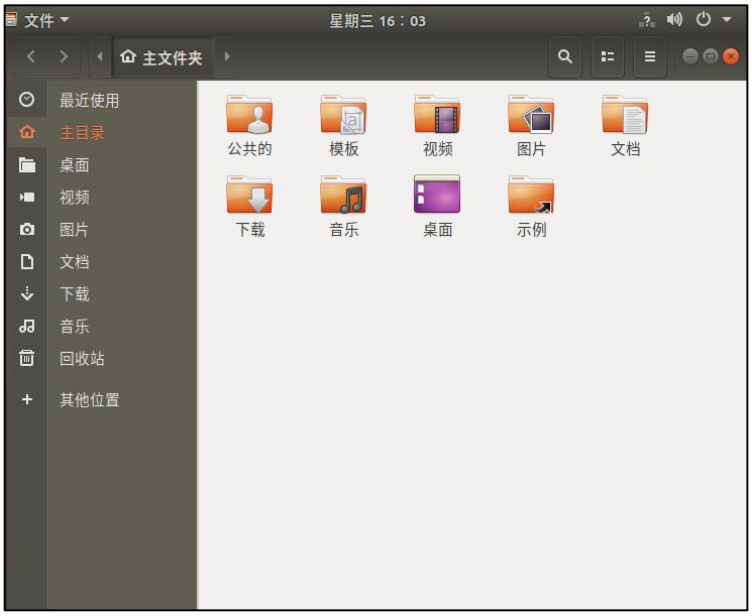


图 4.1.1.6 文件浏览

这里还有其它一些 APP，大家可以自行打开看一下这些 APP 都是干啥的，这里就不一一详细的介绍了。

4.1.2 系统设置

我们会发现，Ubuntu 的默认桌面很小，这是因为 Ubuntu 默认分辨率是 800*600，（注：如果你安装的 Ubuntu 已经全屏，就不需要做下面的操作）因此我们首先要设置系统分辨率，调整到合适的大小。打开系统设置界面，打开方式如图 4.1.2.1 所示：

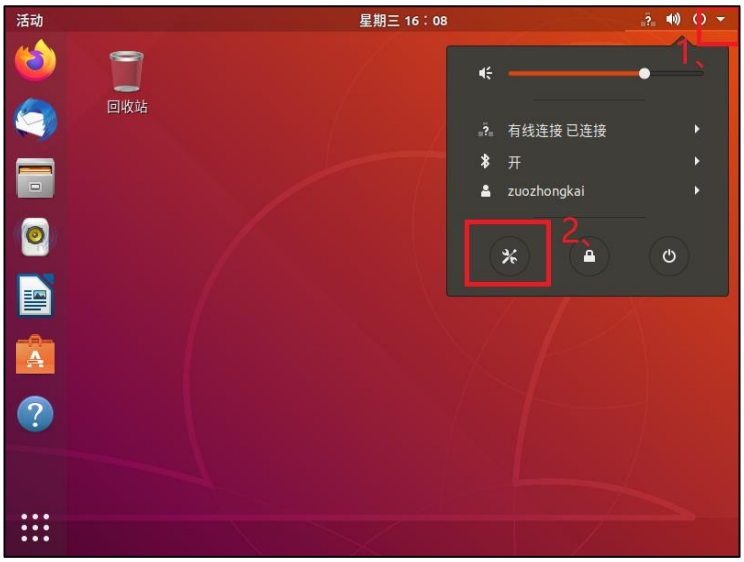


图 4.1.2.1 打开系统设置

打开以后的系统设置界面如图 4.1.2.2 所示：



图 4.1.2.2 系统设置界面

系统设置界面可以完成系统的大部分设置，我们找到“设备”选项并打开，打开以后如图 4.1.2.3 所示：



图 2.1.2.3 设备界面

图 4.1.2.3 就是“设备”界面，选中“显示”这一栏，右边就是显示相关设置，就有分辨率选项，点击“分辨率”选项，如图 4.1.2.4 所示：



图 4.1.2.4 分辨率设置项

从图 4.1.2.4 可以看出，Ubuntu 安装好以后默认分辨率是 800×600 的，点击“分辨率”这一栏，可以对 Ubuntu 分辨率进行设置，由于默认分辨率太低，因此图 4.1.2.4 中看到的图像不完整，不过这不影响我们进行分辨率设置，大家根据自己所使用的显示器进行设置，比如我这里设置 Ubuntu 分辨率为 1920×1200。设置好以后点击右上角的“应用”按钮，这里要注意，由于默认分辨率太小了，导致右上角的“应用”按钮看不到，所以需要拖动设置界面，把右侧的“应用”按钮拖出来，如图 4.1.2.5 所示：

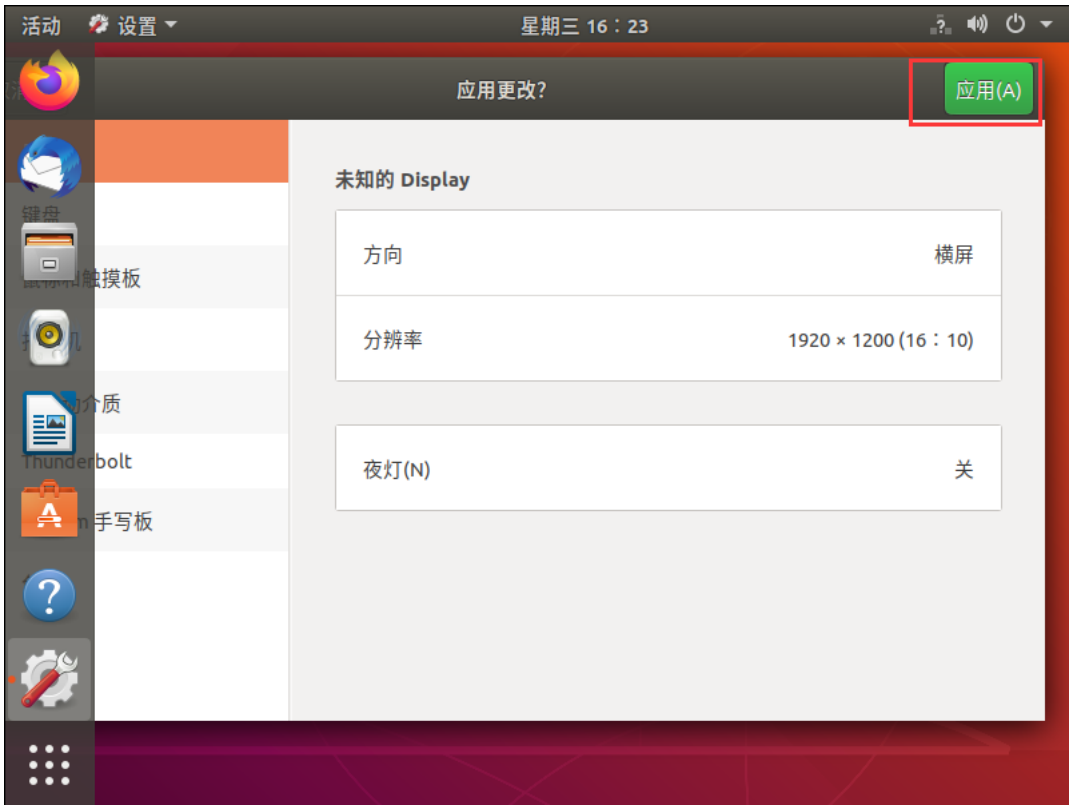


图 4.1.2.5 显示界面的“应用”按钮

点击图 4.1.2.5 中的“应用”按钮以后会弹出来是否保留显示设置对话框，点击“保留更改”，如图 4.1.2.6 所示：

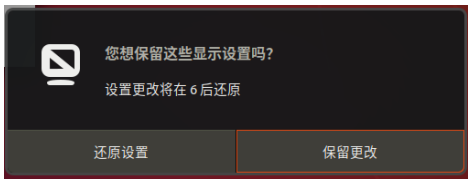


图 4.1.2.6 是否保留显示设置对话框

设置好分辨率以后 Ubuntu 的主界面就大了，看起来也舒服了。通过设置系统分辨率这个例子，我们就知道了如何设置 Ubuntu 系统，如果有需要设置其它东西的话都可以到系统设置里面去进行，这里就不一一详细的介绍了。

4.1.3 系统关机与锁定

当我们不使用 Ubuntu 系统以后就需要将其关机，就和我们使用 Windows 系统一样，千万不要通过直接退出 VMware 软件来关机！！关机很简单，在主界面，点击右上角的关机图标，然后选择关机选项，如图 4.1.3.1 所示：

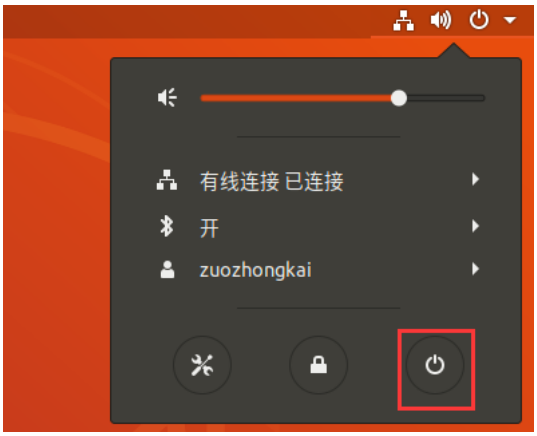


图 4.1.3.1 关机

点击在图 4.1.3.1 中的关机按钮以后，会弹出关机对话框，如图 2.1.3.2 所示：

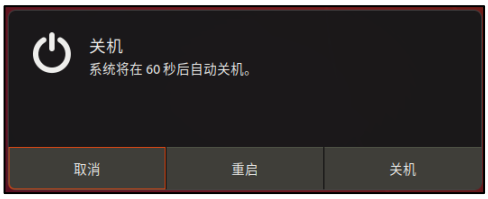


图 4.1.3.2 关机对话框

可以看到有三个选项：取消，重启和关机，想要重启就点击“重启”按钮，想要关机就点击“关机”按钮。

有时候我们需要临时离开，但是既不想关闭 Ubuntu，又不想让别人看我们的 Ubuntu，这个时候就可以锁定系统，点击主界面右上角，打开图 4.1.3.1 所示操作界面，然后点击中间的  图标即可锁定系统。

4.1.4 中文输入测试

我们是中国人，平时用的做多的肯定是中文，那么 Ubuntu 下中文输入是否和 Windows 一样呢？如何在 Ubuntu 下使用中文输入法。本节我们就以创建一个文本为例，介绍如何在 Ubuntu 中使用中文输入法。

Ubuntu 有个奇葩的地方，那就是安装好以后点击鼠标右键没有新建文档选项，这样我们创建文本文档很不方便。我们要做一个简单的设置，点击 Ubuntu 系统主界面上的文件 APP，找到名为“模板”的文件夹，如图 4.1.4.1 所示：



图 4.1.4.1 模板文件夹

打开图 4.1.4.1 中的“模板”文件夹，此文件夹默认是空的，在此文件夹中点击鼠标右键，选择“在终端打开”，如图 4.1.4.2 所示：



图 4.1.4.2 在终端中打开文件夹

打开终端以后使用 gedit 创建一个文件，输入如下命令：

```
sudo gedit 无标题文档 //使用 gedit 创建一个空文件
```

上述命令需要输入中文“无标题文档”，Ubuntu 和 Windwos 下的输入法切换是一样的，按下“WIN+空格键”组合键即可进行输入法选择。

输入上面的命令以后会用 gedit 创建并打开一个空文件，如图 4.1.4.3 所示：

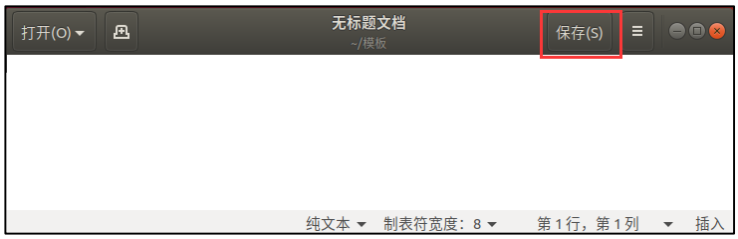


图 4.1.4.3 gedit 创建文件

直接点击图 4.1.4.3 右上角的“保存”按钮，此时就会在图 4.1.4.1 中的“模板”文件夹下创建了一个名为“无标题文档”的空文件，如图 4.1.4.4 所示：



图 4.1.4.4 新建的“无标题文档”文件

重新回到主界面，在桌面上点击鼠标右键，此时就会有“新建文档”选项，如图 4.1.4.5 所示：



图 4.1.4.5 新建文档

点击图 4.1.4.5 中的“无标题文档”，此时就会在桌面上创建一个名为“无标题文档”的文件，如图 4.1.4.6 所示：



图 4.1.4.6 新建的无标题文档

双击打开文档，打开以后如图 4.1.4.7 所示：



图 4.1.4.7 打开文档

打开文档以后，我们可以尝试在里面输入一些英文和数字，输入英文和数字是没有任何问题的，输入中文的话需要切换到 Ubuntu 自带的拼音输入法，前面也讲了，使用“WIN+空格键”，切换到拼音输入法以后就可以输入中文了，如图 4.1.4.8 所示：



图 4.1.4.8 输入中文文本

大家会发现 Ubuntu 下的拼音输入法使用起来跟 Windows 下的输入法差距太大了，没有 Windows 下的输入法好用，没办法，谁让桌面端 Linux 用的少呢，所以也就没有啥公司开发 Linux 下的输入法。

通过上面几个小节中对 Ubuntu 的基本操作来看，基本和 Windows 下的操作差不多，嵌入式开发中不会通过图形界面操作 Ubuntu，而是通过命令行操作的。这也是我们接下来着重要讲的：Ubuntu(Linux)终端操作，会涉及到很多命令，但是常用的命令就那几十个，不需要刻意的去背，使用习惯了就自然记住了。不要看到要记命令就觉得可怕。根据 2080 原则，80%情况下只使用那 20%的命令，实际情况会更少，常用的可能就那 5%~10%的命令。

4.2 Ubuntu 终端操作

本节就是我们学习 Ubuntu 操作系统的重点了，终端操作，也就是俗称的“敲命令”，不管是哪个版本的 Linux 发行版系统，它都会提供终端操作，Linux 下的终端操作类似与 Windows 下的 DOS 操作。要使用终端首先肯定是要打开终端，在主界面上点击鼠标右键，然后选择打开终端，如图 2.2.1 所示：



图 4.2.1 打开终端

打开终端以后如图 4.2.2 所示：

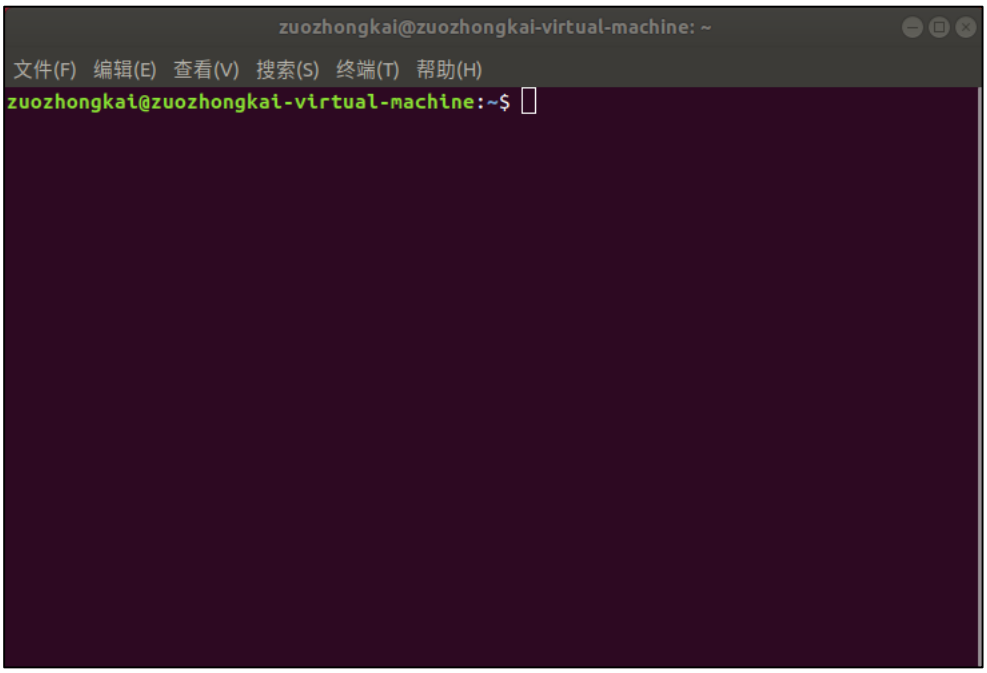


图 4.2.2 终端界面

我们就是在图 4.2.2 所示界面上输入命令的，终端默认会有类似下面一行所示的一串提示符:

```
zuozhongkai@zuozhongkai-virtual-machine: ~$
```

上述字符串中，@前面的“zuozhongkai”是当前的用户名字，@后面的 zuozhongkai-virtual-machine 是我的机器名字。最后面的符号“$”表示当前用户是普通用户，我们可以在提示符后面输入命令，比如输入命令“ls”，命令“ls”是打印出当前所在目录中所有文件和文件夹，如图 4.2.3 所示：

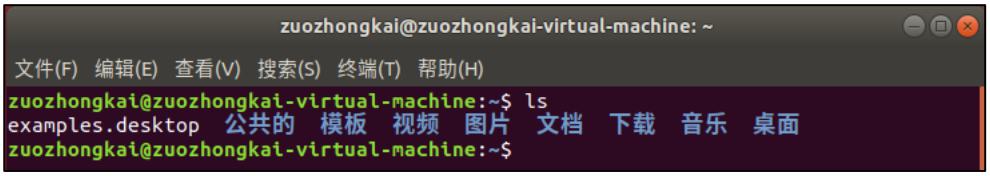


图 4.2.3 ls 命令

在图 4.2.3 中我们输入了“ls”这个命令，然后打印出了当前目录下的所有文件和文件夹，后面我们学习命令的时候就是在终端中输入相应命令的。

4.3 Shell 操作

4.3.1 Shell 简介

学习 linux 的时候会频繁的看到 Shell 这个词语？那么什么是 Shell 呢？网上搜索一下，各种专业的解释一堆，但是对于第一次接触 Linux 的人来说这些专业的词语只会让人更晕。简单的说 Shell 就是敲命令。国内把 Linux 下通过命令行输入命令叫做“敲命令”，国外人玩的比较洋气，人家叫做“Shell”。因此以后看到 Shell 这个词语第一反应就是在终端中敲命令，将多个 Shell 命令按照一定的格式放到一个文本中，那么这个文本就叫做 Shell 脚本。

严格意义上来讲，Shell 是一个应用程序，它负责接收用户输入的命令，然后根据命令做出相应的动作，Shell 负责将应用层或者用户输入的命令传递给系统内核，由操作系统内核来完成相应的工作，然后将结果反馈给应用层或者用户。

4.3.2 Shell 基本操作

前面我们说 Shell 就是“敲命令”，那么既然是命令，那肯定是有格式的，Shell 命令的格式如下：

```
command      -options      [argument]
```

command: Shell 命令名称。

options：选项，同一种命令可能有不同的选项，不同的选项其实现的功能不同。

argument：Shell 命令是可以带参数的，也可以不带参数运行。

同样以命令“ls”为例，下面“ls”命令的三种不同格式其结果也不同：

```
ls
ls -l
ls /usr
```

这三种命令的运行结果如图 4.3.2.1 所示：

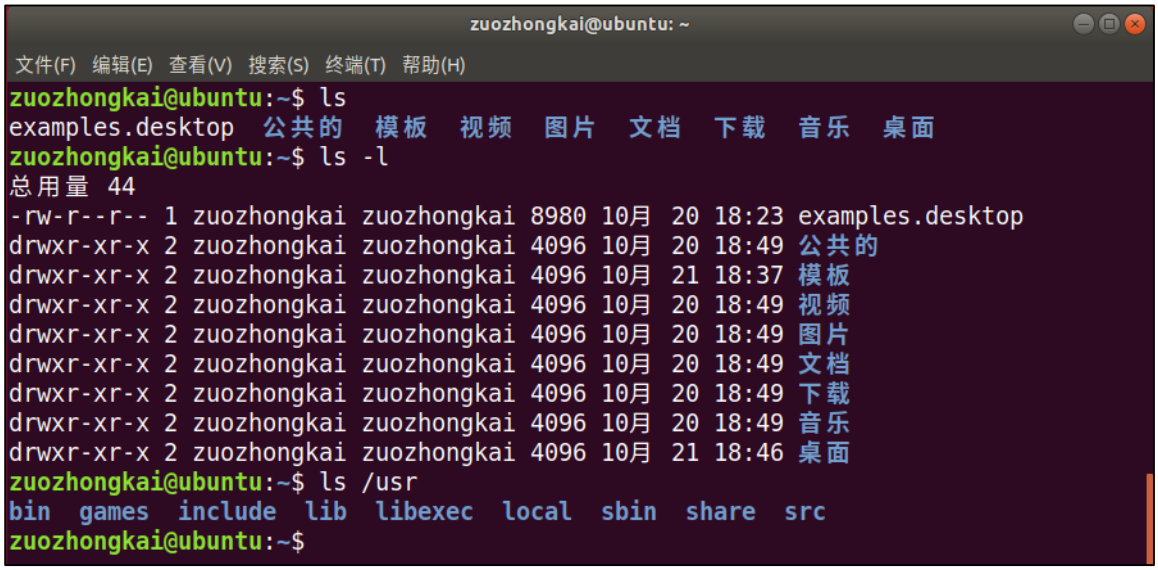


图 4.3.2.1 ls 命令

在图 4.3.2.1 中“ls”命令用来打印出当前目录下的所有文件和文件夹，而“ls -l”同样是打印出当前目录下的所有文件和文件夹，但是此命令会列出所有文件和文件夹的详细信息，比如文件大小、拥有者、创建日期等等。最有一个“ls /usr”是用来打印出目录“/usr”下的所有文件和文件夹。

Shell 命令是支持自动补全功能的，因为 Shell 命令非常多，如果不作自动补全的话就需要用户去记忆这些命令的全部字母。使用自动补全功能以后我们只需要输入命令的前面一部分字母，然后按下 TAB 键，如果只有一个命令匹配的话就会自动补全这个命令剩下的字母。如果有多个命令匹配的话系统就会发出报警声音，此时在按下一次 TAB 键就会列出所有匹配的命令，比如我们输入字母“if”，然后按下 TAB 键，结果如图 4.3.2.2 所示：

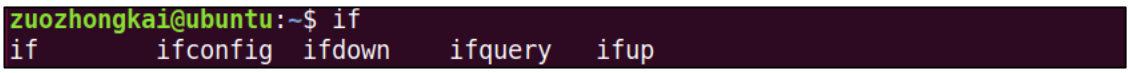


图 4.3.2.2 “if”开始的命令

从图 4.3.2.2 可以看出，以“if”开头的命令有 5 个，我们以“ifconfig”为例(Ubuntu 系统装好以后默认是没有 ifconfig 命令的，需要手动安装，在终端中输入“sudo apt-get install net-tools”)，此命令是用来查看网卡信息的，我们重新输入“ifc”然后在按一下 TAB 键，就会自动补全出“ifconfig”命令，因为以“ifc”开头的命令只有一个，结果如图 4.3.2.3 所示：

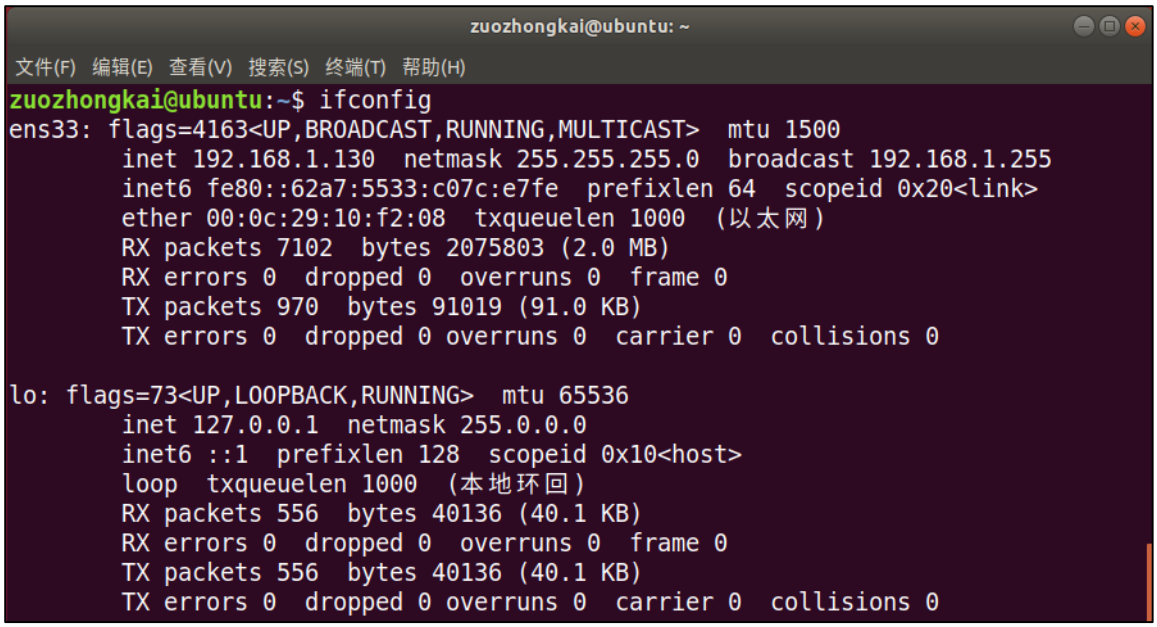


图 4.3.2.3 ifconfig 命令结果

4.3.3 常用 Shell 命令

我们做嵌入式开发用的最多就是 Shell 命令，Shell 命令是所有的 Linux 系统发行版所通用的，并不是说我在 Ubuntu 下学会了 Shell 命令，换另外一个 Linux 发行版操作系统以后就没用了(不同的发行版 Linux 系统可能会自定义一些命令)。本节我们先来介绍一些 Shell 下常用的命令：

1、目录信息查看命令 ls

文件浏览是最基本的操作了，Shell 下文件浏览命令为 ls，格式如下：

```
ls  [选项]  [路径]
```

ls 命令主要用于显示指定目录下的内容，列出指定目录下包含的所有的文件以及子目录，它的主要参数有：

-a ：显示所有的文件以及子目录，包括以“.”开头的隐藏文件。
-l ：显示文件的详细信息，比如文件的形态、权限、所有者、大小等信息。
-t ：将文件按照创建时间排序列出。
-A ：和-a 一样，但是不列出“.”(当前目录)和“..”(父目录)。
-R ：递归列出所有文件，包括子目录中的文件。

Shell 命令里面的参数是可以组合在一起用的，比如组合“-al”就是显示所有文件的详细信息，包括以“.”开头的隐藏文件，ls 命令使用如图 4.3.3.1 所示：

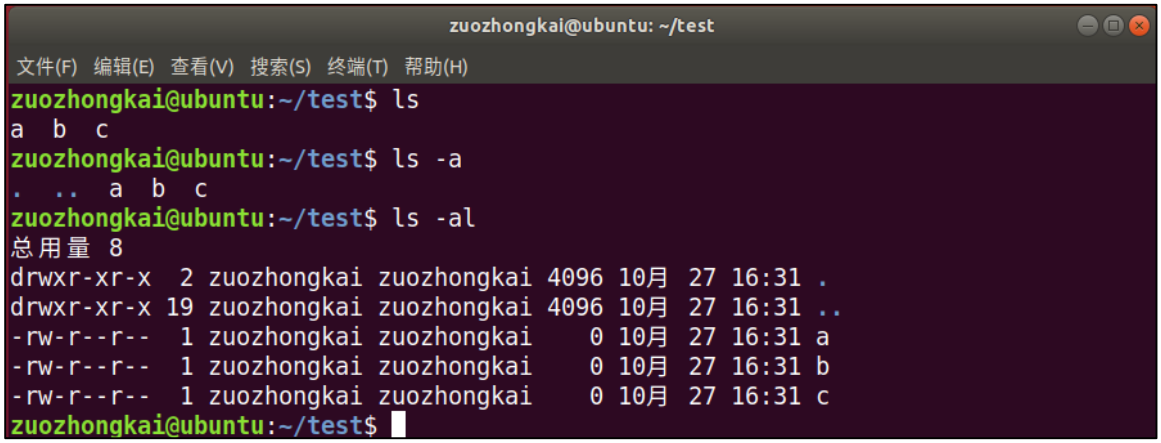


图 4.3.3.1 ls 命令演示

注意图 4.3.3.1 中 tmp 文件夹是我为了演示方便，自己创建的，里面的文件 a，b 和 c 也是我创建的，关于文件夹和文件的创建后面会详细的讲解。

2、目录切换命令 cd

要想在 Shell 中切换到其它的目录，使用的命令是 cd，命令格式如下：

```
cd    [路径]
```

路径就是我们要进入的目录路径，比如下面所示操作：

```
cd   /          //进入到根目录“/”下，Linux 系统的根目录为“/”，
cd   /usr       //进入到目录“/usr”里面。
cd   ..         //进入到上一级目录。
cd   ~          //切换到当前用户主目录
```

比如我们要进入到目录“/usr”下去，并且查看“/usr”下有什么文件，操作如图 4.3.3.2 所示：

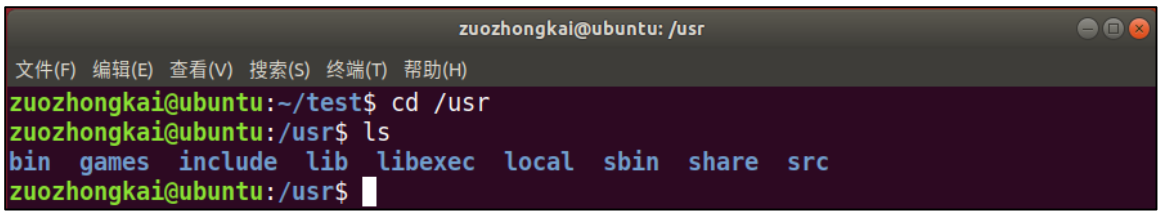


图 4.3.3.2 cd 命令演示

在图 4.3.3.2 中，我们先使用命令“cd /usr”进入到“/usr”目录下，然后使用“ls”命令显示“/usr”目录下的所有文件。仔细观察图 4.3.3.2 可以看到，当我们切换到其它目录以后在符号“$”前面就会以蓝色的字体显示出当前目录名字，如图 4.3.3.3 所示：

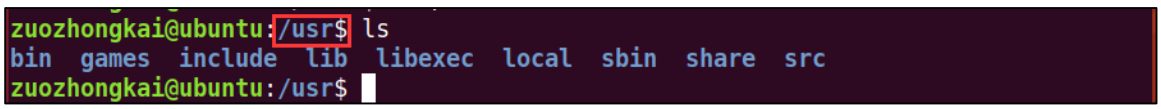


图 4.3.3.3 目录路径显示

3、当前路径显示命令 pwd



图 4.3.3.4 pwd 命令

4、系统信息查看命令 uname

要查看当前系统信息，可以使用命令 uname，命令格式如下：

```
uname  [选项]
```

可选的选项参数如下：

-r ：列出当前系统的具体内核版本号。

-s ：列出系统内核名称。

-o ：列出系统信息。

使用如图 4.3.3.5 所示：

```
zuozhongkai@ubuntu:~$ uname
Linux
zuozhongkai@ubuntu:~$ uname -r
5.4.0-52-generic
zuozhongkai@ubuntu:~$ uname -s
Linux
zuozhongkai@ubuntu:~$ uname -o
GNU/Linux
zuozhongkai@ubuntu:~$
```

图 4.3.3.5 uanme 命令操作

5、清屏命令 clear

clear 命令用于清除终端上的所有内容，只留下一行提示符。

6、切换用户执行身份命令 sudo

Ubuntu(Linux)是一个允许多用户的操作系统，其中权限最大的就是超级用户 root，有时候我们执行一些操作的时候是需要用 root 用户身份才能执行，比如安装软件。通过 sudo 命令可以使我们暂时将身份切换到 root 用户。当使用 sudo 命令的时候是需要输入密码的，这里要注意输入密码的时候是没有任何提示的！命令格式如下：

```
sudo  [选项]  [命令]
```

选项主要参数如下：

-h ：显示帮助信息。

-l ：列出当前用户可执行与不可执行的命令

-p ：改变询问密码的提示符。

假如我们现在要创建一个新的用户 test，创建新用户的命令为“adduser”，创建新用户的权限只有 root 用户才有，我们在装系统的时候创建的那个用户是没有这个权限的，比如我的“zuozhongkai”用户。所以创建新用户的话需要使用“sudo”命令以 root 用户执行“adduser”这个命令，如图 4.3.3.6 所示：

```
zuozhongkai@ubuntu:~$ adduser test
adduser: 只有 root 才能将用户或组添加到系统。
zuozhongkai@ubuntu:~$ sudo adduser test
[sudo] zuozhongkai 的密码：
正在添加用户"test"...
正在添加新组"test" (1001)...
正在添加新用户"test" (1001) 到组"test"...
创建主目录"/home/test"...
正在从"/etc/skel"复制文件...
输入新的 UNIX 密码：
重新输入新的 UNIX 密码：
passwd: 已成功更新密码
正在改变 test 的用户信息
请输入新值，或直接敲回车键以使用默认值
        全名 []:
        房间号码 []:
        工作电话 []:
        家庭电话 []:
        其它 []:
这些信息是否正确？ [Y/n] y
```

图 4.3.3.6 sudo 命令演示

在图 4.3.3.6 中，我们一开始直接使用“adduser test”命令添加用户的时候提示我们“adduser：只有 root 才能将用户或组添加到系统。”所以我们要在前面加上“sudo”命令，表示以 root 用户执行 adduser 操作。

7、添加用户命令 adduser

在讲解 sudo 命令的时候我们已经用过命令“adduser”，此命令需要 root 身份去运行。命令格式如下：

```
adduser [参数] [用户名]
```

常用的参数如下：

-system ：添加一个系统用户

-home DIR ：DIR 表示用户的主目录路径

-uid ID ：ID 表示用户的 uid。

-ingroup GRP ：表示用户所属的组名。

adduser 的使用我们前面已经演示过了，大家可以试着再添加一个用户。

8、删除用户命令 deluser

前面讲了添加用户的命令，那肯定也有删除用户的命令，删除用户使用命令“deluser”，命令参数如下：

```
deluser [参数] [用户名]
```

主要参数有：

-system ：当用户是一个系统用户的时候才能删除。

-remove-home ：删除用户的主目录

-remove-all-files ：删除与用户有关的所有文件。

-backup ：备份用户信息

同样的，命令“deluser”也要使用“sudo”来以 root 用户运行，以删除我们前面创建的用户 test 为例，deluser 使用如图 4.3.3.7 所示：

```
zuozhongkai@ubuntu:~$ sudo deluser test
正在删除用户 'test'...
警告：组"test"没有其他成员了。
完成。
zuozhongkai@ubuntu:~$
```

图 4.3.3.7 命令 deluser 演示

9、切换用户命令 su

前面在讲解命令“sudo”的时候说过，“sudo”是以 root 用户身份执行一个命令，并没有更改当前的用户身份，所有需要 root 身份执行的命令都必须在前面加上“sudo”。命令“su”可以直接将当前用户切换为 root 用户，切换到 root 用户以后就可以尽情的进行任何操作了！因为你已经获得了系统最高权限，在 root 用户下，所有的命令都可以无障碍执行，不需要在前面加上“sudo”，“su”命令格式如下：

```
su [选项] [用户名]
```

常用选项参数如下：

-c –command ：执行指定的命令，执行完毕以后恢复原用户身份。

-login ：改变用户身份，同时改变工作目录和 PATH 环境变量。

-m ：改变用户身份的时候不改变环境变量

-h ：显示帮助信息

以切换到 root 用户为例，使用如图 4.3.3.8 所示：

```
zuozhongkai@ubuntu:~$ sudo su
[sudo] zuozhongkai 的密码：
root@ubuntu:/home/zuozhongkai#
```

图 4.3.3.8 su 命令演示

在图 4.3.3.8 中，先使用命令“sudo su”切换到 root 用户，su 命令不写明用户名的话默认切换到 root 用户。然后输入密码，密码正确的话就会切换到 root 用户，可以看到切换到 root 用户以后提示符的“@”符号前面的用户名变成了“root”，表示当前的用户是 root 用户。并且以“#”结束。

注意！！由于 root 用户权限太大，稍微不注意就可能删除掉系统文件，导致系统奔溃，因此强烈建议大家，不要以 root 用户运行 Ubuntu。当要用到 root 身份执行某些命令的时候使用“sudo”命令即可。

要切换回原来的用户，使用命令“sudo su 用户名”即可，比如我要从 root 切换回 zuozhongkai 这个用户，操作如图 4.3.3.9 所示：

```
root@ubuntu:/home/zuozhongkai# sudo su zuozhongkai
zuozhongkai@ubuntu:~$
```

图 4.3.3.9 切换回原来用户

10、显示文件内容命令 cat

查看文件内容是最常见的操作了，在 windows 下可以直接使用记事本查看一个文本文件内容，linux 下也有类似记事本的软件，叫做 gedit，找到一个文本文件，双击打开，默认使用的就是 gedit，如图 4.3.3.10 所示：



图 4.3.3.10 gedit 打开文档

我们现在讲解的是 Shell 命令，那么 Shell 下有没有办法读取文件的内容呢？肯定有的，那就是命令“cat”，命令格式如下：

```
cat [选项] [文件]
```

选项主要参数如下：

-n ：由 1 开始对所有输出的行进行编号。

-b ：和-n 类似，但是不对空白行编号。

-s ：当遇到连续两个行以上空白行的话就合并为一个行空白行。

比如我们以查看文件“/etc/environment”的内容为例，结果如图 4.3.3.11 所示：

```
zuozhongkai@ubuntu:~$ cat /etc/environment
PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games"
zuozhongkai@ubuntu:~$ cat /etc/environment -n
     1  PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games"
```

图 4.3.3.11 命令 cat 演示

11、显示和配置网络属性命令 ifconfig

ubuntu 默认没有 ifconfig 命令，需要自己手动安装，在终端输入如下命令：

```
sudo apt-get install net-tools
```

ifconfig 是一个跟网络属性配置和显示密切相关的命令，通过此命令我们可以查看当前网络属性，也可以通过此命令配置网络属性，比如设置网络 IP 地址等等，此命令格式如下：

```
ifconfig interface options | address
```

主要参数如下：

interface ：网络接口名称，比如 eth0 等。

up ：开启网络设备。

down ：关闭网络设备。

add ：IP 地址，设置网络 IP 地址。

netmask add 子网掩码。

命令 ifconfig 的使用如图 4.3.3.12 所示：

```
zuozhongkai@ubuntu:~$ ifconfig
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.1.130  netmask 255.255.255.0  broadcast 192.168.1.255
        inet6 fe80::62a7:5533:c07c:e7fe  prefixlen 64  scopeid 0x20<link>
        ether 00:0c:29:10:f2:08  txqueuelen 1000  (以太网)
        RX packets 30653  bytes 4231234 (4.2 MB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 1137  bytes 105937 (105.9 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (本地环回)
        RX packets 1754  bytes 121732 (121.7 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 1754  bytes 121732 (121.7 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

zuozhongkai@ubuntu:~$ ifconfig ens33
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.1.130  netmask 255.255.255.0  broadcast 192.168.1.255
        inet6 fe80::62a7:5533:c07c:e7fe  prefixlen 64  scopeid 0x20<link>
        ether 00:0c:29:10:f2:08  txqueuelen 1000  (以太网)
        RX packets 30710  bytes 4236436 (4.2 MB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 1137  bytes 105937 (105.9 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

图 4.3.3.12 ifconfig 命令演示

在图 4.3.3.12 中有两个网卡：ens33 和 lo，ens33 是我的电脑实际使用的网卡，lo 是回测网卡。可以看出网卡 ens33 的 IP 地址为 192.168.1.130，我们使用命令“ifconfig”将网卡 ens33 的 IP 地址改为 192.168.1.253，操作如图 4.3.3.13 所示：

```
zuozhongkai@ubuntu:~$ ifconfig ens33
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.1.253  netmask 255.255.255.0  broadcast 192.168.1.255
        inet6 fe80::62a7:5533:c07c:e7fe  prefixlen 64  scopeid 0x20<link>
        ether 00:0c:29:10:f2:08  txqueuelen 1000  (以太网)
        RX packets 31271  bytes 4287944 (4.2 MB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 1151  bytes 107944 (107.9 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

图 4.3.3.13 修改网卡 IP 地址

从图 4.3.3.13 可以看出，我在使用命令“ifconfig”修改网卡 ens33 的 IP 地址的时候使用了“sudo”，说明在 Ubuntu 下修改网卡 IP 地址是需要 root 用户权限的。当修改完以后使用命令“ifconfig ens33”再次查看网卡 ens33 的命令，发现网卡 ens33 的 IP 地址变成了 192.168.1.253

12、系统帮助命令 man

Ubuntu 系统中有很多命令，这些命令都有不同的格式，不同的格式对应不同的功能，要完全记住这些命令和格式几乎是不可能的，必须有一个帮助手册，当我们需要了解一个命令的详细信息的时候查阅这个帮助手册就行了。Ubuntu 提供了一个命令来帮助用户完成这个功能，那就是“man”命令，通过“man”命令可以查看其它命令的语法格式、主要功能、主要参数说明等，“man”命令格式如下：

man [命令名]

比如我们要查看命令“ifconfig”的说明，输入“man ifconfig”即可，如图 4.3.3.14 所示：

```
zuozhongkai@ubuntu:~$ man ifconfig
```

图 4.3.3.14 man 命令演示

在终端中输入图 4.3.3.14 所示的命令，然后点击回车键就会打开“ifconfig”这个命令的详细说明，如图 4.3.3.15 所示：

```
IFCONFIG(8)          Linux System Administrator's Manual          IFCONFIG(8)

NAME
       ifconfig - configure a network interface

SYNOPSIS
       ifconfig [-v] [-a] [-s] [interface]
       ifconfig [-v] interface [aftype] options | address ...

DESCRIPTION
       Ifconfig  is  used  to configure the kernel-resident network interfaces.
       It is used at boot time to set up interfaces as necessary.  After  that,
       it  is  usually  only  needed  when  debugging  or when system tuning is
       needed.

       If no arguments are given, ifconfig displays the status of the currently
       active interfaces.  If a single interface argument is given, it displays
       the status of the given interface only;  if  a  single  -a  argument  is
       given,  it  displays  the  status of all interfaces, even those that are
       down.  Otherwise, it configures an interface.

Address Families
       If the first argument after the interface name is recognized as the name
 Manual page ifconfig(8) line 1 (press h for help or q to quit)
```

图 4.3.3.15 命令“ifconfig”详细介绍信息

图 4.3.3.15 就是命令“ifconfig”的详细介绍信息，按“q”键退出到终端。

13、 系统重启命令 reboot

通过点击 Ubuntu 主界面右上角的齿轮按钮来选择关机或者重启系统，同样的我们也可以使用 Shell 命令“reboot”来重启系统，直接输入命令“reboot”然后点击回车键接口，如图 4.3.3.16 所示：

```
zuozhongkai@ubuntu:~$ reboot
```

图 4.3.3.16 reboot 命令演示

14、系统关闭命令 poweroff

使用命令“reboot”可以重启系统，使用命令“poweroff”就可以关闭系统，在终端中输入命令“poweroff”然后按下回车键即可关闭 Ubuntu 系统，如图 4.3.3.17 所示：

```
zuozhongkai@ubuntu:~$ poweroff
```

图 4.3.3.17 poweroff 命令演示

15、软件安装命令 install

截至目前，我们都没有讲过 Ubuntu 下如何安装软件，因为 Ubuntu 安装软件不像 Windows 下那样，直接双击.exe 文件就开始安装了。Ubuntu 下很多软件是需要先自行下载源码，下载源

码以后自行编译，编译完成以后使用命令“intsall”来安装。当然 Ubuntu 下也有其它的软件安装方法，但是用的最多的就是自行编译源码然后安装，尤其是嵌入式 Linux 开发。命令“install”格式如下：

```
     install [选项]...  [-T]   源文件        目标文件
或：install [选项]...         源文件...     目录
或：install [选项]...    -t   目录          源文件...
或：install [选项]...    -d   目录...
```

“install”命令是将文件(通常是编译后的文件)复制到目的位置，在前三种形式中，将源文件复制到目标文件或将多个源文件复制到一个已存在的目录中同时设置其所有权和权限模式。在第四种形式会创建指定的目录。命令“install”通常和命令“apt-get”组合在一起使用的，关于“apt-get”命令我们稍后会讲解。

以上就是 Shell 最基本一些命令，还有一些其它的命令我们在后面在讲解，循序渐进嘛。

4.4 APT 下载工具

对于长时间使用 Windows 的我们，下载安装软件非常容易，Windows 下有很多的下载软件，Ubuntu 同样有不少的下载软件，本节我们讲解 Ubuntu 下我们用的最多的下载工具：APT 下载工具，APT 下载工具可以实现软件自动下载、配置、安装二进制或者源码的功能。APT 下载工具和我们前面讲解的“install”命令结合在一起构成了 Ubuntu 下最常用的下载和安装软件方法。它解决了 Linux 平台下一安装软件的一个缺陷，即软件之间相互依赖。

APT 采用的 C/S 模式，也就是客户端/服务器模式，我们的 PC 机作为客户端，当需要下载软件的时候就向服务器请求，因此我们需要知道服务器的地址，也叫做安装源或者更新源。点击 Ubuntu 主界面左上角的“活动”，然后在搜索栏搜索“软件和更新”，如图 4.4.1.1 所示：



图 4.4.1.1 搜索“软件和更新”

打开图 4.4.1.1 中的“软甲和更新”，打开以后如图 4.4.1.2 所示：



图 4.4.1.2 软件和更新设置

在图 4.4.1.2 中的“Ubuntu 软件”选项卡下面的“下载自”就是 APT 工具的安装源，因为我们是在中国，所以需要选择中国的服务器，否则的话可能会导致下载失败！这个也就是网上说的 Ubuntu 安装成功以后要更新源。

在我们使用 APT 工具下载安装或者更新软件的时候，首先会在下载列表中与本机软件对比，看一下需要下载哪些软件，或者升级哪些软件，默认情况下 APT 会下载最新的软件包，被安装的软件包所依赖的其它软件也会被下载安装。说了这么多，APT 下载工具究竟怎么用呢？APT 工具常用的命令如下：

1、更新本地数据库

如果想查看本地哪些软件可以更新的话可以使用如下命令：

```
sudo apt-get update
```

这个命令会访问源地址，并且获取软件列表并保存在本电脑上，过程如图 4.4.1.3 所示：

```
zuozhongkai@ubuntu:~$ sudo apt-get update
[sudo] zuozhongkai 的密码：
获取:1 http://security.ubuntu.com/ubuntu bionic-security InRelease [88.7 kB]
命中:2 http://cn.archive.ubuntu.com/ubuntu bionic InRelease
获取:3 http://cn.archive.ubuntu.com/ubuntu bionic-updates InRelease [88.7 kB]
获取:4 http://security.ubuntu.com/ubuntu bionic-security/main i386 Packages [845 kB]
获取:5 http://cn.archive.ubuntu.com/ubuntu bionic-backports InRelease [74.6 kB]
获取:6 http://cn.archive.ubuntu.com/ubuntu bionic-updates/main i386 Packages [1,140 kB]
获取:7 http://security.ubuntu.com/ubuntu bionic-security/main amd64 Packages [1,397 kB]
获取:8 http://cn.archive.ubuntu.com/ubuntu bionic-updates/main amd64 Packages [1,725 kB]
获取:9 http://security.ubuntu.com/ubuntu bionic-security/main Translation-en [276 kB]
获取:10 http://security.ubuntu.com/ubuntu bionic-security/main amd64 DEP-11 Metadata [49.0 kB]
获取:11 http://security.ubuntu.com/ubuntu bionic-security/universe i386 Packages [964 kB]
获取:12 http://cn.archive.ubuntu.com/ubuntu bionic-updates/main Translation-en [367 kB]
获取:13 http://cn.archive.ubuntu.com/ubuntu bionic-updates/main amd64 DEP-11 Metadata [294 kB]
获取:14 http://security.ubuntu.com/ubuntu bionic-security/universe amd64 Packages [1,078 kB]
获取:15 http://cn.archive.ubuntu.com/ubuntu bionic-updates/restricted amd64 Packages [184 kB]
获取:16 http://cn.archive.ubuntu.com/ubuntu bionic-updates/restricted Translation-en [24.6 kB]
获取:17 http://cn.archive.ubuntu.com/ubuntu bionic-updates/universe i386 Packages [1,544 kB]
获取:18 http://cn.archive.ubuntu.com/ubuntu bionic-updates/universe amd64 Packages [1,680 kB]
获取:19 http://security.ubuntu.com/ubuntu bionic-security/universe Translation-en [241 kB]
获取:20 http://security.ubuntu.com/ubuntu bionic-security/universe amd64 DEP-11 Metadata [58.8 kB]
获取:21 http://security.ubuntu.com/ubuntu bionic-security/universe DEP-11 64x64 Icons [120 kB]
获取:22 http://security.ubuntu.com/ubuntu bionic-security/multiverse amd64 DEP-11 Metadata [2,460 B]
获取:23 http://cn.archive.ubuntu.com/ubuntu bionic-updates/universe Translation-en [353 kB]
获取:24 http://cn.archive.ubuntu.com/ubuntu bionic-updates/universe amd64 DEP-11 Metadata [287 kB]
获取:25 http://cn.archive.ubuntu.com/ubuntu bionic-updates/multiverse amd64 DEP-11 Metadata [2,464 B]
获取:26 http://cn.archive.ubuntu.com/ubuntu bionic-backports/universe amd64 DEP-11 Metadata [9,288 B]
已下载 12.9 MB，耗时 6秒 (2,163 kB/s)
正在读取软件包列表... 完成
```

图 4.4.1.3 更新本地数据库

2、检查依赖关系

有时候本地某些软件可能存在依赖关系，所谓依赖关系就是 A 软件依赖于 B 软件。通过如下命令可以查看依赖关系，如果存在依赖关系的话 APT 会提出解决方案：

```
sudo apt-get check
```

上述命令的执行结果如图 4.4.1.4 所示：

```
zuozhongkai@ubuntu:~$ sudo apt-get check
正在读取软件包列表... 完成
正在分析软件包的依赖关系树
正在读取状态信息... 完成
```

图 4.4.1.4 检查依赖关系

3、软件安装

这个是重点了，安装软件，使用如下命令：

```
sudo apt-get install package-name
```

可以看出上述命令是由“apt-get”和“install”组合在一起的，“package-name”就是要安装的软件名字，“apt-get”负责下载软件，“install”负责安装软件。比如我们要安装软件 Ubuntu 下的串口工具“minicom”，我们就可以使用如下命令：

```
sudo apt-get install minicom
```

执行上述命令以后就会自动下载和安装 minicom 软件，如图 4.4.1.5 所示：

```
zuozhongkai@ubuntu:~$ sudo apt-get install minicom
正在读取软件包列表... 完成
正在分析软件包的依赖关系树
正在读取状态信息... 完成
将会同时安装下列软件：
  lrzsz
下列【新】软件包将被安装：
  lrzsz minicom
升级了 0 个软件包，新安装了 2 个软件包，要卸载 0 个软件包，有 40 个软件包未被升级。
需要下载 313 kB 的归档。
解压缩后会消耗 1,471 kB 的额外空间。
您希望继续执行吗？ [Y/n] y
获取:1 http://cn.archive.ubuntu.com/ubuntu bionic-updates/universe amd64 lrzsz amd64 0.12.21-
10~build0.18.04.1 [74.8 kB]
获取:2 http://cn.archive.ubuntu.com/ubuntu bionic/universe amd64 minicom amd64 2.7.1-1 [239 k
B]
已下载 313 kB，耗时 2秒 (153 kB/s)
正在选中未选择的软件包 lrzsz。
(正在读取数据库 ... 系统当前共安装有 167203 个文件和目录。)
正准备解包 .../lrzsz_0.12.21-10~build0.18.04.1_amd64.deb  ...
正在解包 lrzsz (0.12.21-10~build0.18.04.1) ...
正在选中未选择的软件包 minicom。
正准备解包 .../minicom_2.7.1-1_amd64.deb  ...
正在解包 minicom (2.7.1-1) ...
正在设置 minicom (2.7.1-1) ...
正在设置 lrzsz (0.12.21-10~build0.18.04.1) ...
正在处理用于 gnome-menus (3.13.3-11ubuntu1.1) 的触发器 ...
正在处理用于 mime-support (3.60ubuntu1) 的触发器 ...
正在处理用于 desktop-file-utils (0.23-1ubuntu3.18.04.2) 的触发器 ...
正在处理用于 man-db (2.8.3-2ubuntu0.1) 的触发器 ...
```

图 4.4.1.5 安装 minicom 软件

图 4.4.1.5 就是安装 minicom 这个软件的过程，在图 4.4.1.5 中安装的过程中，会有如下所示询问：

```
您希望继续执行吗？[Y/n]
```

如果希望继续执行的话就输入 y，如果不希望继续执行的话就输入 n。安装完成以后我们直接在终端输入如下命令打开 minicom 这个串口软件：

```
minicom -s
```

打开以后的 minicom 软件如图 4.4.1.6 所示：

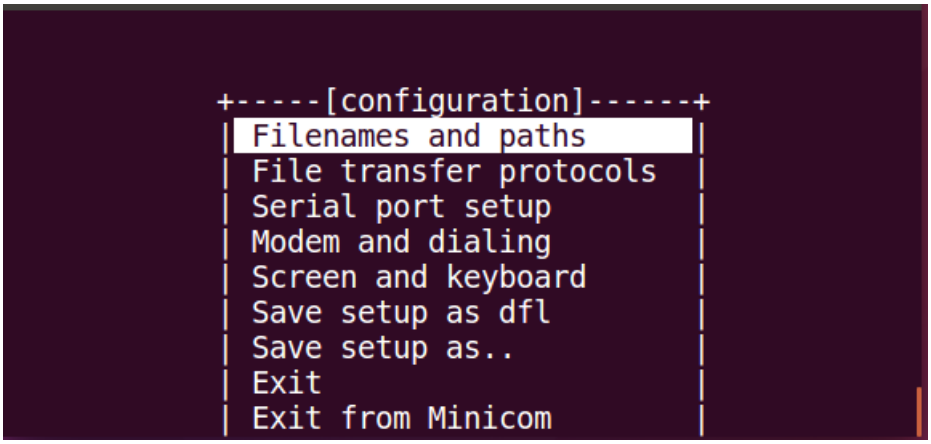


图 4.4.1.6 minicom 软件

关于 minicom 的使用大家可以上网搜索一下，这里就不详细讲解了，要退出 minicom 可以直接按下 ESC 键

4、软件更新

有时候我们需要更新软件，更新软件的话使用命令：

```
sudo apt-get upgrade package-name
```

其中 package-name 为要升级的软件名字，比如我们升级刚刚安装的 minicom 这个软件，如图 4.4.1.7 所示：

```
zuozhongkai@ubuntu:~$ sudo apt-get upgrade minicom
正在读取软件包列表... 完成
正在分析软件包的依赖关系树
正在读取状态信息... 完成
minicom 已经是最新版 (2.7.1-1)。
```

图 4.4.1.7 更新 minicom 软件

从图 4.4.1.7 可以看出，minicom 已经是最新的了，不用更新，不过有其它软件需要更新，因此会自动更新其它的软件。

5、卸载软件

如果要卸载某个软件的话使用如下命令：

```
sudo apt-get remove package-name
```

其中 package-name 是要卸载的软件，比如卸载前面安装的 minicom 这个软件，操作如图 4.4.1.8 所示：

```
zuozhongkai@ubuntu:~$ sudo apt-get remove minicom
正在读取软件包列表... 完成
正在分析软件包的依赖关系树
正在读取状态信息... 完成
下列软件包是自动安装的并且现在不需要了：
  lrzsz
使用'sudo apt autoremove'来卸载它(它们)。
下列软件包将被【卸载】：
  minicom
升级了 0 个软件包，新安装了 0 个软件包，要卸载 1 个软件包，有 0 个软件包未被升级。
解压缩后将会空出 940 kB 的空间。
您希望继续执行吗？ [Y/n] y
(正在读取数据库 ... 系统当前共安装有 167303 个文件和目录。)
正在卸载 minicom (2.7.1-1) ...
正在处理用于 desktop-file-utils (0.23-1ubuntu3.18.04.2) 的触发器 ...
正在处理用于 man-db (2.8.3-2ubuntu0.1) 的触发器 ...
正在处理用于 gnome-menus (3.13.3-11ubuntu1.1) 的触发器 ...
正在处理用于 mime-support (3.60ubuntu1) 的触发器 ...
```

图 4.4.1.8 卸载软件

从图 4.4.1.8 中可以看出软件 minicom 被卸载掉了。关于 APT 下载工具就讲解到这里，我们用的最多的就是“sudo apt-get install package-name”来下载和安装软件。有关 Ubuntu 其它的安装软件的方法打开可以自行上网查阅学习，这里就不一一详解了。

4.5 Ubuntu 下文本编辑

4.5.1 Gedit 编辑器

进行文本编辑是最常用的操作，Windows 下我们会使用记事本来完成，或者其它一些优秀的文本编辑器，比如 notepad++，Ubuut 下有一个自带的文本编辑器，那就是 Gedit。Gedit 是一个窗口式的编辑器，关于 Gedit 的使用前面我们已经讲解了。本节我们重点讲解的是另外一个编辑器：VI/VIM 编辑器。

4.5.2 VI/VIM 编辑器

我们如果要在终端模式下进行文本编辑或者修改文件就可以使用 VI/VIM 编辑器，Ubuntu 自带了 VI 编辑器，但是 VI 编辑器对于习惯了 Windows 下进行开发的人来说不方便，比如竟然不能使用键盘上的上下左右键调整光标位置。因此我推荐大家使用 VIM 编辑器，VIM 编辑器是 VI 编辑器升级版本，VI/VIM 编辑器都是一种基于指令式的编辑器，不需要鼠标，也没有菜单，仅仅使用键盘来完成所有的编辑工作。

我们需要先安装 VIM 编辑器，命令如下：

```
sudo apt-get install vim
```

安装完成以后就可以使用 VIM 编辑器了，VIM 编辑器有 3 种工作模式：输入模式、指令模式和底行模式，通过切换不同的模式可以完成不同的功能，我们就以编辑一个文本文档为例讲解 VIM 编辑器的使用。打开终端，输入命令：vi test.txt，如图 4.5.2.1 所示：

```
zuozhongkai@ubuntu:~$ vim test.txt
```

图 4.5.2.1 新建 test.txt 文档

在终端中输入图 4.5.2.1 中所示的命令以后就会创建一个 test.txt 文档，并且用 VIM 打开了，如图 4.5.2.2 所示：

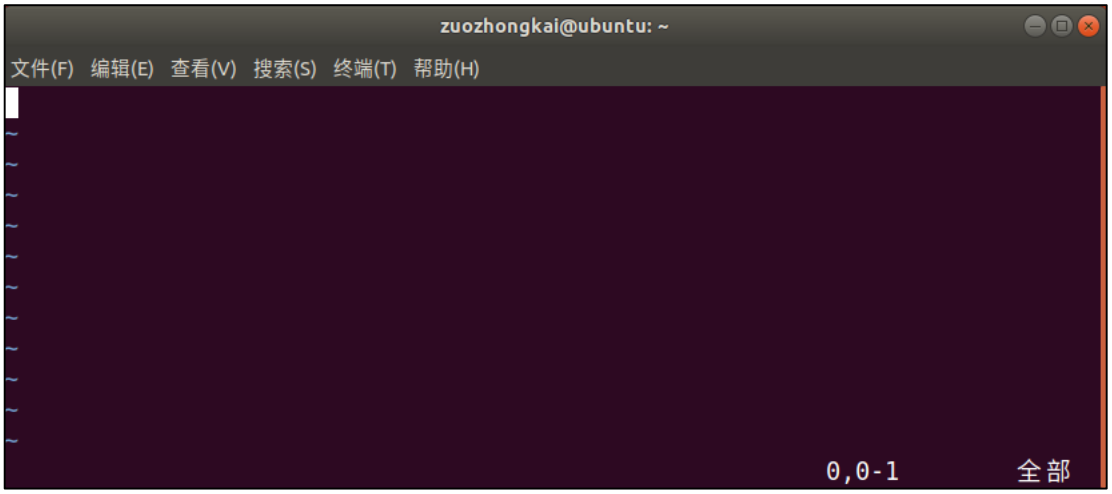


图 4.5.2.2 VIM 打开的 test.txt 文档

我们试着在图 4.5.2.2 中输入数字，发现根本没法输入，这不是因为你的键盘坏了。因为 VIM 默认是以只读模式打开的文档，因此我们要切换到输入模式，切换到输入模式的命令如下：

i ：在当前光标所在字符的前面，转为输入模式。
I ：在当前光标所在行的行首转换为输入模式。
a ：在当前光标所在字符的后面，转为输入模式。
A ：在光标所在行的行尾，转换为输入模式。
o ：在当前光标所在行的下方，新建一行，并转为输入模式。
O ：在当前光标所在行的上方，新建一行，并转为输入模式。
s ：删除光标所在字符。
r ：替换光标处字符。

最常用的就是“a”，我们在图 4.5.2.2 中按下键盘上的“a”键，这时候终端左下角会提示“插入”字样，表示我们进入到了输入模式，如图 4.5.2.3 所示：

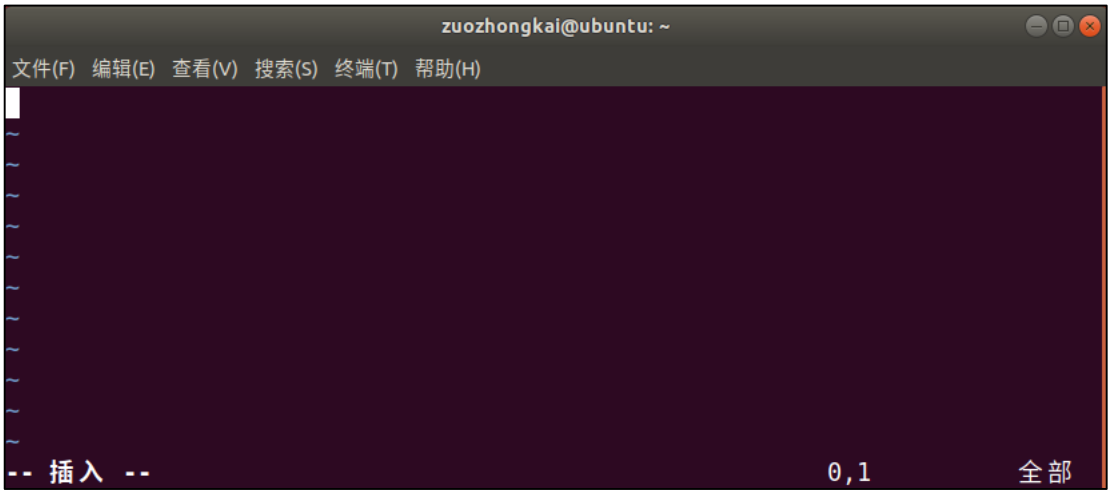


图 4.5.2.3 切换到插入模式

图 4.5.2.3 表明我们可以正常输入文本了，我们可以输入图 4.5.2.4 所示文本：

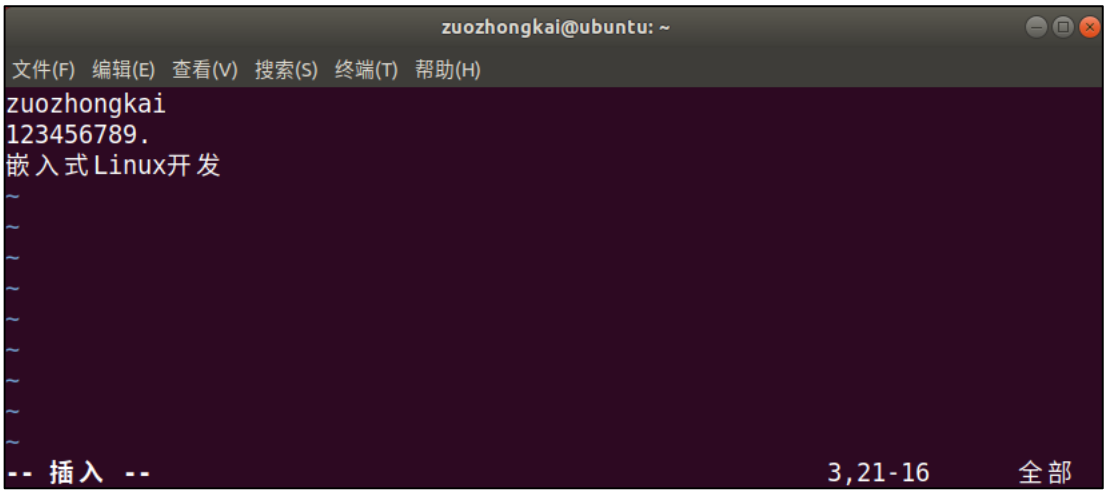


图 4.5.2.4 输入文本

在图 4.5.2.4 中我们在 test.txt 中输入了字母、数字和中文，我们输入完成以后需要保存文本啊，Windows 下的记事本可以使用快捷键 Ctrl+S 来保存，VIM 是否也可以使用 Ctrl+S 来保存呢？你会发现当你按下 Ctrl+S 键以后你的终端不能操作了！！！这是因为在 Ubuntu 下 Ctrl+S 快捷键不是用来完成保存的功能的，而是暂停该终端！所以你一旦在使用终端的时候按下 Ctrl+S 快捷键，那么你的终端肯定不会再有任何反应，如果你按下 Ctrl+S 关闭了当前终端的话可以按下 Ctrl+Q 来重新打开终端。

既然 Ctrl+S 不能保存文本文档，那么有没有其它方法保存文本文档呢？肯定是有的，我们需要从 VIM 现在的输入模式切换到指令模式，方式就是按下键盘的 ESC 键，按下 ESC 键以后终端坐下角的“插入”字样就会消失，此时你就不能在输入任何文本了，如果想再次输入文本的话就按下“a”键重新进入到输入模式。指令模式顾名思义就是输入指令的模式，这些指令是控制文本的指令，我们将这些指令进行分类，如下所示：

1、移动光标指令：

h(或左方向键) ： 光标左移一个字符。
l(或右方向键) ： 光标右移一个字符。
j(或下方向键) ： 光标下移一行。

k(或上方向键) ：光标上移一行。
nG ：光标移动到第 n 行首。
n+ ：光标下移 n 行。
n- ：光标上移 n 行。

2、屏幕翻滚指令

Ctrl+f ：屏幕向下翻一页，相当于下一页。
Ctrl+b ：屏幕向上翻一页，相当于上一页。

3、复制、删除和粘贴指令

cc ：删除整行，并且修改整行内容。
dd ：删除该行，不提供修改功能。
ndd ：删除当前行向下 n 行。
x ：删除光标所在的字符。
X ：删除光标前面的一个字符。
nyy ：复制当前行及其下面 n 行。
p ：粘贴最近复制的内容。

上面就是 VI/VIM 的命令模式下最常用的一些命令，还有一些不常用的我没有列出来，感兴趣的可以自行上网查阅。从上面的命令可以看出，并没有保存文本的命令，那是因为保存文档的命令是在底行模式中，我们要先进入到指令模式，进入底行模式的方式是先进入指令模式下，然后在指令模式下输入“：”进入底行模式，如图 4.5.2.5 所示：

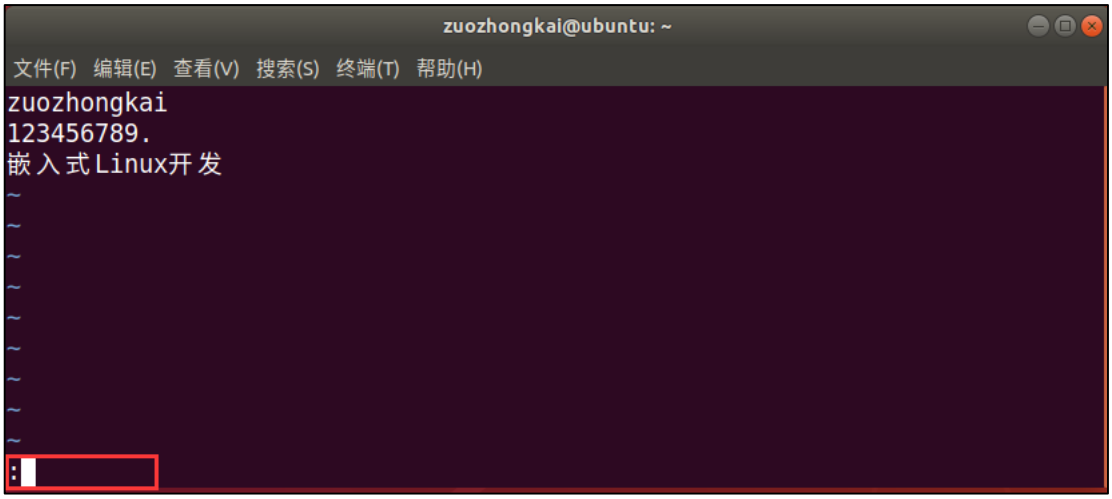


图 4.5.2.5 “:”底行模式

在图 4.5.2.5 中当进入底行模式以后会在终端的左下角就会出现符号“：”，我们可以在“：”后面输入命令，常用的命令如下：

x ：保存当前文档并且退出。
q ：退出。
w ：保存文档。
q! ：退出 VI/VIM，不保存文档。

如果我们要退出并保存文本的话需要在“：”底行模式下输入“wq”，如图 4.5.2.6 所示：

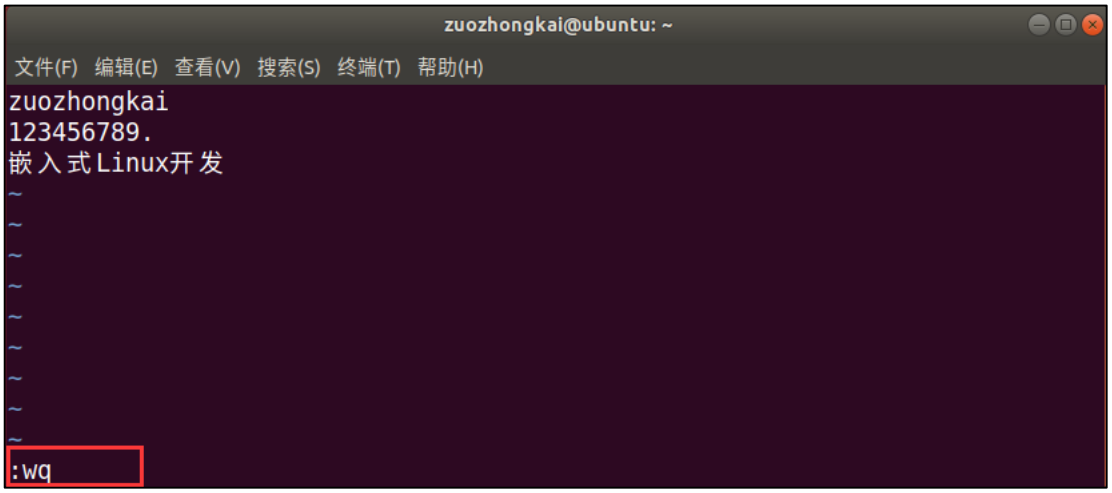


图 4.5.2.6 保存并退出 VIM

在“：”底行模式下输入“wq”以后按下回车键就保存 test.txt 并退出 VI/VIM 编辑器，退出以后我们可以使用命令“cat”来查看刚刚新建的 test.txt 文档的内容，如图 4.5.2.7 所示：

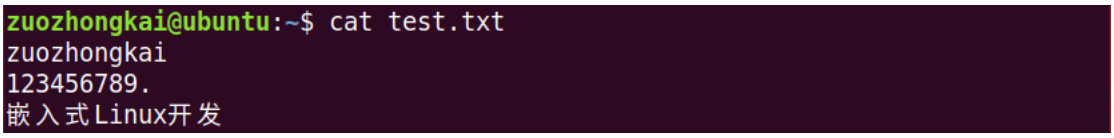


图 4.5.2.7 查看文档内容

从图 4.5.2.7 中可以看出，test.txt 中的内容就是我们用 VIM 输入的内容，至此我们就完整的进行了一遍 VI/VIM 创建文档、编辑文档和保存文档。

在上面讲解进入 VIM 的底行模式的时候之说了在指令模式下输入“：”的方法，还可以在指令模式下输入“/”进入底行模式，输入“/”以后如图 4.5.2.8 所示。

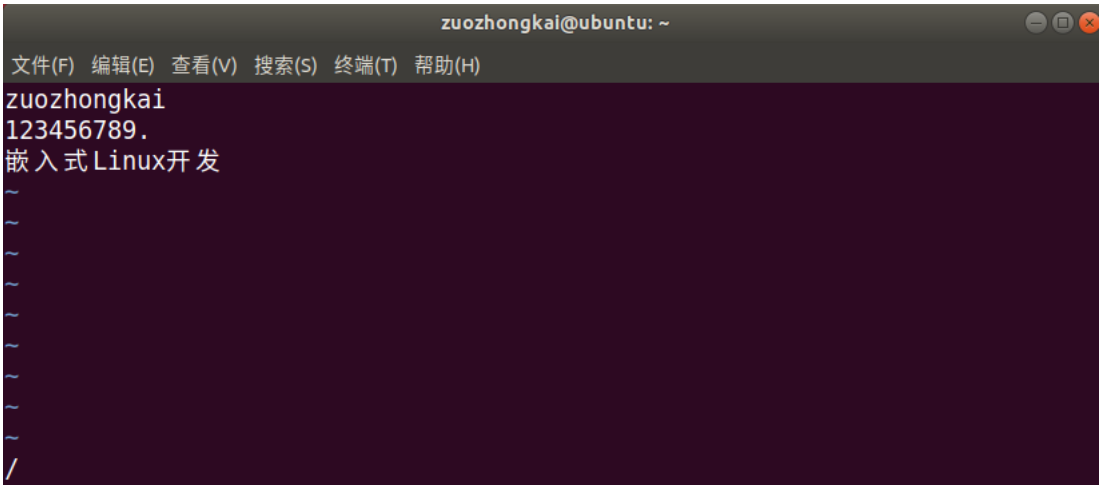


图 4.5.2.8 “/”底行模式

在“/”底行模式下我们可以在文本中搜索指定的内容，比如搜索 test.txt 文件中“嵌入式”三个字，使用方法如图 4.5.2.9 所示：

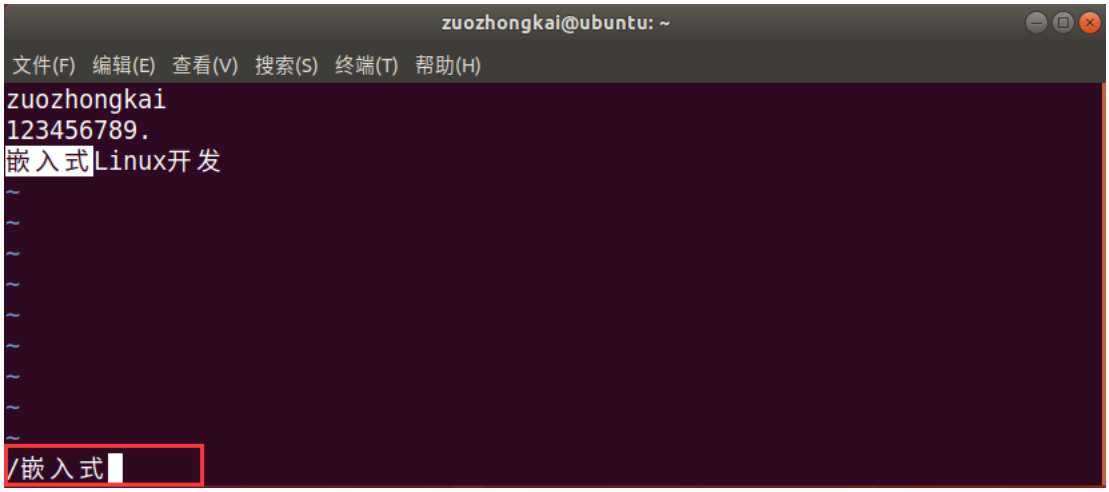


图 4.5.2.9 搜索文本

在“/”后面输入要搜索的内容，然后按下回车键就会在 test.txt 中找到与字符串“嵌入式”匹配的部分，如图 4.5.2.10 所示：

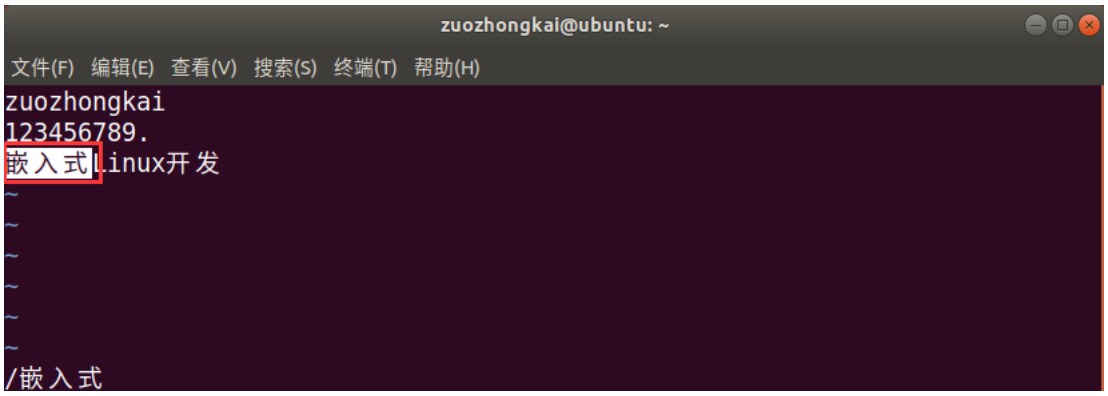


图 4.5.2.10 查找到指定内容

图 4.5.2.10 中可以看出，在 test.txt 中找到了“嵌入式”这个词，并且标记出来位置。我们以后要在一个文档中搜素是否存在某个字符串的时候就可以使用这种方法。有关 VI/VIM 编辑器的讲解就到这里，我们完整的练习了一遍如何使用 VIM 创建文档、编辑文档、保存文档和在文档中搜索字符串。有关更多更详细的 VIM 编辑器的操作大家自行上网查阅相关文档和博客。

4.6 Linux 文件系统

操作系统的基本功能之一就是文件管理，而文件的管理是由文件系统来完成的。Linux 支持多种文件系统，本节我们就来讲解 Linux 下的文件系统、文件系统类型、文件系统结构和文件系统相关 Shell 命令。

4.6.1 Linux 文件系统简介以及类型

1、Linux 文件系统简介

操作系统就是处理各种数据的，这些数据在硬盘上就是二进制，人类肯定不能直接看懂这些二进制数据，要有一个翻译器，将这些二进制的数据还原为人类能看懂的文件形式，这个工作就是由文件系统来完成的，文件系统的目的就是实现数据的查询和存储，由于使用场合、使用环境的不同，Linux 有多种文件系统，不同的文件系统支持不同的体系。文件系统是管理数据

的，而可以存储数据的物理设备有硬盘、U 盘、SD 卡、NAND FLASH、NOR FLASH、网络存储设备等。不同的存储设备其物理结构不同，不同的物理结构就需要不同的文件系统去管理，比如管理 NAND FLASH 的话使用 YAFFS 文件系统，管理硬盘、SD 卡的话就是 ext 文件系统等等。

我们在使用 Windows 的时候新买一个硬盘回来一般肯定是将这个硬盘分为好几个盘，比如 C 盘、D 盘等等。这个叫磁盘的分割，Linux 下也支持磁盘分割，Linux 下常用的磁盘分割工具为：fdisk，fdisk 这个工具我们后面会详细讲解怎么用，因为我们移植 Linux 的时候需要将 SD 卡分为三个分区来存储不同的东西。在 Windows 下我们创建一个新的盘符以后都要做格式化处理，格式化其实就是给这个盘符创建文件系统的过程，我们在 Windows 格式化某个盘的时候都会让你选择文件系统，如图 2.6.1.1 所示：

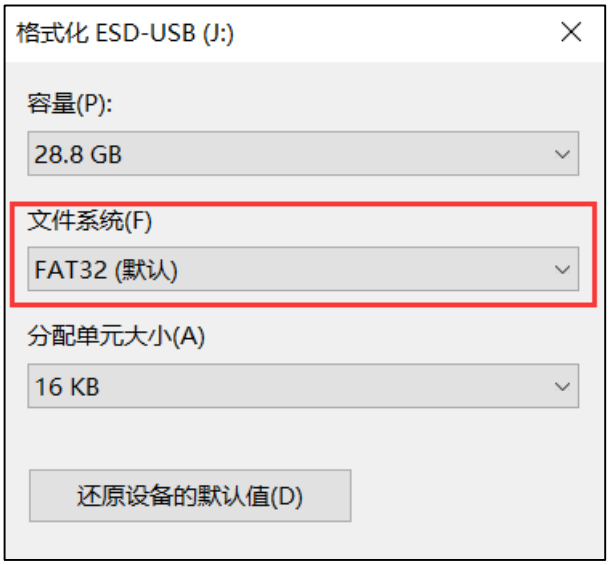


图 2.6.1.1 格式化磁盘

图 4.6.1.1 就是格式化磁盘的时候选择文件系统，Windows 下一般有 FAT、NTFS 和 exFAT 这些文件系统。同样的，在 Linux 下我们使用 fdisk 创建好分区以后也是要先在创建好的分区上面创建文件系统，也就是格式化。

在 Windows 下有磁盘分区的概念，比如 C，D，E 盘等，在 Linux 下没有这个概念，因此 Linux 下你找不到像 C、D、E 盘这样的东西。前面我们说了 Linux 下可以给磁盘分割，但是没有 C、D、E 盘那怎么访问这些分区呢？在 Linux 下创建一个分区并且格式化好以后我们要将其“挂载”到一个目录下才能访问这个分区。Windows 的文件系统挂载过程是其内部完成的，用户是看不到的，Linux 下我们使用 mount 命令来挂载磁盘。挂载磁盘的时候是需要确定挂载点的，也就是你的这个磁盘要挂载到哪个目录下。

2、Linux 文件系统类型

前面我们说了，在 Windows 下有 FAT、NTFS 和 exFAT 这样的文件系统，在 Linux 下又有哪些文件系统呢，Linux 下的文件系统主要有 ext2、ext3、ext4 等文件系统。Linux 还支持其他的 UNIX 文件系统，比如 XFS、JFS、UFS 等，也支持 Windows 的 FAT 文件系统和网络文件系统 NFS 等。这里我们主要讲一下 Linux 自带的 ext2、ext3 和 ext4 文件系统。

ext2 文件系统：

ext2 是 Linux 早期的文件系统，但是随着技术的发展 ext2 文件系统已经不推荐使用了，ext2 是一个非日志文件系统,大多数的 Linux 发行版都不支持 ext2 文件系统了。

ext3 文件系统：

ext3 是在 ext2 的基础上发展起来的文件系统，完全兼容 ext2 文件系统，ext3 是一个日志文件系统，ext3 支持大文件，ext3 文件系统的特点有如下：

高可靠性：使用 ext3 文件系统的话，即使系统非正常关机、发生死机等情况，恢复 ext3 文件系统也只需要数十秒。

数据完整性：ext3 提高了文件系统的完整性，避免意外死机或者关机对文件系统的伤害。

文件系统速度：ext3 的日志功能对磁盘驱动器读写头进行了优化，文件系统速度相对与 ext2 来说没有降低。

数据转换：从 ext2 转换到 ext3 非常容易，只需要两条指令就可以完成转换。用户不需要花时间去备份、恢复、格式化分区等，用 ext3 文件系统提供的工具 tune2fs 即可轻松的将 ext2 文件系统转换为 ext3 日志文件系统。ext3 文件系统不需要经过任何修改，可以直接挂载成 ext2 文件系统。

ext4 文件系统：

ext4 文件系统是在 ext3 上发展起来的，ext4 相比与 ext3 提供了更佳的性能和可靠性，并且功能更丰富，ext4 向下兼容 ext3 和 ext2，因此可以将 ext2 和 ext3 挂载为 ext4。那么我们安装的 Ubuntu 使用的哪个版本的文件系统呢？在终端中输入如下命令来查询当前磁盘挂载的啥文件系统：

```
df  -T –h
```

结果如图 4.6.1.2 所示：



图 4.6.1.2 Ubuntu 使用的文件系统

在图 4.6.1.2 中，框起来的就是我们安装 Ubuntu 的这个磁盘，在 Linux 下一切皆为文件，“/dev/sda1”就是我们的磁盘分区，可以看出这个磁盘分区类型是 ext4，它的挂载点是“/”，也就是根目录。

4.6.2 Linux 文件系统结构

在 Windows 下直接打开 C 盘，我们进入的就是 C 盘的根目录，打开 D 盘进入的就是 D 盘的根目录，比如 C 盘根目录如下：



图 2.6.2.1 C 盘根目录

在 Linux 下因为没有 C、D 盘之说，因此 Linux 只有一个根目录，没有 C 盘根目录、D 盘根目录之类的。其实如果你的 Windows 只有一个 C 盘的话那么整个系统也就只有一个根目录。

Windows 下的 C 盘根目录就是“C:”,在 Linux 下的根目录就是“/”，你没有看错，Linux 根目录就是用“/”来表示的，打开 Ubuntu 的文件浏览器，文件浏览器在左侧的导航栏，图标如图 4.6.2.2 所示：



图 4.6.2.2 文件浏览器

打开以后的文件浏览器如图 4.6.2.3 所示：

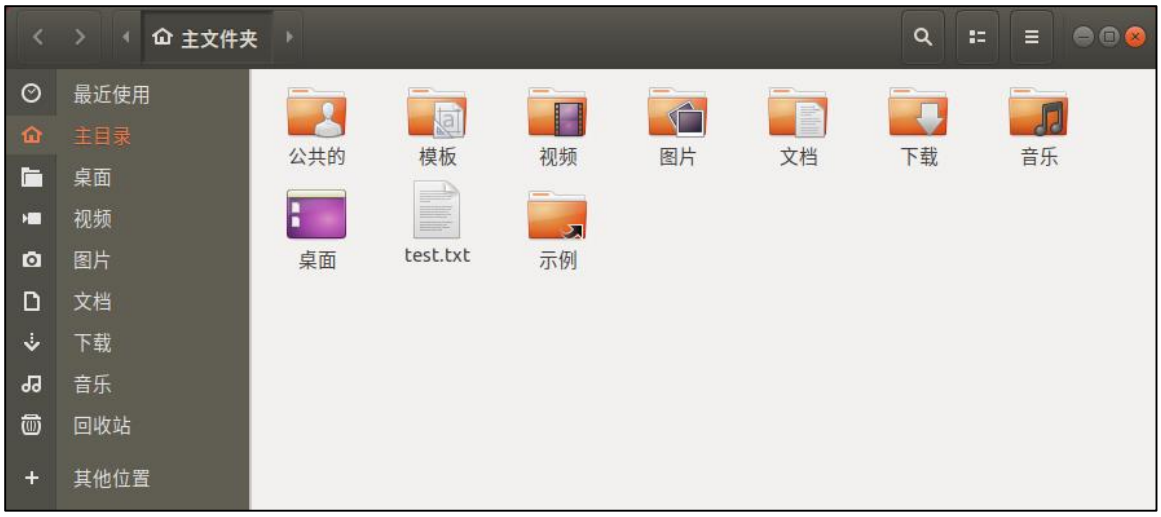


图 4.6.2.3 文件浏览器

直接打开文件浏览器以后，我们默认不是处于根目录中的，不像 Windows，我们直接打开 C 盘就处于 C 盘根目录下。Ubuntu 是支持多用户的，Ubuntu 为每个用户创建了一个根目录，比

如我电脑现在登陆的是“zuozhongkai”这个用户，因此默认进入的是“zuozhongkai”这个用户的根目录。我们点击图 4.6.2.3 中左侧的“其他位置”，选择“计算机”，如图 4.6.2.4 所示：



图 4.6.2.4 打开“计算机”

打开以后如图 4.6.2.5 所示：

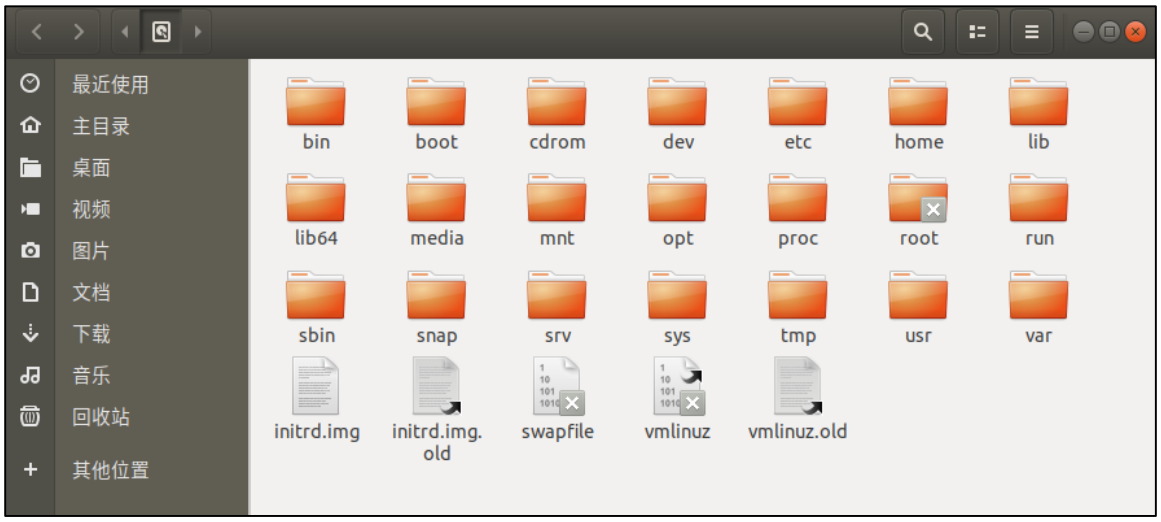


图 4.6.2.5 根目录“/”

图 4.6.2.5 就是 Ubuntu 的根目录“/”，这时候肯定就有人有疑问，刚刚说 Ubuntu 会给每个用户创建一个根目录，那这些用户的根目录在哪里？是不是和根目录“/”是一个地位的？这个所谓的用户根目录其实就是“/”下的一个文件夹，以我的“zuozhongkai”这个用户为例，其用户根目录就是：/home/zuozhongkai。只要你创建了一个用户，那么系统就会在/home 这个目录下创建一个以这个用户名命名的文件夹，这个文件夹就是这个用户的根目录。

用户可以对自己的用户根目录下的文件进行随意的读写操作，但是如果要修改根目录“/”下的文件就会提示没有权限。打开终端以后默认进入的是当前用户根目录，比如我们打开终端以后输入“ls”命令查看当前目录下有什么文件，结果如图 4.6.2.6 所示：

```
zuozhongkai@ubuntu:~$ ls
examples.desktop  test.txt  公共的  模板  视频  图片  文档  下载  音乐  桌面
```

图 4.6.2.6 目录查看

可以看出图 4.6.2.6 中的文件和图 4.6.2.3 中的一模一样，都是“zuozhongkai”这个账户的根目录。我们来看一下根目录“/”下都有哪些文件，在终端中输入如下命令：

```
cd  /     //进入到根目录“/
ls        //查看根目录“/”下的文件以及文件夹
```

执行上述两行命令以后，终端如图 4.6.2.7 所示：

```
zuozhongkai@ubuntu:~$ cd /
zuozhongkai@ubuntu:/$ ls
bin   dev   initrd.img      lib64       mnt   root  snap      sys  var
boot  etc   initrd.img.old  lost+found  opt   run   srv       tmp  vmlinuz
cdrom home  lib             media       proc  sbin  swapfile  usr  vmlinuz.old
```

图 4.6.2.7 查看根目录“/”

图 4.6.2.7 中列举出了根目录“/”下面的所有文件夹，这里我们仔细观察一下，当我们进入到根目录“/”里面以后终端提示符“$”前面的符号“~”变成了“/”，这是因为当我们在终端中切换了目录以后“$”前面就会显示切换以后的目录路径。我们来看一下根目录“/”中的一些重要的文件夹：

/bin ：存储一些二进制可执行命令文件，/usr/bin 也存放了一些基于用户的命令文件。
/sbin ：存储了很多系统命令，/usr/sbin 也存储了许多系统命令。
/root ：超级用户 root 的根目录文件。
/home ：普通用户默认目录，在该目录下，每个用户都有一个以本用户名命名的文件夹。
/boot ：存放 Ubuntu 系统内核和系统启动文件。
/mnt ：通常包括系统引导后被挂载的文件系统的挂载点。
/dev ：存放设备文件，我们后面学习 Linux 驱动主要是跟这个文件夹打交道的。
/etc ：保存系统管理所需的配置文件和目录。
/lib ：保存系统程序运行所需的库文件，/usr/lib 下存放了一些用于普通用户的库文件。
/lost+found ：一般为空，当系统非正常关机以后，此文件夹会保存一些零散文件。
/var ：存储一些不断变化的文件，比如日志文件
/usr ：包括与系统用户直接有关的文件和目录，比如应用程序和所需的库文件。
/media ：存放 Ubuntu 系统自动挂载的设备文件。
/proc ：虚拟目录，不实际存储在磁盘上，通常用来保存系统信息和进程信息。
/tmp ：存储系统和用户的临时文件，该文件夹对所有的用户都提供读写权限。
/opt ：可选文件和程序的存放目录。
/sys ：系统设备和文件层次结构，并向用户程序提供详细的内核数据信息。

4.6.3 文件操作命令

本节我们来学习一下在终端进行文件操作的一些常用命令：

1、创建新文件命令—touch

在前面学习 VIM 的时候我们知道可以用 vi 指令来创建一个文本文档，本节我们就学习一个功能更全面的文件创建命令—touch。touch 不仅仅可以用用来创建文本文档，其它类型的文档也可以创建，命令格式如下：

```
touch  [参数]  [文件名]
```

使用 touch 创建文件的时候，如果[文件名]的文件不存在，那就直接创建一个以[文件名]命名的文件，如果[文件名]文件存在的话就仅仅修改一下此文件的最后修改日期，常用的命令参数如下：

-a ：只更改存取时间。
-c ：不建立任何文件。
-d<日期> ：使用指定的日期，而并非现在日期。
-t<时间> ：使用指定的时间，而并非现在时间。

进入到用户根目录下，直接使用命令“cd~”即可快速进入用户根目录，进入用户根目录以后使用 touch 命令创建一个名为 test 的文件，创建过程如图 4.6.2.1 所示：

```
zuozhongkai@ubuntu:~$ cd ~   //进入用户根目录
zuozhongkai@ubuntu:~$ ls     //查看当前目录下的文件
examples.desktop  tmp      模板   图片   下载   桌面
test.txt          公共的   视频   文档   音乐
zuozhongkai@ubuntu:~$ touch test //创建test文件
zuozhongkai@ubuntu:~$ ls test    //查看创建的test文件
test
zuozhongkai@ubuntu:~$ ls test -l //查看创建的test文件详细信息
-rw-rw-r-- 1 zuozhongkai zuozhongkai 0 12月  22 01:24 test
```

图 4.6.3.1 touch 命令操作

2、文件夹创建命令—mkdir

既然可以创建文件，那么肯定也可以创建文件夹，创建文件夹使用命令“mkdir”，命令格式如下：

```
mkdir   [参数]   [文件夹名目录名]
```

主要参数如下：

-p ：如所要创建的目录其上层目录目前还未创建，那么会一起创建上层目录。

我们在用户根目录下创建两个分别名为“testdir1”和“testdir2”的文件夹，操作如图 4.6.2.2 所示：

```
zuozhongkai@ubuntu:~$ ls  //查看当前目录下的文件
examples.desktop  test.txt  公共的   视频   文档   音乐
test              tmp       模板     图片   下载   桌面
zuozhongkai@ubuntu:~$ mkdir testdir1 //创建testdir1文件夹
zuozhongkai@ubuntu:~$ mkdir testdir2 //创建testdir2文件夹
zuozhongkai@ubuntu:~$ ls             //查看当前目录下的所有文件，看看文件夹创建是否成功
examples.desktop  testdir1  test.txt  公共的   视频   文档   音乐
test              testdir2  tmp       模板     图片   下载   桌面
```

图 4.6.3.2 创建文件夹

在图 4.6.3.2 中，我们使用命令“mkdir”创建了“testdir1”和“testdir2”这两个文件夹。

3、文件及目录删除命令—rm

既然有创建文件的命令，那肯定有删除文件的命令，要删除一个文件或者文件夹可以使用命令“rm”，此命令可以完成删除一个文件或者多个文件及文件夹，它可以实现递归删除。对于链接文件，只删除链接，原文件保持不变，所谓的链接文件，其实就是 Windows 下的快捷方式文件，此命令格式如下：

```
rm  [参数]   [目的文件或文件夹目录名]
```

命令主要参数如下：

-d ：直接把要删除的目录的硬连接数据删成 0，删除该目录。

-f ：强制删除文件和文件夹(目录)。

-i ：删除文件或者文件夹(目录)之前先询问用户。

-r ：递归删除，指定文件夹(目录)下的所有文件和子文件夹全部删除掉。

-v ：显示删除过程。

我们使用 rm 命令来删除前面使用命令“touch”创建的 test 文件，操作过程如图 4.6.2.3 所示：

```
zuozhongkai@ubuntu:~$ ls //查看当前目录下的所有文件
examples.desktop  testdir1  test.txt  公共的  视频  文档  音乐
test              testdir2  tmp       模板    图片  下载  桌面
zuozhongkai@ubuntu:~$ rm test //删除文件test
zuozhongkai@ubuntu:~$ ls      //查看当前目录，看看test文件是否删除
examples.desktop  testdir2  tmp     模板  图片  下载  桌面
testdir1          test.txt  公共的  视频  文档  音乐
```

图 4.6.3.3 删除文件

命令“rm”也可以直接删除文件夹，我们可以试一下删除前面创建的 testdir1 文件夹，先直接使用命令“rm testdir1”测试一下是否可以删除，结果如图 4.6.2.4 所示：

```
zuozhongkai@ubuntu:~$ rm testdir1
rm: 无法删除'testdir1': 是一个目录
```

图 4.6.2.4 删除文件夹

在图 4.6.3.4 中可以看出，直接使用命令“rm”是无法删除文件夹(目录)的，我们需要加上参数“-rf”，也就是强制递归删除文件夹(目录)，操作结果如图 4.6.3.5 所示：

```
zuozhongkai@ubuntu:~$ ls //查看当前目录下的所有文件
examples.desktop  testdir2  tmp     模板  图片  下载  桌面
testdir1          test.txt  公共的  视频  文档  音乐
zuozhongkai@ubuntu:~$ rm testdir1 -rf //使用参数“-rf”强制递归删除文件夹
zuozhongkai@ubuntu:~$ ls              //查看删除文件夹以后的当前目录
examples.desktop  test.txt  公共的  视频  文档  音乐
testdir2          tmp       模板    图片  下载  桌面
```

图 4.6.3.5 带参数删除文件夹

从图 4.6.2.5 可以看出，当在命令“rm”中加入参数“-rf”以后就可以删除掉文件夹“testdir1”了。

4、文件夹(目录)删除命令—rmdir

上面我们讲解了如何使用命令“rm”删除文件夹，那就是要加上参数“-rf”，其实 Linux 提供了直接删除文件夹(目录)的命令—rmdir，它可以不加任何参数的删除掉指定的文件夹(目录)，命令格式如下：

```
rmdir [参数] [文件夹(目录)]
```

命令主要参数如下：

-p 删除指定的文件夹(目录)以后，若上层文件夹(目录)为空文件夹(目录)的话就将其一起删除。

我们使用命令“rmdir”删除掉前面创建的“testdir2”文件夹，操作过程如图 4.6.3.6 所示：

```
zuozhongkai@ubuntu:~$ ls   //查看当前目录下的所有文件
examples.desktop  test.txt  公共的  视频  文档  音乐
testdir2          tmp       模板    图片  下载  桌面
zuozhongkai@ubuntu:~$ rmdir testdir2 //删除文件夹testdir2
zuozhongkai@ubuntu:~$ ls                //查看删除文件夹以后的目录文件
examples.desktop  tmp     模板  图片  下载  桌面
test.txt          公共的  视频  文档  音乐
```

图 4.6.3.6 命令 rmdir 删除文件夹

5、文件复制命令—cp

在 Windows 下我们可以通过在文件上点击鼠标右键来进行文件的复制和粘贴，在 Ubuntu 下我们也可以通过点击文件右键进行文件的复制和粘贴。但是本节我们来讲解如何在终端下使用命令来进行文件的复制，Linux 下的复制命令为“cp”，命令描述如下：

```
cp [参数] [源地址] [目的地址]
```

主要参数描述如下：

-a ：此参数和同时指定“-dpR”参数相同

-d ：在复制有符号连接的文件时，保留原始的连接。

-f ：强行复制文件，不管要复制的文件是否已经存在于目标目录。

-I ：覆盖现有文件之前询问用户。

-p ：保留源文件或者目录的属性。

-r 或-R ：递归处理，将指定目录下的文件及子目录一并处理

我们在用户根目录下，使用前面讲解的命令“mkdir”创建两个文件夹：test1 和 test2，过程如图 4.6.3.7 所示。

```
zuozhongkai@ubuntu:~$ ls  //查看当前目录下的文件
examples.desktop  tmp     模板  图片  下载  桌面
test.txt          公共的  视频  文档  音乐
zuozhongkai@ubuntu:~$ mkdir test1 test2 //创建test1和test2两个文件夹
zuozhongkai@ubuntu:~$ ls                  //查看当前目录下的所有文件
examples.desktop  test2     tmp     模板  图片  下载  桌面
test1             test.txt  公共的  视频  文档  音乐
```

图 4.6.3.7 创建 test1 和 test2 两个文件夹

进入上面创建的 test1 文件夹，然后在 test1 文件夹里面创建一个 a.c 文件，操作过程如图 4.6.3.8 所示：

```
zuozhongkai@ubuntu:~$ cd test1          //进入test1文件夹
zuozhongkai@ubuntu:~/test1$ touch a.c //创建a.c文件
zuozhongkai@ubuntu:~/test1$ ls          //查看当前目录下所有文件
a.c
```

图 4.6.2.8 创建 a.c 文件

我们先将图 4.6.3.8 中的 a.c 这个文件做个备份，也就是复制到同文件夹 test1 里面，新的文件命名为 b.c。然后在将 test1 文件夹中的 a.c 和 b.c 这两个文件都复制到文件夹 test2 中，操作如图 4.6.3.9 所示

```
zuozhongkai@ubuntu:~/test1$ cp a.c b.c      //拷贝a.c到本文件夹中，并重命名为b.c
zuozhongkai@ubuntu:~/test1$ ls              //查看当前目录下的所有文件
a.c  b.c
zuozhongkai@ubuntu:~/test1$ cp *.c ../test2 //拷贝a.c和b.c到文件夹test2中
zuozhongkai@ubuntu:~/test1$ cd ../test2     //进入上级目录中的test2文件夹
zuozhongkai@ubuntu:~/test2$ ls              //查看test2文件夹下的文件
a.c  b.c
```

图 4.6.3.9 拷贝文件

在图 4.6.3.9 中，我们添加了一些高级使用技巧，首先是拷贝 a.c 和 b.c 文件到 test2 文件夹中，我们使用了通配符“*”，“*.c”就表示 test1 下的所有以“.c”结尾的文件，也就是 a.c 和 b.c。“../test2”中的“../”表示上级目录，因此“../test2”就是上级目录下的 test2 文件夹。

上面都是文件复制，我们接下来学习一下文件夹复制，我们将 test2 文件夹复制到同目录下，新拷贝的文件夹命名为 test3，操作如图 4.6.3.10 所示：

```
zuozhongkai@ubuntu:~$ cp -rf test2/ test3/  //复制test2文件夹为test3文件夹
zuozhongkai@ubuntu:~$ ls                    //查看当前目录下的所有文件
examples.desktop  test2  test.txt  公共的  视频  文档  音乐
test1             test3  tmp       模板    图片  下载  桌面
zuozhongkai@ubuntu:~$ cd test3              //进入test3文件夹
zuozhongkai@ubuntu:~/test3$ ls              //查看test3中的所有文件
a.c  b.c
```

图 4.6.2.10 拷贝文件夹

6、文件移动命令—mv

有时候我们需要将一个文件或者文件夹移动到另外一个地方去，或者给一个文件或者文件夹进行重命名，这个时候我们就可以使用命令“mv”了，此命令格式如下：

```
mv [参数] [源地址] [目的地址]
```

主要参数描述如下：

-b ：如果要覆盖文件的话覆盖前先进行备份。

-f ：若目标文件或目录与现在的文件重复，直接覆盖目的文件或目录。

-I ：在覆盖之前询问用户。

使用上面讲解“cp”命令的时候创建了三个文件夹，在上面创建的 test1 文件夹里面创建一个 c.c 文件，然后将 c.c 这个文件重命名为 d.c。最后将 d.c 这个文件移动到 test2 文件夹里面，操作如图 4.6.3.11 所示：

```
zuozhongkai@ubuntu:~$ cd test1              //进入test1文件夹
zuozhongkai@ubuntu:~/test1$ touch c.c       //创建c.c文件
zuozhongkai@ubuntu:~/test1$ ls              //查看当前目录下的所有文件
a.c  b.c  c.c
zuozhongkai@ubuntu:~/test1$ mv c.c d.c      //移动c.c到当前目录下，并重命名为d.c
zuozhongkai@ubuntu:~/test1$ ls              //查看当前目录下的所有文件
a.c  b.c  d.c
```

图 4.6.3.11 移动文件操作

我们再将 test1 中的 d.c 文件移动到 test2 文件夹里面，操作如图 4.6.3.12 所示：

```
zuozhongkai@ubuntu:~/test2$ ls          //查看test2下所有文件
a.c  b.c
zuozhongkai@ubuntu:~/test2$ cd ../test1    //进入test1文件夹中
zuozhongkai@ubuntu:~/test1$ ls             //查看test1文件夹所有文件
a.c  b.c  d.c
zuozhongkai@ubuntu:~/test1$ mv d.c ../test2 //将d.c移动到test2中
zuozhongkai@ubuntu:~/test1$ ls ../test2     //查看test2下的所有文件
a.c  b.c  d.c
```

图 4.6.3.12 移动文件操作

4.6.4 文件压缩和解压缩

文件的压缩和解压缩是非常常见的操作，在 Windows 下我们有很多压缩和解压缩的工具，比如 zip、360 压缩等等。在 Ubuntu 下也有压缩工具，本节我们学习 Ubuntu 下图形化以及命令行这两种压缩和解压缩操作。

1、图形化压缩和解压缩

图形化压缩和解压缩和 Windows 下基本一样，在要压缩或者解压的文件上点击鼠标右键，然后选择要进行的操作，我们先讲解一下如何进行文件的压缩。首先找到要压缩的文件，然后在要压缩的文件上点击鼠标右键，选择“压缩”选项，如图 4.6.4.1 所示：

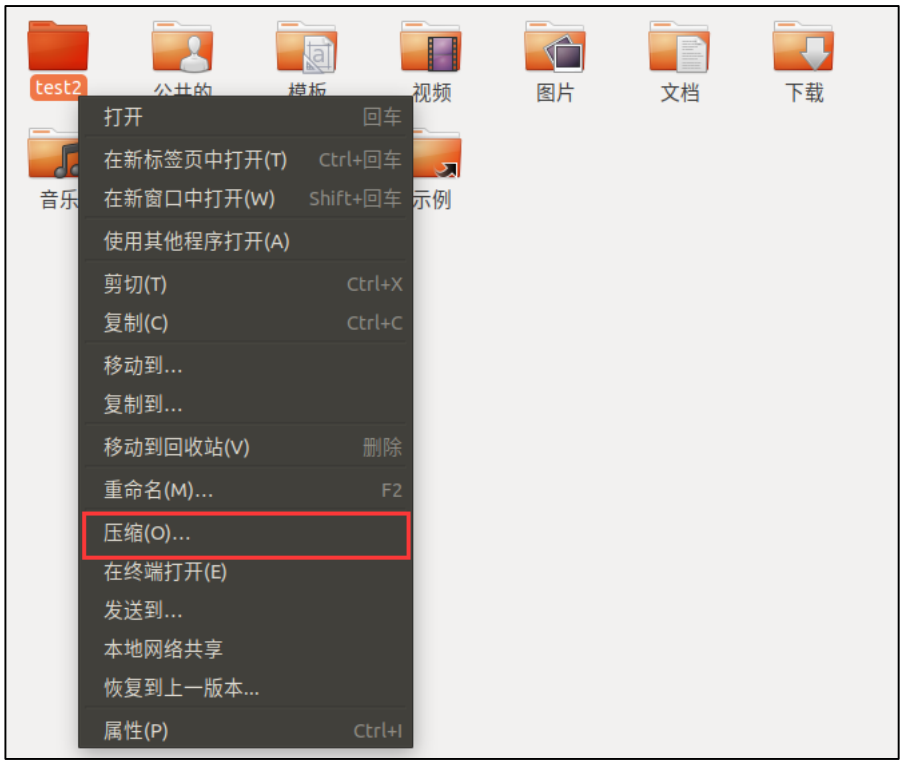


图 4.6.4.1 文件压缩

在图 4.6.4.1 中我们要对 test 这个文件夹进行压缩，点击“压缩”以后会弹出图 4.6.4.2 所示界面让选择压缩后的文件名和压缩格式。



图 4.6.4.2 压缩命名和格式选择

在图 4.6.4.2 中，设置好压缩以后的文件名，然后选择压缩格式，比如我选择的“.zip”这个格式，点击图 4.6.4.2 右上角的“创建”，压缩完成以后如图 4.6.4.3 所示：



图 4.6.4.3 压缩完成的文件

上面就是使用图形化进行文件压缩的过程，我们接下来对刚刚压缩的 test2.zip 进行解压缩，鼠标放到 test2.zip 上然后点击鼠标右键，选择“提取到此处”，如图 4.6.4.4 所示：



图 4.6.4.4 解压缩文件

点击图 4.6.4.4 中的“提取到此处”以后，系统就会自动进行解压缩，上面就是在 Ubuntu 中使用图形化工具进行文件的压缩和解压缩。

2、命令行进行文件的压缩和解压缩

上面我们学习了如何使用图形化工具在 Ubuntu 下进行文件的压缩和解压缩，本节我们学学如何使用命令行进行压缩和解压缩，我们后面的开发中所有涉及到压缩和解压缩的操作都是

在命令行下完成的。命令行下进行压缩和解压缩常用的命令有三个：zip、unzip 和 tar，我们依次来学习：

3、命令 zip

zip 命令看名字就知道是针对.zip 文件的，用于将一个或者多个文件压缩成一个.zip 结尾的文件，命令格式如下：

```
zip [参数] [压缩文件名.zip] [被压缩的文件]
```

主要参数函数如下：

-b<工作目录> 指定暂时存放文件的目录。

-d ：从 zip 文件中删除一个文件。

-F ：尝试修复已经损毁的压缩文件。

-g ：将文件压缩入现有的压缩文件中，不需要新建压缩文件。

-h ：帮助。

-j ：只保存文件的名，不保存目录。

-m ：压缩完成以后删除源文件。

-n<字尾符号> ：不压缩特定扩展名的文件。

-q ：不显示压缩命令执行过程。

-r ：递归压缩，将指定目录下的所有文件和子目录一起压缩。

-v ：显示指令执行过程。

-num ：压缩率，为 1~9 的数值。

上面讲解了如何使用图形化压缩工具对文件夹 test2 进行压缩，这里我们使用命令“zip”对 test2 文件夹进行压缩，操作如图 4.6.4.5 所示：

```
zuozhongkai@ubuntu:~$ ls  //显示当前目录下所有文件
examples.desktop  test2  test.txt  公共的  视频  文档  音乐
test1             test3  tmp       模板    图片  下载  桌面
zuozhongkai@ubuntu:~$ zip -rv test2.zip test2  //使用zip命令进行压缩
  adding: test2/         (in=0) (out=0) (stored 0%)
  adding: test2/d.c      (in=0) (out=0) (stored 0%)
  adding: test2/b.c      (in=0) (out=0) (stored 0%)
  adding: test2/a.c      (in=0) (out=0) (stored 0%)
total bytes=0, compressed=0 -> 0% savings
zuozhongkai@ubuntu:~$ ls                          //显示压缩以后的当前目录下所有文件
examples.desktop  test2      test3     tmp     模板  图片  下载  桌面
test1             test2.zip  test.txt  公共的  视频  文档  音乐
```

图 4.6.4.5 使用 zip 进行文件压缩

图 4.6.4.5 就是使用 zip 命令进行 test2 文件夹的压缩，我们使用的命令如下：

```
zip -rv test2.zip test2
```

上述命令中，-rv 表示递归压缩并且显示压缩命令执行过程。

4、命令 unzip

unzip 命令用于对.zip 格式的压缩包进行解压，命令格式如下：

```
unzip [参数] [压缩文件名.zip]
```

主要参数如下：

-l ：显示压缩文件内所包含的文件。

-t ：检查压缩文件是否损坏，但不解压。

-v ：显示命令显示的执行过程。

-Z ：只显示压缩文件的注解。

-C ：压缩文件中的文件名称区分大小写。
-j ：不处理压缩文件中的原有目录路径。
-L ：将压缩文件中的全部文件名改为小写。
-n ：解压缩时不要覆盖原有文件。
-P<密码> ：解压密码。
-q ：静默执行，不显示任何信息。
-x<文件列表> ：指定不要处理.zip 中的哪些文件。
-d<目录> ：把压缩文件解到指定目录下。
对上面压缩的 test2.zip 文件使用 unzip 命令进行解压缩，操作如图 4.6.4.6 所示：

```
zuozhongkai@ubuntu:~$ rm -rf test2 //删除以前的test2文件夹，防止干扰解压过程
zuozhongkai@ubuntu:~$ ls            //显示当前目录下所有文件
examples.desktop  test2.zip  test.txt  公共的  视频  文档  音乐
test1             test3      tmp       模板    图片  下载  桌面
zuozhongkai@ubuntu:~$ unzip test2.zip  //解压缩test2.zip文件
Archive:  test2.zip
   creating: test2/
 extracting: test2/d.c
 extracting: test2/b.c
 extracting: test2/a.c
zuozhongkai@ubuntu:~$ ls                //显示当前目录下所有文件
examples.desktop  test2      test3     tmp     模板  图片  下载  桌面
test1             test2.zip  test.txt  公共的  视频  文档  音乐
```

图 4.6.4.6 命令 unzip 使用演示

5、命令 tar

我们前面讲的 zip 和 unzip 这两个是命令只适用于.zip 格式的压缩和解压，其它压缩格式就用不了了，比如 Linux 下最常用的.bz2 和.gz 这两种压缩格式。其它格式的压缩和解压使用命令 tar，tar 将压缩和解压缩集合在一起，使用不同的参数即可，命令格式如下：

```
tar [参数] [压缩文件名] [被压缩文件名]
```

常用参数如下：
-c ：创建新的压缩文件。
-C<目的目录> ：切换到指定的目录。
-f<备份文件> ：指定压缩文件。
-j ：用 tar 生成压缩文件，然后用 bzip2 进行压缩。
-k ：解开备份文件时，不覆盖已有的文件。
-m ：还原文件时，不变更文件的更改时间。
-r ：新增文件到已存在的备份文件的结尾部分。
-t ：列出备份文件内容。
-v ：显示指令执行过程。
-w ：遭遇问题时先询问用户。
-x ：从备份文件中释放文件，也就是解压缩文件。
-z ：用 tar 生成压缩文件，用 gzip 压缩。
-Z ：用 tar 生成压缩文件，用 compress 压缩。
使用 tar 命令来进行.zip 和.gz 格式的文件压缩，操作如图 4.6.4.7 所示：

```
zuozhongkai@ubuntu:~$ ls   //显示当前目录下的所有文件
examples.desktop  test2  test.txt  公共的  视频  文档  音乐
test1             test3  tmp       模板    图片  下载  桌面
zuozhongkai@ubuntu:~$ tar -vcjf test1.tar.bz2 test1 //压缩为.bz2格式
test1/
test1/c.c
test1/b.c
test1/a.c
zuozhongkai@ubuntu:~$ ls                          //查看压缩后的文件是否存在
examples.desktop  test1.tar.bz2  test3     tmp     模板  图片  下载  桌面
test1             test2          test.txt  公共的  视频  文档  音乐
zuozhongkai@ubuntu:~$ tar -vczf test1.tar.gz test1 //压缩为.gz格式
test1/
test1/c.c
test1/b.c
test1/a.c
zuozhongkai@ubuntu:~$ ls                          //查看压缩后的文件是否存在
examples.desktop  test1.tar.bz2  test2  test.txt  公共的  视频  文档  音乐
test1             test1.tar.gz   test3  tmp       模板    图片  下载  桌面
```

图 4.6.4.7　tar 命令进行压缩

在图 4.6.4.7 中，我们使用如下两个命令将 test1 文件夹压缩为.bz2 和.gz 这两个格式：

```
tar  -vcjf    test1.tar.bz2  test1
tar  -vczf    test1.tar.gz   test1
```

在上面两行命令中，-vcjf 表示创建 bz2 格式的压缩文件，-vczf 表示创建.gz 格式的压缩文件。学习了如何使用 tar 命令来完成压缩，我们再来学习使用 tar 命令完成文件的解压，操作如图 4.6.4.8 所示：

```
zuozhongkai@ubuntu:~$ ls      //显示当前目录下所有文件
examples.desktop  test2.tar.gz  test.txt  公共的  视频  文档  音乐
test1.tar.bz2     test3         tmp       模板    图片  下载  桌面
zuozhongkai@ubuntu:~$ tar -vxjf test1.tar.bz2  //解压缩.bz2格式文件
test1/
test1/c.c
test1/b.c
test1/a.c
zuozhongkai@ubuntu:~$ ls                       //检查test1.tar.bz2是否解压缩成功
examples.desktop  test1.tar.bz2  test3     tmp     模板  图片  下载  桌面
test1             test2.tar.gz   test.txt  公共的  视频  文档  音乐
zuozhongkai@ubuntu:~$ tar -vxzf test2.tar.gz  //解压缩.gz格式文件
test2/
test2/d.c
test2/b.c
test2/a.c
zuozhongkai@ubuntu:~$ ls                       //检查test2.tar.gz是否解压缩成功
examples.desktop  test2         test.txt  模板  文档  桌面
test1             test2.tar.gz  tmp       视频  下载
test1.tar.bz2     test3         公共的    图片  音乐
```

图 4.6.4.8 tar 解压缩命令

图 4.6.4.8 中我们使用如下所示两行命令完成.bz2 和.gz 格式文件的解压缩：

```
tar  -vxjf    test1.tar.bz2
tar  -vxzf    test2.tar.gz
```

上述两行命令中，-vxjf 用来完成.bz2 格式压缩文件的解压，-vxzf 用来完成.gz 格式压缩文件的解压。关于 Ubuntu 下的命令行压缩和解压缩就讲解到这里，重点是 tar 命令，要熟练掌握使用 tar 命令来完成.bz2 和.gz 格式的文件压缩和解压缩。

4.6.5 文件查询和搜索

文件的查询和搜索也是最常用的操作，在嵌入式 Linux 开发中常常需要在 Linux 源码文件中查询某个文件是否存在，或者搜索哪些文件都调用了某个函数等等。本节我们就讲解两个最常用的文件查询和搜索命令：find 和 grep。

1、命令 find

find 命令用于在目录结构中查找文件，其命令格式如下：

```
find    [路径]  [参数]  [关键字]
```

路径是要查找的目录路径，如果不写的话表示在当前目录下查找，关键字是文件名的一部分，主要参数如下：

-name<filename> ：按照文件名称查找，查找与 filename 匹配的文件，可使用通配符。
-depth ：从指定目录下的最深层的子目录开始查找。
-gid<群组识别码>：查找符合指定的群组识别码的文件或目录。
-group<群组名称>：查找符合指定的群组名称的文件或目录。
-size<文件大小> ：查找符合指定文件大小的文件。
-type<文件类型> ：查找符合指定文件类型的文件。
-user<拥有者名称> ：查找符合指定的拥有者名称的文件或目录。

find 命令的参数有很多，常用的就这些，关于其它的参数大家可以自行上网查找，我们来看一下如何使用 find 命令进行文件搜索，我们搜索目录/etc 中以“vim”开头的文件为例，操作如图 4.6.5.1 所示：

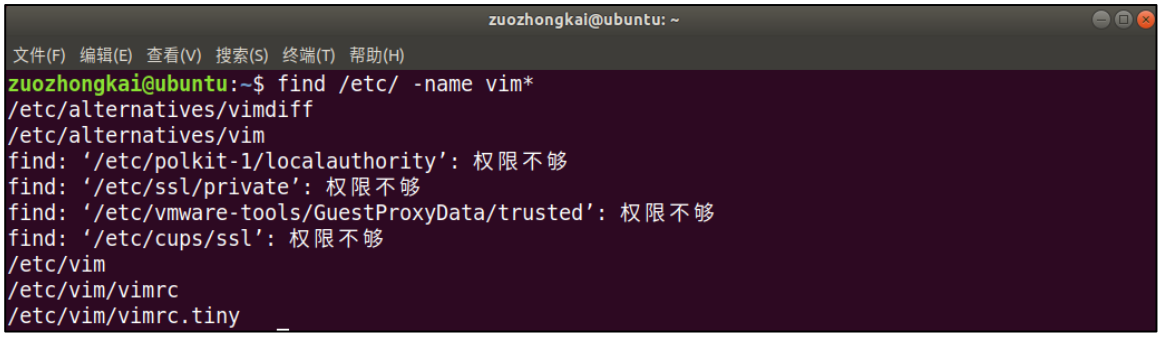


图 4.6.5.1 find 命令操作

从图 4.6.5.1 可以看出，在目录/etc 下，包含以“vim*”开头的文件有/etc/vim、/etc/vim/vimrc 等等，就不一一列出了。

2、命令 grep

find 命令用于在目录中搜索文件，我们有时候需要在文件中搜索一串关键字，grep 就是完成这个功能的，grep 命令用于查找包含指定关键字的文件，如果发现某个文件的内容包含所指定的关键字，grep 命令就会把包含指定关键字的这一行标记出来，grep 命令格式如下：

```
grep [参数]  关键字  文件列表
```

grep 命令一次只能查一个关键字，主要参数如下：

-b ：在显示符合关键字的那一列前，标记处该列第 1 个字符的位编号。
-c ：计算符合关键字的列数。
-d<进行动作> ：当指定要查找的是目录而非文件时，必须使用此参数！否则 grep 指令将回报信息并停止搜索。
-i ：忽略字符大小写。

-v ：反转查找，只显示不匹配的行。

-r ：在指定目录中递归查找。

比如我们在目录/usr 下递归查找包含字符“Ubuntu”的文件，操作如图 4.6.5.2 所示：

```
zuozhongkai@ubuntu:~$ grep -ir "Ubuntu" /usr
匹配到二进制文件 /usr/games/sol
匹配到二进制文件 /usr/src/linux-headers-5.4.0-42-generic/scripts/recordmcount
匹配到二进制文件 /usr/src/linux-headers-5.4.0-42-generic/scripts/bin2c
匹配到二进制文件 /usr/src/linux-headers-5.4.0-42-generic/scripts/conmakehash
匹配到二进制文件 /usr/src/linux-headers-5.4.0-42-generic/scripts/sign-file
匹配到二进制文件 /usr/src/linux-headers-5.4.0-42-generic/scripts/insert-sys-cert
匹配到二进制文件 /usr/src/linux-headers-5.4.0-42-generic/scripts/basic/fixdep
匹配到二进制文件 /usr/src/linux-headers-5.4.0-42-generic/scripts/mod/sumversion.o
匹配到二进制文件 /usr/src/linux-headers-5.4.0-42-generic/scripts/mod/mk_elfconfig
匹配到二进制文件 /usr/src/linux-headers-5.4.0-42-generic/scripts/mod/file2alias.o
匹配到二进制文件 /usr/src/linux-headers-5.4.0-42-generic/scripts/mod/modpost
/usr/src/linux-headers-5.4.0-42-generic/scripts/mod/.empty.o.cmd:cmd_scripts/mod/empty.o := g
cc -Wp,-MD,scripts/mod/.empty.o.d  -nostdinc -isystem /usr/lib/gcc/x86_64-linux-gnu/7/include
  -I/build/linux-hwe-5.4-huXhHV/linux-hwe-5.4-5.4.0/arch/x86/include -I./arch/x86/include/gen
erated -I/build/linux-hwe-5.4-huXhHV/linux-hwe-5.4-5.4.0/include -I./include -I/build/linux-h
we-5.4-huXhHV/linux-hwe-5.4-5.4.0/arch/x86/include/uapi -I./arch/x86/include/generated/uapi -
I/build/linux-hwe-5.4-huXhHV/linux-hwe-5.4-5.4.0/include/uapi -I./include/generated/uapi -inc
lude /build/linux-hwe-5.4-huXhHV/linux-hwe-5.4-5.4.0/include/linux/kconfig.h -Iubuntu/include
  -include /build/linux-hwe-5.4-huXhHV/linux-hwe-5.4-5.4.0/include/linux/compiler_types.h -D_
```

图 4.6.5.2 命令 grep 演示

4.6.6 文件类型

这里的文件类型不是说这个文件是音乐文件还是文本文件，在用户根目录下使用命令“ls -l”来查看用户根目录下所有文件的详细信息，如图 2.6.5.1 所示：

```
zuozhongkai@ubuntu:~$  ls -l
总用量 72
-rw-r--r-- 1 zuozhongkai zuozhongkai 8980 12月  18 02:08 examples.desktop
drwxrwxr-x 2 zuozhongkai zuozhongkai 4096 12月  24 21:56 test1
-rw-rw-r-- 1 zuozhongkai zuozhongkai  194 12月  24 21:58 test1.tar.bz2
drwxrwxr-x 2 zuozhongkai zuozhongkai 4096 12月  23 01:00 test2
-rw-rw-r-- 1 zuozhongkai zuozhongkai  171 12月  24 22:09 test2.tar.gz
drwxrwxr-x 2 zuozhongkai zuozhongkai 4096 12月  23 00:45 test3
-rw-rw-r-- 1 zuozhongkai zuozhongkai   68 12月  21 01:40 test.txt
drwxrwxr-x 2 zuozhongkai zuozhongkai 4096 12月  19 21:41 tmp
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月  18 02:21 公共的
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月  18 02:21 模板
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月  18 02:21 视频
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月  18 02:21 图片
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月  18 02:21 文档
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月  18 02:21 下载
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月  18 02:21 音乐
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月  22 02:08 桌面
```

图 2.6.5.1 文件详细信息

在图 2.6.5.1 中，每个文件的详细信息占一行，每行最前面都是一个符号就标记了当前文件类型，比如 test1 的第一个字符是“d”，test1.tar.bz2 文件第一个字符是“-”。这些字符表示的文件类型如下：

- ：普通文件，一些应用程序创建的，比如文档、图片、音乐等等。

d ：目录文件。

c ：字符设备文件，Linux 驱动里面的字符设备驱动，比如串口设备，音频设备等。

b ：块设备文件，存储设备驱动，比如硬盘，U 盘等。

l ：符号连接文件，相当于 Windwos 下的快捷方式。

s ：套接字文件。

p ：管道文件，主要指 FIFO 文件。

我们后面学习 Linux 驱动开发的时候基本是在和字符设备文件和块设备文件打交道。

4.7 Linux 用户权限管理

4.7.1 Ubuntu 用户系统

Ubuntu 是一个多用户系统，我们可以给不同的使用者创建不同的用户账号，每个用户使用各自的账号登陆，使用用户账号的目的一是方便系统管理员管理，控制不同用户对系统的访问权限，另一方面是为用户提供安全性保护。

我们前面在安装 Ubuntu 系统的时候被要求创建一个账户，当我们创建好账号以后，系统会在目录/home 下以该用户名创建一个文件夹，所有与该用户有关的文件都会被存储在这个文件文件夹中。同样的，创建其它用户账号的时候也会在目录/home 下生成一个文件夹来存储该用户的文件，图 2.7.1.1 就是我的电脑上“zuozhongkai”这个账户的文件夹。

```
zuozhongkai@ubuntu:~$ ls
examples.desktop  公共的  模板  视频  图片  文档  下载  音乐  桌面
```

图 4.7.1.1 用户账号根目录

装系统的时候创建的用户其权限比后面创建的用户大一点，但是没有 root 用户权限大，Ubuntu 下用户类型分为以下 3 类：

- 初次创建的用户，此用户可以完成比普通用户更多的功能。
- root 用户，系统管理员，系统中的玉皇大帝，拥有至高无上的权利。
- 普通用户，安装完操作系统以后被创建的用户。

以上三种用户，每个用户都有一个 ID 号，称为 UID，操作系统通过 UID 来识别是哪个用户，用户相关信息可以在文件/etc/passwd 中查看到，如图 4.7.1.2 所示：

```
zuozhongkai@ubuntu:~$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
```

图 4.7.1.2 passwd 文件内容

从配置文件 passwd 中可以看到，每个用户名后面都有两个数字，比如用户“zuozhongkai”后面“1000:1000”，第一个数字是用户的 ID，另一个是用户的 GID，也就是用户组 ID。Ubuntu 里面每个用户都属于一个用户组里面，用户组就是一组有相同属性的用户集合。

4.7.2 权限管理

在使用 Windows 的时候我们很少接触到用户权限，最多就是打开某个软件出问题的时候会选择以“管理员身份”打开。Ubuntu 下我们会常跟用户权限打交道，权限就是用户对于系统资源的使用限制情况，root 用户拥有最大的权限，可以为所欲为，装系统的时候创建的用户拥有 root 用户的部分权限，其它普通用户的权限最低。对于我们做嵌入式开发的人一般不关注用户的权限问题，因为嵌入式基本是单用户，做嵌入式开发重点关注的是文件的权限问题。

对于一个文件通常有三种权限：读(r)、写(w)和执行(x)，使用命令“ls -l”可以查看某个目录下所有文件的权限信息，如图 4.7.2.1 所示：

```
zuozhongkai@ubuntu:~$ ls -l
总用量 44
-rw-r--r-- 1 zuozhongkai zuozhongkai 8980 10月  20 18:23 examples.desktop
-rw-r--r-- 1 zuozhongkai zuozhongkai    0 10月  28 12:05 test.c
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 10月  20 18:49 公共的
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 10月  21 18:37 模板
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 10月  20 18:49 视频
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 10月  20 18:49 图片
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 10月  20 18:49 文档
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 10月  20 18:49 下载
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 10月  20 18:49 音乐
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 10月  28 10:47 桌面
```

图 4.7.2.1 文件权限信息

在图 4.7.2.1 中我们以文件 test.c 为例讲解，文件 test.c 文件信息如下：

```
-rw-r--r-- 1 zuozhongkai zuozhongkai       0 10 月  28 12:05 test.c
```

其中“-rw-rw-r--”表示文件权限与用户和用户组之间的关系，第一位表示文件类型，上一小节已经说了。剩下的 9 位以 3 位为一组，分别表示文件拥有者的权限，文件拥有者所在用户组的权限以及其它用户权限。后面的“zuozhongkai zuozhongkai”分别代表文件拥有者(用户)和该用户所在的用户组，因此文件 test.c 的权限情况如下：

①、文件 test.c 的拥有者是用户 zuozhongkai，其对文件 test.c 的权限是“rw-”，也就是对该文件拥有读和写两种权限。

②、用户 zuozhongkai 所在的用户组也叫做 zuozhongkai，其组内用户对于文件 test.c 的权限是“rw-”，也是拥有读和写这两种权限。

③、其它用户对于文件 test.c 的权限是“r--”，也就是只读权限。

对于文件，可读权限表示可以打开查看文件内容，可写权限表示可以对文件进行修改，可执行权限就是可以运行此文件(如果是软件的话)。对于文件夹，拥有可读权限才可以使用命令 ls 查看文件夹中的内容，拥有可执行权限才能进入到文件夹内部。

如果某个用户对某个文件不具有相应的权限的话就不能进行相应的操作，比如根目录“/”下的文件只有 root 用户才有权限进行修改，如果以普通用户去修改的话就会提示没有权限。比如我们要在根目录“/” 创建一个文件 mytest，使用命令“touch mytest”，结果如图 4.7.2.2 所示：

```
zuozhongkai@ubuntu:/$ touch mytest
touch: 无法创建'mytest': 权限不够
```

图 4.7.2.2 创建文件

在图 4.7.2.2 中，我以用户“zuozhongkai”在根目录“/”创建文件 mytest，结果提示我无法创建“mytest”，因为权限不够，因为只有 root 用户才能在根目录“/”下创建文件。我们可以使用命令“sudo”命令暂时切换到 root 用户，这样就可以在根目录“/”下创建文件 mytest 了，如图 4.7.2.3 所示：

```
zuozhongkai@ubuntu:/$ sudo touch mytest
[sudo] zuozhongkai 的密码：
zuozhongkai@ubuntu:/$ ls
bin   dev   initrd.img      lib64       mnt     proc  sbin  swapfile  usr      vmlinuz.old
boot  etc   initrd.img.old  lost+found  mytest  root  snap  sys       var
cdrom home  lib             media       opt     run   srv   tmp       vmlinuz
```

图 4.7.2.3 使用 sudo 命令创建文件

在图 4.7.2.3 中我们使用命令“sudo”以后就可以在根目录“/”创建文件 mytest，在进行其它的操作的时候，遇到提示权限不够的时候都可以使用 sudo 命令暂时以 root 用户身份去执行。

上面我们讲了，文件的权限有三种：读(r)、写(w)和执行(x)，除了用 r、w 和 x 表示以外，我们也可以使用二进制数表示，三种权限就可以使用 3 位二进制数来表示，一种权限对应一个二进制位，如果该位为 1 就表示具备此权限，如果该位为 0 就表示没不具备此权限，如表 4.7.2.1 所示：

字母	二进制	八进制
r	100	4
w	010	2
x	001	1

表 4.7.2.1 文件权限数字表示方法

如果做过单片机开发的话，就会发现和单片机里面的寄存器位一样，将三种权限 r、w 和 x 进行不同的组合，即可得到不同的二进制数和八进制数，3 位权限可以组出 8 种不同的权限组合，如表 4.7.2.2 所示：

权限	二进制数字	八进制数字
---	000	0
--x	001	1
-w-	010	2
-wx	011	3
r--	100	4
r-x	101	5
rw-	110	6
rwx	111	7

表 4.7.2.2 文件所有权限组合

表 4.7.2.2 中权限所对应的八进制数字就是每个权限对应的位相加，比如权限 rwx 就是 4+2+1=7。前面的文件 test.c 其权限为“rw-rw-r--”，因此其十进制表示就是：664。

另外我们也开始使用 a、u、g 和 o 表示文件的归属关系，用=、+和-表示文件权限的变化，如表 4.7.2.3 所示：

字母	意义
r	可读权限
w	可写权限
x	可执行权限
a	所有用户
u	归属用户
g	归属组
o	其它用户
=	具备权限

+	添加某权限
-	去除某权限

表 4.7.2.3 权限修改字母表示方式

对于文件 test.c，我们想要修改其归属用户(zuozhongkai)对其拥有可执行权限，那么就可以使用：u+x。如果希望设置归属用户及其所在的用户组都对其拥有可执行权限就可以使用：gu+x。

4.7.3 权限管理命令

我们也可以使用 Shell 来操作文件的权限管理，主要用到“chmod”和“chown”这两个命令，我们一个一个来看。

1、权限修改命令 chmod

命令“chmod”用于修改文件或者文件夹的权限，权限可以使用前面讲的数字表示也可以使用字母表示，命令格式如下：

```
chmod [参数] [文件名/目录名]
```

主要参数如下：

-c ：效果类似“-v”参数，但仅回显更改的部分。

-f ：不显示错误信息。

-R ：递归处理，指定目录下的所有文件及其子文件目录一起处理。

-v ：显示指令的执行过程。

我们先来学习以下如何使用命令“chmod”修改一个文件的权限，在用户根目录下创建一个文件 test，然后查看其默认权限，操作如图 4.7.3.1 所示：

```
zuozhongkai@ubuntu:~$ touch test
ubuntu:~$ ls -l
LibreOffice Calc
总用量 48
-rw-r--r-- 1 zuozhongkai zuozhongkai 8980 12月  18 02:08 examples.desktop
-rw-rw-r-- 1 zuozhongkai zuozhongkai    0 12月  25 22:24 test
drwxrwxr-x 2 zuozhongkai zuozhongkai 4096 12月  19 21:41 tmp
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月  18 02:21 公共的
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月  18 02:21 模板
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月  18 02:21 视频
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月  18 02:21 图片
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月  18 02:21 文档
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月  18 02:21 下载
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月  18 02:21 音乐
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月  22 02:08 桌面
```

图 4.7.3.1 创建文件 test

在图 4.7.3.1 中我们创建了一个文件：test，这个文件的默认权限为“rw-rw-r--”，我们将其权限改为“rwxrw-rw”，对应数字就是 766，操作如下：

```
zuozhongkai@ubuntu:~$ ls -l test    //查看文件test权限
-rw-rw-r-- 1 zuozhongkai zuozhongkai 0 12月  25 22:24 test
zuozhongkai@ubuntu:~$ chmod 766 test //修改文件test的权限为766
zuozhongkai@ubuntu:~$ ls -l test     //查看修改以后的文件test权限
-rwxrw-rw- 1 zuozhongkai zuozhongkai 0 12月  25 22:24 test
```

图 4.7.3.2 修改权限

在图 4.7.3.2 中，我们修改文件 test 的权限为 766，修改完成以后的 test 文件权限为“rwxrw-rw-”，和我们设置的一样，说明权限修改成功。

上面我们是通过数字来修改权限的，我们接下来使用字母来修改权限，操作如图 4.7.3.3 所示：

```
zuozhongkai@ubuntu:~$ touch a.c //创建文件a.c
zuozhongkai@ubuntu:~$ ls -l a.c //显示文件a.c的权限
-rw-rw-r-- 1 zuozhongkai zuozhongkai 0 12月  25 22:33 a.c
zuozhongkai@ubuntu:~$ chmod u+x a.c  //给文件a.c的归属用户添加可执行权限
zuozhongkai@ubuntu:~$ ls -l a.c      //显示修改权限后的文件a.c
-rwxrw-r-- 1 zuozhongkai zuozhongkai 0 12月  25 22:33 a.c
```

图 4.7.3.3 使用字母修改文件权限

上面两个例子都是修改文件的权限，接下来我们修改文件夹的权限，新建一个 test 文件夹，在文件夹 test 里面创建 a.c、b.c 和 c.c 三个文件，如图 4.7.3.4 所示：

```
zuozhongkai@ubuntu:~$ ls -l test/
总用量 0
-rw-rw-r-- 1 zuozhongkai zuozhongkai 0 12月  26 00:20 a.c
-rw-rw-r-- 1 zuozhongkai zuozhongkai 0 12月  26 00:20 b.c
-rw-rw-r-- 1 zuozhongkai zuozhongkai 0 12月  26 00:20 c.c
```

图 4.7.3.4 test 文件夹

在图 4.7.3.4 中 test 文件夹下的文件 a.c、b.c 和 c.c 的权限均为“rw-rw-r--”，我们将 test 文件夹下的所有文件权限都改为“rwxrwxrwx”，也就是数字 777，操作如图 4.7.3.5 所示：

```
zuozhongkai@ubuntu:~$ chmod -R 777 test/ //递归修改文件权限
zuozhongkai@ubuntu:~$ ls -l test/        //查看修改权限以后的文件
总用量 0
-rwxrwxrwx 1 zuozhongkai zuozhongkai 0 12月  26 00:20 a.c
-rwxrwxrwx 1 zuozhongkai zuozhongkai 0 12月  26 00:20 b.c
-rwxrwxrwx 1 zuozhongkai zuozhongkai 0 12月  26 00:20 c.c
```

图 4.7.3.5 递归修改文件夹权限

2、文件归属者修改命令 chown

命令 chown 用来修改某个文件或者目录的归属者用户或者用户组，命令格式如下：

```
chown  [参数]  [用户名.<组名>]  [文件名/目录]
```

其中[用户名.<组名>]表示要将文件或者目录改为哪一个用户或者用户组，用户名和组名用“.”隔开，其中用户名和组名中的任何一个都可以省略，命令主要参数如下：

-c ：效果同-v 类似，但仅回报更改的部分。

-f ：不显示错误信息。

-h ：只对符号连接的文件做修改，不改动其它任何相关的文件。

-R ：递归处理，将指定的目录下的所有文件和子目录一起处理。

-v ：显示处理过程。

在用户根目录下创建一个 test 文件，查看其文件夹所属用户和用户组，如图 4.7.3.6 所示：

```
zuozhongkai@ubuntu:~$ touch test
zuozhongkai@ubuntu:~$ ls -l test
-rw-rw-r-- 1 zuozhongkai zuozhongkai 0 12月  26 00:36 test
```

图 4.7.3.6 test 文件信息查询

从图 4.7.3.6 中可以看出，文件 test 的归属用户为 zuozhongkai，所属的用户组为 zuozhongkai，将文件 test 归属用户改为 root 用户，所属的用户组也改为 root，操作如图 4.7.3.7 所示：

```
zuozhongkai@ubuntu:~$ ls -l test    //显示文件test的归属用户和归属组
-rw-rw-r-- 1 zuozhongkai zuozhongkai 0 12月  26 00:45 test
zuozhongkai@ubuntu:~$ sudo chown root.root test  //修改文件test的归属用户和归属组
zuozhongkai@ubuntu:~$ ls -l test   //查看修改以后的文件test归属用户和归属组
-rw-rw-r-- 1 root root 0 12月  26 00:45 test
```

图 4.7.3.7 修改文件归属用户和归属组

命令 shown 同样也可以递归处理来修改文件夹的归属用户和用户组，用法和命令 chown 一样，这里就不演示了。

4.8 Linux 磁盘管理

4.8.1 Linux 磁盘管理基本概念

Linux 的磁盘管理体系和 Windows 有很大的区别，在 Windows 下经常会遇到“分区”这个概念，在 Linux 中一般不叫“分区”而叫“挂载点”。 “挂载点”就是将一个硬盘的一部分做成文件夹的形式，这个文件夹的名字就是“挂载点”，不管在哪个发行版的 Linux 中，用户是绝对看到不到 C 盘、D 盘这样的概念的，只能看到以文件夹形式存在的“挂载点”.

文件/etc/fstab 详细的记录了 Ubuntu 中硬盘分区的情况，如图 4.8.1.1 所示：

```
zuozhongkai@ubuntu:~$ cat /etc/fstab
# /etc/fstab: static file system information.
#
# Use 'blkid' to print the universally unique identifier for a
# device; this may be used with UUID= as a more robust way to name devices
# that works even if disks are added and removed. See fstab(5).
#
# <file system> <mount point>   <type>  <options>       <dump>  <pass>
# / was on /dev/sda1 during installation
UUID=6807330c-ef05-4fa4-93ca-a6419ed7b57d /               ext4    errors=remount-ro 0       1
/swapfile                                 none            swap    sw                0       0
```

图 4.8.1.1 文件 fstab

在图 4.8.1.1 中有一行“/ was on /dev/sda1 during installation”，意思是根目录“/”是在/dev/sda1 上的，其中“/”是挂载点，“/dev/sda1”就是我们装 Ubuntu 系统的硬盘。由于我们的系统是安装在虚拟机中的，因此图 4.8.1.1 没有出现实际的硬盘。可以通过如下命令查看当前系统中的磁盘：

```
ls  /dev/sd*
```

上述命令就是打印出所有以/dev/sd 开头的设备文件，如图 4.8.1.2 所示：

```
zuozhongkai@ubuntu:~$ ls /dev/sd*
/dev/sda  /dev/sda1
```

图 4.8.1.2 查看硬盘设备文件

在图 4.8.1.2 中有四个磁盘设备文件，其中 sd 表示是 SATA 硬盘或者其它外部设备，最后面的数字表示该硬盘上的第 n 个分区，比如/dev/sda1 就表示磁盘 sda 上的第一个分区。图 4.8.1.2 中都是以/dev/sda 开头的，说明当前只有一个硬盘。如果再插上 U 盘、SD 卡啥的就可能会出现/dev/sdb，/dev/sdc 等等。如果你的 U 盘有两个分区那么可能就会出现/dev/sdb1、dev/sdb2 这样的设备文件。比如我现在插入我的 U 盘，插入 U 盘会提示 U 盘是接到主机还是虚拟机，如图 4.8.1.3 所示：

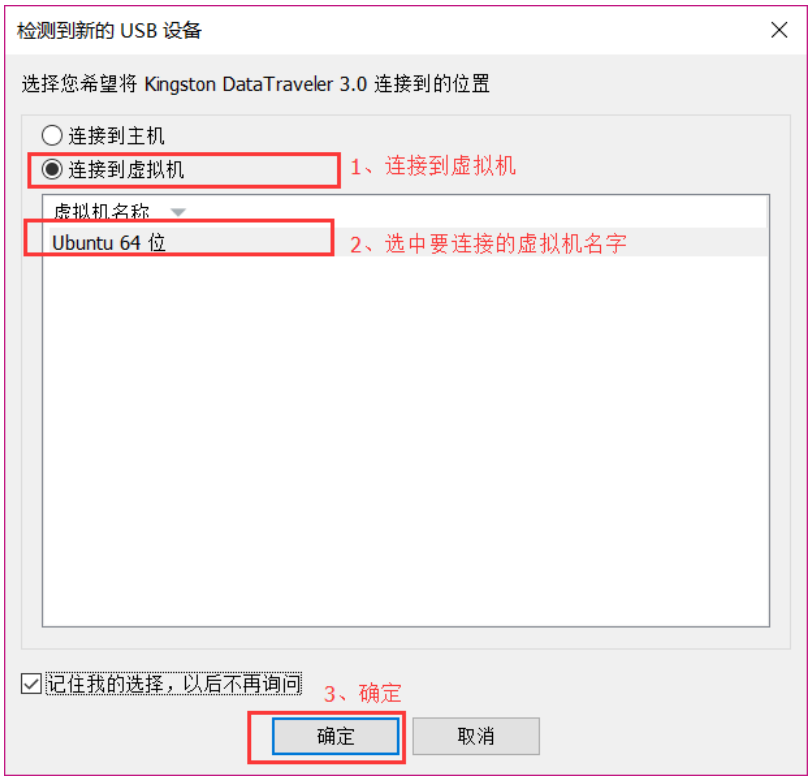


图 4.8.1.3 U 盘连接选择

设置好图 4.8.1.3 以后，点击“确定”按钮 U 盘就会自动连接到虚拟机中，也就是连接到 Ubuntu 系统中，我们再次使用命令“ls /dev/sd*”来查看当前的“/dev/sd*”设备文件，如图 4.8.1.4 所示：

```
zuozhongkai@ubuntu:~$ ls /dev/sd*
/dev/sda  /dev/sda1  /dev/sdb  /dev/sdb1
```

图 4.8.1.4 插入 U 盘后的设备文件

从图 4.8.1.4 可以看出，相比图 4.8.1.2 多了/dev/sdb 和/dev/sdb1 这两个文件，其中/dev/sdb 就是 U 盘文件，/dev/sdb1 表示 U 盘的第一个分区，因为我的 U 盘就一个分区。

4.8.2 磁盘管理命令

本节我们学习以下跟磁盘操作有关的命令，这些命令如下：

1、磁盘分区命令 fdisk

如果要对某个磁盘进行分区，可以使用命令 fdisk，命令格如下：

```
fdisk    [参数]
```

主要参数如下：

-b<分区大小>　　：指定每个分区的大小。

-l　　：列出指定设备的分区表。

-s<分区编号>　　：将指定的分区大小输出到标准的输出上，单位为块。

-u　：搭配“-l”参数，会用分区数目取代柱面数目，来表示每个分区的起始地址。

比如我要对 U 盘进行分区，千万不要对自己装 Ubuntu 系统进行分区！！！可以使用如下命令：

```
sudo  fdisk  /dev/sdb
```

结果如图 4.8.2.1 所示：

```
zuozhongkai@ubuntu:~$ sudo fdisk /dev/sdb
[sudo] zuozhongkai 的密码：

欢迎使用 fdisk (util-linux 2.31.1)。
更改将停留在内存中，直到您决定将更改写入磁盘。
使用写入命令前请三思。


命令(输入 m 获取帮助)：
```

图 4.8.2.1 U 盘分区界面

在图 4.8.2.1 中提示我们输入“m”可以查看帮助，因为 fdisk 还有一些字命令，通过输入“m”可以查看都有哪些子命令，如图 4.8.2.2 所示：

```
命令(输入 m 获取帮助)：  m

帮助：

  DOS (MBR)
   a   开关 可启动 标志
   b   编辑嵌套的 BSD 磁盘标签
   c   开关 dos 兼容性标志

  常规
   d   删除分区
   F   列出未分区的空闲区
   l   列出已知分区类型
   n   添加新分区
   p   打印分区表
   t   更改分区类型
   v   检查分区表
   i   打印某个分区的相关信息

  杂项
   m   打印此菜单
   u   更改 显示/记录 单位
   x   更多功能(仅限专业人员)

  脚本
   I   从 sfdisk 脚本文件加载磁盘布局
   O   将磁盘布局转储为 sfdisk 脚本文件

  保存并退出
   w   将分区表写入磁盘并退出
   q   退出而不保存更改

  新建空磁盘标签
   g   新建一份 GPT 分区表
   G   新建一份空 GPT (IRIX) 分区表
   o   新建一份的空 DOS 分区表
   s   新建一份空 Sun 分区表
```

图 4.8.2.2 fdisk 命令的子命令

图 4.8.2.2 中常用的命令如下：

p ：显示现有的分区
n ：建立新分区
t ：更改分区类型
d ：删除现有的分区
a ：更改分区启动标志

w ：对分区的更改写入到硬盘或者存储器中。

q ：不保存退出。

2、格式化命令 mkfs

使用命令 fdisk 创建好一个分区以后，我们需要对其格式化，也就是在这个分区上创建一个文件系统，Linux 下的格式化命令为 mkfs，命令格式如下：

```
mkfs  [参数]  [-t 文件系统类型]  [分区名称]
```

主要参数如下：

fs ：指定建立文件系统时的参数

-V ：显示版本信息和简要的使用方法。

-v ：显示版本信息和详细的使用方法。

比如我们要格式化 U 盘的分区/dev/sdb1 为 FAT 格式，那么就可以使用如下命令：

```
mkfs  –t  vfat  /dev/sdb1
```

3、挂载分区命令 mount

我们创建好分区并且格式化以后肯定是要使用硬盘或者 U 盘的，那么如何访问磁盘呢？比如我的 U 盘就一个分区，为/dev/sdb1，如果直接打开文件/dev/sdb1 会发现根本就不是我们要的结果。我们需要将/dev/sdb1 这个分区挂载到一个文件夹中，然后通过这个文件访问 U 盘，磁盘挂载命令为 mount，命令格式如下：

```
mount  [参数]  -t  [类型]  [设备名称]  [目的文件夹]
```

命令主要参数有：

-V ：显示程序版本。

-h ：显示辅助信息。

-v ：显示执行过程详细信息。

-o ro ：只读模式挂载。

-o rw ：读写模式挂载。

-s-r ：等于-o ro。

-w ：等于-o rw。

挂载点是一个文件夹，因此在挂载之前先要创建一个文件夹，一般我们把挂载点放到“/mnt”目录下，在“/mnt”下创建一个 tmp 文件夹，然后将 U 盘的/dev/sdb1 分区挂载到/mnt/tmp 文件夹里面，操作如图 4.8.2.3 所示：

```
zuozhongkai@ubuntu:~$ ls /mnt  //查看/mnt目录下的所有文件
zuozhongkai@ubuntu:~$
zuozhongkai@ubuntu:~$ sudo mkdir /mnt/tmp  //创建文件夹/mnt/tmp
[sudo] zuozhongkai 的密码：
zuozhongkai@ubuntu:~$ ls /mnt              //查看/mnt目录下的所有文件
tmp
zuozhongkai@ubuntu:~$ sudo mount -t vfat /dev/sdb1 /mnt/tmp //挂载U盘到/mnt/tmp中
zuozhongkai@ubuntu:~$ ls /mnt/tmp //查看/mnt/tmp中的文件，也就是U盘里面的文件
AD                                IMX6UL_ZERO_V1.0.zip        开发板光盘
ARM裸机与嵌入式Linux驱动开发V1.0.docx  System Volume Information
```

图 4.8.2.3 挂载 U 盘

在图 4.8.2.3 中我们将 U 盘以 fat 格式挂载到目录/mnt/tmp 中，然后我们就可以通过访问/mnt/tmp 来访问 U 盘了。

4、卸载命令 umount

当我们不在需要访问已经挂载的 U 盘，可以通过 umount 将其从卸载点卸除，命令格式如下：

```
umount [参数] -t [文件系统类型] [设备名称]
```

-a ：卸载/etc/mtab 中的所有文件系统。

-h ：显示帮助。

-n ：卸载时不要将信息存入到/etc/mtab 文件中

-r ：如果无法成功卸载，则尝试以只读的方式重新挂载。

-t<文件系统类型> ：仅卸载选项中指定的文件系统。

-v ：显示执行过程。

上面我们将 U 盘挂载到了文件夹/mnt/tmp 里面，这里我们使用命令 umount 将其卸载掉，操作如图 4.8.2.4 所示：

```
zuozhongkai@ubuntu:~$ ls /mnt/tmp //查看卸载之前的文件夹/mnt/tmp
AD                                IMX6UL_ZERO_V1.0.zip        开发板光盘
ARM裸机与嵌入式Linux驱动开发V1.0.docx  System Volume Information
zuozhongkai@ubuntu:~$ sudo umount -t vfat /dev/sdb1 //卸载U盘的/dev/sdb1分区
zuozhongkai@ubuntu:~$ ls /mnt/tmp                   //查看卸载以后的/mnt/tmp文件夹
zuozhongkai@ubuntu:~$
zuozhongkai@ubuntu:~$
```

图 4.8.2.4 卸载 U 盘

在图 4.8.2.4 中，我们使用命令 umount 卸载了 U 盘，卸载以后当我们再去访问文件夹/mnt/tmp 的时候发现里面没有任何文件了，说明我们卸载成功了。

第五章 Linux C 编程入门

在 Windows 下我们可是使用各种各样的 IDE 进行编程，比如强大的 Visual Studio。但是在 Ubuntu 下如何进行编程呢？Ubuntu 下也有一些可以进行编程的工具，但是大多都只是编辑器，也就是只能进行代码编辑，如果要编译的话就需要用到 GCC 编译器，使用 GCC 编译器肯定就要接触到 Makefile。本章就讲解如何在 Ubuntu 下进行 C 语言的编辑和编译、GCC 和 Makefile 的使用和编写。通过本章的学习可以掌握 Linux 进行 C 编程的基本方法，为以后的 Linux 驱动学习做准备。

5.1 Hello World!

我们所说的编写代码包括两部分：代码编写和编译，在 Windows 下可以使用 Visual Studio 来完成这两部分，可以在 Visual Studio 下编写代码然后直接点击编译就可以了。但是在 Linux 下这两部分是分开的，比如我们用 VIM 进行代码编写，编写完成以后在使用 GCC 编译器进行编译，其中代码编写工具很多，比如 VIM 编辑器、Emacs 编辑器、VScode 编辑器等等，本章使用 Ubuntu 自带的 VIM 编辑器。先来编写一个最简单的“Hello World”程序，把 Linux 下的 C 编程完整的走一遍。

5.1.1 编写代码

先在用户根目录下创建一个工作文件夹：C_Program，所有的 C 语言练习都保存到这个工作文件夹下，创建过程如图 5.1.1.1 所示：

```
zuozhongkai@ubuntu:~$ mkdir C_Program
zuozhongkai@ubuntu:~$ ls
C_Program  examples.desktop  公共的  模板  视频  图片  文档  下载  音乐  桌面
```

图 5.1.1.1 创建工作目录

进入图 5.1.1.1 创建的 C_Program 工作文件夹，为了方便管理，我们后面每个例程都创建一个文件夹来保存所有与本例程有关的文件，创建一个名为“3.1”的文件夹来保存我们的“Hello World”程序相关的文件，操作如图 5.1.1.2 所示：

```
zuozhongkai@ubuntu:~/C_Program$ mkdir 3.1
zuozhongkai@ubuntu:~/C_Program$ ls
3.1
zuozhongkai@ubuntu:~/C_Program$
```

图 5.1.1.2 创建工程文件夹

前面说了我们使用 VI 编辑器，在使用 VI 编辑器之前我们先做如下设置：

1、设置 TAB 键为 4 字节

VI 编辑器默认 TAB 键为 8 空格，我们改成 4 空格，用 vi 打开文件/etc/vim/vimrc，在此文件最后面输入如下代码：

```
set ts=4
```

添加完成如图 5.1.1.3 所示：

```
" Source a global configuration file if available
if filereadable("/etc/vim/vimrc.local")
  source /etc/vim/vimrc.local
endif

set ts=4
```

图 5.1.1.3 设置 TAB 为四个空格

修改完成以后保存并关闭文件。

2、VIM 编辑器显示行号

VIM 编辑器默认是不显示行号的，不显示行号不利于代码查看，我们设置 VIM 编辑器显示行号，同样是通过在文件/etc/vim/vimrc 中添加代码来实现，在文件最后面加入下面一行代码即可：

```
set nu
```

添加完成以后的/etc/vim/vimrc 文件如图 5.1.1.4 所示：

```
" Source a global configuration file if available
if filereadable("/etc/vim/vimrc.local")
  source /etc/vim/vimrc.local
endif

set ts=4
set nu
```

图 5.1.1.4 设置 VIM 编辑器显示行号

VIM 编辑器可以自行定制，网上有很多的博客讲解如何设置 VIM，感兴趣的可以上网看一下。设置好 VIM 编辑器以后就可以正式开始编写代码了，进入前面创建的“3.1”这个工程文件夹里面，使用 vi 指令创建一个名为“main.c”的文件，然后在里面输入如下代码：

示例代码 5.1.1.1 main.c 文件代码

```
1 #include <stdio.h>
2
3 int main(int argc, char *argv[])
4 {
5   printf("Hello World!\n");
6 }
```

编写完成以后保存退出 vi 编辑器，可以使用“cat”命令查看代码是否编写成功，如图 5.1.1.5 所示：

```
zuozhongkai@ubuntu:~/C_Program/3.1$ cat main.c
#include <stdio.h>

int main(int argc, char *argv[])
{
    printf("Hello World!\n");
}
```

图 5.1.1.5 查阅程序源码

从图 5.1.1.5 可以看出 main.c 文件是编辑成功的，代码编辑成功以后我们需要对其进行编译。

5.1.2 编译代码

Ubuntu 下的 C 语言编译器是 GCC，Ubuntu 默认没有安装 GCC 工具，因此需要我们手动安装 gcc、g++、make 等工具，这里我们直接安装 build-essential 软件包即可，build-essential 提供了编译程序所需的所有软件包，输入如下命令：

```
sudo apt-get install build-essential
```

等待安装完成，然后输入如下命令查看 GCC 编译器的版本号：

```
gcc -v
```

在终端中输入上述命令以后终端输出如图 5.1.2.1 所示：

```
zuozhongkai@ubuntu:~/C_Program/3.1$ gcc -v
Using built-in specs.
COLLECT_GCC=gcc
COLLECT_LTO_WRAPPER=/usr/lib/gcc/x86_64-linux-gnu/7/lto-wrapper
OFFLOAD_TARGET_NAMES=nvptx-none
OFFLOAD_TARGET_DEFAULT=1
Target: x86_64-linux-gnu
Configured with: ../src/configure -v --with-pkgversion='Ubuntu 7.5.0-3ubuntu1~18.04' --with-bugurl=fil
e:///usr/share/doc/gcc-7/README.Bugs --enable-languages=c,ada,c++,go,brig,d,fortran,objc,obj-c++ --pre
fix=/usr --with-gcc-major-version-only --program-suffix=-7 --program-prefix=x86_64-linux-gnu- --enable
-shared --enable-linker-build-id --libexecdir=/usr/lib --without-included-gettext --enable-threads=pos
ix --libdir=/usr/lib --enable-nls --enable-bootstrap --enable-clocale=gnu --enable-libstdcxx-debug --e
nable-libstdcxx-time=yes --with-default-libstdcxx-abi=new --enable-gnu-unique-object --disable-vtable-
verify --enable-libmpx --enable-plugin --enable-default-pie --with-system-zlib --with-target-system-zl
ib --enable-objc-gc=auto --enable-multiarch --disable-werror --with-arch-32=i686 --with-abi=m64 --with
-multilib-list=m32,m64,mx32 --enable-multilib --with-tune=generic --enable-offload-targets=nvptx-none
--without-cuda-driver --enable-checking=release --build=x86_64-linux-gnu --host=x86_64-linux-gnu --tar
get=x86_64-linux-gnu
Thread model: posix
gcc version 7.5.0 (Ubuntu 7.5.0-3ubuntu1~18.04)
```

图 5.1.2.1 gcc 版本查询

如果输入命令“gcc -v”命令以后，你的终端输出类似图 5.1.2.1 中的信息，那么说明你的电脑已经有 GCC 编译器了。最后下面的“gcc version 7.5.0”说明本机的 GCC 编译器版本为 7.5.0 的。注意观察在图 5.1.2.1 中有“Target: x86_64-linux-gnu”一行，这说明这个 GCC 编译器是针对 X86 架构的，因此只能编译在 X86 架构 CPU 上运行的程序。如果想要编译在 ARM 上运行的程序就需要针对 ARM 的 GCC 编译器，也就是交叉编译器！我们是 ARM 开发，因此肯定要安装针对 ARM 架构的 GCC 交叉编译器，当然了，这是后面的事，现在我们不用管这些，只要知道不同的目标架构，其 GCC 编译器是不同的。

如何使用 GCC 编译器来编译 main.c 文件呢？GCC 编译器是命令模式的，因此需要输入命令来使用 gcc 编译器来编译文件，输入如下命令：

```
gcc main.c
```

上述命令的功能就是使用 gcc 编译器来编译 main.c 这个 c 文件，过程如图 5.1.2.2 所示：

```
zuozhongkai@ubuntu:~/C_Program/3.1$ gcc main.c
zuozhongkai@ubuntu:~/C_Program/3.1$ ls
a.out  main.c
zuozhongkai@ubuntu:~/C_Program/3.1$
```

图 5.1.2.2 编译 main.c 文件

在图 5.1.2.2 中可以看到，当编译完成以后会生成一个 a.out 文件，这个 a.out 就是编译生成发的可执行文件，执行此文件看看是否和我们代码的功能一样，执行的方法很简单使用命令：“./+可执行文件”，比如本例程就是命令：./a.out，操作如图 5.1.2.3 所示：

```
zuozhongkai@ubuntu:~/C_Program/3.1$ ls
a.out  main.c
zuozhongkai@ubuntu:~/C_Program/3.1$ ./a.out
Hello World!
zuozhongkai@ubuntu:~/C_Program/3.1$
```

图 5.1.2.3 执行编译得到的文件

在图 5.1.2.3 中执行 a.out 文件以后终端输出了“Hello World！”，这正是 main.c 要实现的功能，说明我们的程序没有错误。a.out 这个文件的命名是 GCC 编译器自动命名的，那我们能不能决定编译完生成的可执行文件名字呢？肯定可以的，在使用 gcc 命令的时候加上-o 来指定生成的可执行文件名字，比如编译 main.c 以后生成名为“main”的可执行文件，操作如图 5.1.2.4 所示：

```
zuozhongkai@ubuntu:~/C_Program/3.1$ ls
a.out  main.c
zuozhongkai@ubuntu:~/C_Program/3.1$ gcc main.c -o main //指定编译生成的可执行文件名字为main
zuozhongkai@ubuntu:~/C_Program/3.1$ ls
a.out  main  main.c
zuozhongkai@ubuntu:~/C_Program/3.1$ ./main
Hello World!
zuozhongkai@ubuntu:~/C_Program/3.1$
```

图 5.1.2.4 指定可执行文件名字

在图 5.1.2.4 中，我们使用“gcc main.c –o main”来编译 main.c 文件，使用参数“-o”来指定编译生成的可执行文件名字，至此我们就完成 Linux 下 C 编程和编译的一整套过程。

5.2 GCC 编译器

5.2.1 gcc 命令

在上一小节我们已经使用过 GCC 编译器来编译 C 文件了，我们使用到是 gcc 命令，gcc 命令格式如下：

```
gcc [选项]  [文件名字]
```

主要选项如下：

-c：只编译不链接为可执行文件，编译器将输入的.c 文件编译为.o 的目标文件。

-o：<输出文件名>用来指定编译结束以后的输出文件名，如果使用这个选项的话 GCC 默认编译出来的可执行文件名字为 a.out。

-g：添加调试信息，如果要使用调试工具(如 GDB)的话就必须加入此选项，此选项指示编译的时候生成调试所需的符号信息。

-O：对程序进行优化编译，如果使用此选项的话整个源代码在编译、链接的的时候都会进行优化，这样产生的可执行文件执行效率就高。

-O2：比-O 更幅度更大的优化，生成的可执行效率更高，但是整个编译过程会很慢。

5.2.2 编译错误警告

在 Windows 下不管我们用啥编译器，如果程序有语法错误的话编译的时候都会指示出来，比如开发 STM32 的时候所使用的 MDK 和 IAR，我们可以根据错误信息方便的修改 bug。那 GCC 编译器有没有错误提示呢？肯定是有的，我们可以测试以下，新名为“3.2”的文件夹，使用 vi 在文件夹“3.2”中创建一个 main.c 文件，在文件里面输入如下代码：

示例代码 5.2.2.1 main.c 文件代码

```
1  #include <stdio.h>
2
3  int main(int argc, char *argv[])
4  {
5      int a, b;
6
7      a = 3;
8      b = 4
9      printf("a+b=\n", a + b);
10 }
```

在上述代码中有两处错误：

第 8 行、第一处是“b=4”少写了个一个“；”号。

第 9 行、第二处应该是 printf(“a+b=%d\n”， a + b);

我们编译以下上述代码，看看 GCC 编译器是否能够检查出错误，编译结果如图 5.2.2.1 所示：

```
zuozhongkai@ubuntu:~/C_Program/3.2$ gcc main.c -o main
main.c: In function ‘main’:
main.c:9:2: error: expected ‘;’ before ‘printf’
  printf("a+b=\n", a + b);
  ^~~~~~
zuozhongkai@ubuntu:~/C_Program/3.2$
```

图 5.2.2.1 错误提示

从图 5.2.2.1 中可以看出有一个 error，提示在 main.c 文件的第 9 行有错误，错误类型是在 printf 之前没有“；”号，这就是第一处错误，我们在“b = 4”后面加上分号，然后接着编译，结果又提示有一个错误，如图 5.2.2.2 所示：

```
zuozhongkai@ubuntu:~/C_Program/3.2$ gcc main.c -o main
main.c: In function ‘main’:
main.c:9:9: warning: too many arguments for format [-Wformat-extra-args]
  printf("a+b=\n", a + b);
         ^~~~~~~~
zuozhongkai@ubuntu:~/C_Program/3.2$
```

图 5.2.2.2 错误提示

在图 5.2.2.2 中，提示我们说文件 main.c 的第 9 行：printf(“a+b=\n”, a + b)有 warning，错误是因为太多参数了，我们将其改为：

```
printf(“a+b=%d\n”, a + b);
```

修改完成以后接着重新编译一下，结果如图 5.2.2.3 所示：

```
zuozhongkai@ubuntu:~/C_Program/3.2$ gcc main.c -o main
zuozhongkai@ubuntu:~/C_Program/3.2$ ls
main  main.c
zuozhongkai@ubuntu:~/C_Program/3.2$
```

图 5.2.2.3 编译成功

在图 5.2.2.3 中我们编译成功，生成了可执行文件 main，执行一下 main，看看结果和我们设计的是否一样，如图 5.2.2.4 所示：

```
zuozhongkai@ubuntu:~/C_Program/3.2$ ./main
a+b=7
zuozhongkai@ubuntu:~/C_Program/3.2$
```

图 5.2.2.4 执行结果

可以看出，GCC 编译器和其它编译器一样，不仅能够检测出错误类型，而且标记除了错误发生在哪个文件？哪一行？方便我们去修改代码。

5.2.3 编译流程

GCC 编译器的编译流程是：预处理、编译、汇编和链接。预处理就是展开所有的头文件、替换程序中的宏、解析条件编译并添加到文件中。编译是将经过预编译处理的代码编译成汇编代码，也就是我们常说的程序编译。汇编就是将汇编语言文件编译成二进制目标文件。链接就是将汇编出来的多个二进制目标文件链接在一起，形成最终的可执行文件，链接的时候还会涉及到静态库和动态库等问题。

上一小节演示的例程都只有一个文件，而且文件非常简单，因此可以直接使用 gcc 命令生成可执行文件，并没有先将 c 文件编译成.o 文件，然后再链接在一起。

5.3 Makefile 基础

5.3.1 何为 Makefile

上一小节我们讲了如何使用 GCC 编译器在 Linux 进行 C 语言编译，通过在终端执行 gcc 命令来完成 C 文件的编译，如果我们的工程只有一两个 C 文件还好，需要输入的命令不多，当文件有几十、上百甚至上万个的时候用终端输入 GCC 命令的方法显然是不现实的。如果我们能够编写一个文件，这个文件描述了编译哪些源码文件、如何编译那就好了，每次需要编译工程的时只需要使用这个文件就行了。这种问题怎么可能难倒聪明的程序员，为此提出了一个解决大工程编译的工具：make，描述哪些文件需要编译、哪些需要重新编译的文件就叫做 Makefile，Makefile 就跟脚本文件一样，Makefile 里面还可以执行系统命令。使用的时候只需要一个 make 命令即可完成整个工程的自动编译，极大的提高了软件开发的效率。如果大家以前一直使用 IDE 来编写 C 语言的话肯定没有听说过 Makefile 这个东西，其实这些 IDE 是有的，只不过这些 IDE 对其进行了封装，提供给大家的是已经经过封装后的图形界面了，我们在 IDE 中添加要编译的 C 文件，然后点击按钮就完成了编译。在 Linux 下用的最多的是 GCC 编译器，这是个没有 UI 的编译器，因此 Makefile 就需要我们自己来编写了。作为一个专业的程序员，是一定要懂得 Makefile 的，一是因为在 Linux 下你不得不懂 Makefile，再就是通过 Makefile 你就能了解整个工程的处理过程。

由于 Makefile 的知识比较多，完全可以单独写本书，因此本章我们只讲解 Makefile 基础入门，如果想详细的研究 Makefile，推荐大家阅读《跟我一起写 Makefile》这份文档，文档已经放到了开发板光盘->4、参考资料里面了，本章也有很多地方参考了此文档。

5.3.2 Makefile 的引入

我们完成这样一个小工程，通过键盘输入两个整形数字，然后计算他们的和并将结果显示在屏幕上，在这个工程中我们有 main.c、input.c 和 calcu.c 这三个 C 文件和 input.h、calcu.h 这两个头文件。其中 main.c 是主体，input.c 负责接收从键盘输入的数值，calcu.h 进行任意两个数相加，其中 main.c 文件内容如下：

示例代码 5.3.2.1 main.c 文件代码

```
1  #include <stdio.h>
2  #include "input.h"
3  #include "calcu.h"
4
5  int main(int argc, char *argv[])
6  {
7      int a, b, num;
8
9      input_int(&a, &b);
10     num  = calcu(a, b);
11     printf("%d + %d = %d\r\n", a, b, num);
12 }
```

input.c 文件内容如下：

示例代码 5.3.2.2 input.c 文件代码

```
1  #include <stdio.h>
```

```
2 #include "input.h"
3
4 void input_int(int *a, int *b)
5 {
6     printf("input two num:");
7     scanf("%d %d", a, b);
8     printf("\r\n");
9 }
```

calcu.c 文件内容如下：

示例代码 5.3.2.3 calcu.c 文件代码

```
1 #include "calcu.h"
2
3 int calcu(int a, int b)
4 {
5     return (a + b);
6 }
```

文件 input.h 内容如下：

示例代码 5.3.2.4 input.h 文件代码

```
1 #ifndef _INPUT_H
2 #define _INPUT_H
3
4 void input_int(int *a, int *b);
5 #endif
```

文件 calcu.h 内容如下：

示例代码 5.3.2.5 calcu.h 文件代码

```
1 #ifndef _CALCU_H
2 #define _CALCU_H
3
4 int calcu(int a, int b);
5 #endif
```

以上就是我们这个小工程的所有源文件，我们接下来使用 3.1 节讲的方法来对其进行编译，在终端输入如下命令：

```
gcc main.c calcu.c input.c -o main
```

上面命令的意思就是使用 gcc 编译器对 main.c、calcu.c 和 input.c 这三个文件进行编译，编译生成的可执行文件叫做 main。编译完成以后执行 main 这个程序，测试一下软件是否工作正常，结果如图 5.3.2.1 所示：

```
zuozhongkai@ubuntu:~/C_Program/3.3$ ls
calcu.c  calcu.h  input.c  input.h  main  main.c
zuozhongkai@ubuntu:~/C_Program/3.3$ ./main
input two num:5 6

5 + 6 = 11
```

图 5.3.2.1 程序测试

可以看出我们的代码按照我们所设想的工作了，使用命令“gcc main.c calcu.c input.c -o main”看起来很简单是吧，只需要一行就可以完成编译，但是我们这个工程只有三个文件啊！如果几千个文件呢？再就是如果有一个文件被修改了以，使用上面的命令编译的时候所有的文件都会重新编译，如果工程有几万个文件(Linux 源码就有这么多文件！)，想想这几万个文件编译一次所需要的时间就可怕。最好的办法肯定是哪个文件被修改了，只编译这个被修改的文件即可，其它没有修改的文件就不需要再次重新编译了，为此我们改变我们的编译方法，如果第一次编译工程，我们先将工程中的文件都编译一遍，然后后面修改了哪个文件就编译哪个文件，命令如下：

```
gcc -c main.c
gcc -c input.c
gcc -c calcu.c
gcc main.o input.o calcu.o -o main
```

上述命令前三行分别是将 main.c、input.c 和 calcu.c 编译成对应的.o 文件，所以使用了“-c”选项，“-c”选项我们上面说了，是只编译不链接。最后一行命令是将编译出来的所有.o 文件链接成可执行文件 main。假如我们现在修改了 calcu.c 这个文件，只需要将 caclue.c 这一个文件重新编译成.o 文件，然后在将所有的.o 文件链接成可执行文件即，只需要下面两条命令即可：

```
gcc -c calcu.c
gcc main.o input.o calcu.o -o main
```

但是这样就又有一个问题，如果修改的文件一多，我自己可能都不记得哪个文件修改过了，然后忘记编译，然后……，为此我们需要这样一个工具：

1、如果工程没有编译过，那么工程中的所有.c 文件都要被编译并且链接成可执行程序。

2、如果工程中只有个别 C 文件被修改了，那么只编译这些被修改的 C 文件即可。

3、如果工程的头文件被修改了，那么我们需要编译所有引用这个头文件的 C 文件，并且链接成可执行文件。

很明显，能够完成这个功能的就是 Makefile 了，在工程目录下创建名为“Makefile”的文件，文件名一定要叫做“Makefile”！！！区分大小写的哦！如图 5.3.2.2 所示：

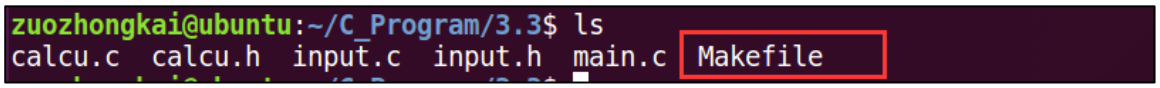


图 5.3.2.2 Makefile 文件

在图 5.3.2.2 中 Makefile 和 C 文件是处于同一个目录的，在 Makefile 文件中输入如下代码：

示例代码 5.3.2.6 Makefile 文件代码

```
1  main: main.o input.o calcu.o
2      gcc -o main  main.o input.o calcu.o
3  main.o: main.c
4      gcc -c main.c
5  input.o: input.c
6      gcc -c input.c
7  calcu.o: calcu.c
8      gcc -c calcu.c
9
10 clean:
11     rm *.o
```

```
12      rm main
```

上述代码中所有行首需要空出来的地方一定要使用“TAB”键！不要使用空格键！这是 Makefile 的语法要求，编写好得 Makefile 如图 5.3.2.3 所示：

```
1  main: main.o input.o calcu.o
2      gcc -o main  main.o input.o calcu.o
3  main.o: main.c
4      gcc -c main.c
5  input.o: input.c
6      gcc -c input.c
7  calcu.o: calcu.c
8      gcc -c calcu.c
9
10 clean:
11     rm *.o
12     rm main
13
```

图 5.3.2.3 Makefile 源码

Makefile 编写好以后我们就可以使用 make 命令来编译我们的工程了，直接在命令行中输入“make”即可，make 命令会在当前目录下查找是否存在“Makefile”这个文件，如果存在的话就会按照 Makefile 里面定义的编译方式进行编译，如图 5.3.2.4 所示：

```
zuozhongkai@ubuntu:~/C_Program/3.3$ ls   //查看目录下所有文件
calcu.c  calcu.h  input.c  input.h  main.c  Makefile
zuozhongkai@ubuntu:~/C_Program/3.3$ make //make命令编译工程
gcc -c main.c
gcc -c input.c
gcc -c calcu.c
gcc -o main  main.o input.o calcu.o
zuozhongkai@ubuntu:~/C_Program/3.3$ ls   //查看编译完成后的文件夹，看看编译是否成功
calcu.c  calcu.o  input.h  main    main.o
calcu.h  input.c  input.o  main.c  Makefile
```

图 5.3.2.4 Make 编译工程

在图 5.3.2.4 中，使用命令“make”编译完成以后就会在当前工程目录下生成各种.o 和可执行文件，说明我们编译成功了。使用 make 命令编译工程的时候可能会提示如图 5.3.2.5 所示错误:

```
zuozhongkai@ubuntu:~/C_Program/3.3$ make
Makefile:2: *** missing separator。 停止。
```

图 5.3.2.5 Make 失败

图 5.3.2.5 中的错误来源一般有两点:

1、Makefile 中命令缩进没有使用 TAB 键！

2、VI/VIM 编辑器使用空格代替了 TAB 键，修改文件/etc/vim/vimrc，在文件最后面加上如下所示代码：

```
set noexpandtab
```

我们修改一下 input.c 文件源码，随便加几行空行就行了，保证 input.c 被修改过即可，修改完成以后再执行一下“make”命令重新编译一下工程，结果如图 5.3.2.6 所示：

```
zuozhongkai@ubuntu:~/C_Program/3.3$ make
gcc -c input.c
gcc -o main  main.o input.o calcu.o
```

图 5.3.2.6 重新编译工程

从图 5.3.2.6 中可以看出因为我们修改了 input.c 这个文件，所以 input.c 和最后的可执行文件 main 重新编译了，其它没有修改过的文件就没有编译。而且我们只需要输入“make”这个命令即可，非常方便，但是 Makefile 里面的代码都是什么意思呢？这就是接下来我们要讲解的。

5.4 Makefile 语法

5.4.1 Makefile 规则格式

Makefile 里面是由一系列的规则组成的，这些规则格式如下：

```
目标……：  依赖文件集合……
    命令 1
    命令 2
    ……
```

比如下面这条规则：

```
main：  main.o  input.o  calcu.o
        gcc  -o  main  main.o  input.o  calcu.o
```

这条规则的目标是 main，main.o、input.o 和 calcu.o 是生成 main 的依赖文件，如果要更新目标 main，就必须先更新它的所有依赖文件，如果依赖文件中的任何一个有更新，那么目标也必须更新，“更新”就是执行一遍规则中的命令列表。

命令列表中的每条命令必须以 TAB 键开始，不能使用空格！

命令列表中的每条命令必须以 TAB 键开始，不能使用空格

命令列表中的每条命令必须以 TAB 键开始，不能使用空格！

make 命令会为 Makefile 中的每个以 TAB 开始的命令创建一个 Shell 进程去执行。

了解了 Makefile 的基本运行规则以后我们再来分析一下 5.3 节中“示例代码 5.3.2.6”中的 Makefile，代码如下：

```
1  main: main.o input.o calcu.o
2      gcc -o main  main.o input.o calcu.o
3  main.o: main.c
4      gcc -c main.c
5  input.o: input.c
6      gcc -c input.c
7  calcu.o: calcu.c
8      gcc -c calcu.c
9
10 clean:
11     rm *.o
12     rm main
```

上述代码中一共有 5 条规则，1~2 行为第一条规则，3~4 行为第二条规则，5~6 行为第三条规则，7~8 行为第四条规则，10~12 为第五条规则，make 命令在执行这个 Makefile 的时候其执行步骤如下：

首先更新第一条规则中的 main，第一条规则的目标成为默认目标，只要默认目标更新了那么就认为 Makefile 的工作，完成了整个 Makefile 就是为了完成这个工作。在第一次编译的时候由于 main 还不存在，因此第一条规则会执行，第一条规则依赖于文件 main.o、input.o 和 calcu.o 这个三个.o 文件，这三个.o 文件目前还都没有，因此必须先更新这三个文件。make 会查找以这

三个.o 文件为目标的规则并执行。以 main.o 为例，发现更新 main.o 的是第二条规则，因此会执行第二条规则，第二条规则里面的命令为“gcc –c main.c”，这行命令很熟悉了吧，就是不链接编译 main.c，生成 main.o，其它两个.o 文件同理。最后一个规则目标是 clean，它没有依赖文件，因此会默认为依赖文件都是最新的，所以其对应的命令不会执行，当我们想要执行 clean 的话可以直接使用命令“make clean”，执行以后就会删除当前目录下所有的.o 文件以及 main，因此 clean 的功能就是完成工程的清理，“make clean”的执行过程如图 3.4.1.1 所示：

```
zuozhongkai@ubuntu:~/C_Program/3.3$ ls
calcu.c  calcu.o  input.h  main    main.o
calcu.h  input.c  input.o  main.c  Makefile
zuozhongkai@ubuntu:~/C_Program/3.3$ make clean
rm *.o
rm main
zuozhongkai@ubuntu:~/C_Program/3.3$ ls
calcu.c  calcu.h  input.c  input.h  main.c  Makefile
```

图 5.4.1.1 make clean 执行过程

从图 54.1.1 可以看出，当执行“make clean”命令以后，前面编译出来的.o 和 main 可执行文件都被删除掉了，也就是完成了工程清理工作。

我们在来总结一下 Make 的执行过程：

1、make 命令会在当前目录下查找以 Makefile(makefile 其实也可以)命名的文件。

2、当找到 Makefile 文件以后就会按照 Makefile 中定义的规则去编译生成最终的目标文件。

3、当发现目标文件不存在，或者目标所依赖的文件比目标文件新(也就是最后修改时间比目标文件晚)的话就会执行后面的命令来更新目标。

这就是 make 的执行过程，make 工具就是在 Makefile 中一层一层的查找依赖关系，并执行相应的命令。编译出最终的可执行文件。Makefile 的好处就是“自动化编译”，一旦写好了 Makefile 文件，以后只需要一个 make 命令即可完成整个工程的编译，极大的提高了开发效率。把 make 和 Makefile 和做菜类似，目标都是呈现出一场盛宴，它们之间的对比关系如表 5.4.1.1 所示：

make	做菜	描述
make 工具	厨师	负责将材料加工成“美味”。
gcc 编译器	厨具	加工“美味”的工具。
Makefile	食材	加工美味所需的原材料。
最终的可执行文件	最终的菜肴	最终目的，呈现盛宴

表 3.4.1.1 make 和做菜对比

总结一下，Makefile 中规则用来描述在什么情况下使用什么命令来构建一个特定的文件，这个文件就是规则的“目标”，为了生成这个“目标”而作为材料的其它文件称为“目标”的依赖，规则的命令是用来创建或者更新目标的。

除了 Makefile 的“终极目标”所在的规则以外，其它规则的顺序在 Makefile 中是没有意义的，“终极目标”就是指在使用 make 命令的时候没有指定具体的目标时，make 默认的那个目标，它是 Makefile 文件中第一个规则的目标，如果 Makefile 中的第一个规则有多个目标，那么这些目标中的第一个目标就是 make 的“终极目标”。

5.4.2 Makefile 变量

跟 C 语言一样 Makefile 也支持变量的，先看一下前面的例子：

```
main: main.o input.o calcu.o
	gcc -o main  main.o input.o calcu.o
```

上述 Makefile 语句中，main.o input.o 和 calcue.o 这三个依赖文件，我们输入了两遍，我们这个 Makefile 比较小，如果 Makefile 复杂的时候这种重复输入的工作就会非常费时间，而且非常容易输错，为了解决这个问题，Makefile 加入了变量支持。不像 C 语言中的变量有 int、char 等各种类型，Makefile 中的变量都是字符串！类似 C 语言中的宏。使用变量将上面的代码修改，修改以后如下所示：

示例代码 5.4.2.1 Makefile 变量使用

```
1 #Makefile 变量的使用
2 objects = main.o input.o calcu.o
3 main: $(objects)
4     gcc -o main $(objects)
```

我们来分析一下“示例代码 5.4.2.1”，第 1 行是注释，Makefile 中可以写注释，注释开头要用符号“#”，不能用 C 语言中的“//”或者“/**/”！第 2 行我们定义了一个变量 objects，并且给这个变量进行了赋值，其值为字符串“main.o input.o calcu.o”，第 3 和 4 行使用到了变量 objects，Makefile 中变量的引用方法是“$(变量名)”，比如本例中的“$(objects)”就是使用变量 objects。

在“示例代码 5.4.2.1”中我们在定义变量 objects 的时候使用“=”对其进行了赋值，Makefile 变量的赋值符还有其它两个“:=”和“?=”，我们来看一下这三种赋值符的区别：

1、赋值符“=”

使用“=”在给变量的赋值的时候，不一定要用已经定义好的值，也可以使用后面定义的值，比如如下代码：

示例代码 3.4.2.1 赋值符"="使用

```
1 name = zzk
2 curname = $(name)
3 name = zuozhongkai
4
5 print:
6     @echo curname: $(curname)
```

我们来分析一下上述代码，第 1 行定义了一个变量 name，变量值为“zzk”，第 2 行也定义了一个变量 curname，curname 的变量值引用了变量 name，按照我们 C 写语言的经验此时 curname 的值就是“zzk”。第 3 行将变量 name 的值改为了“zuozhongkai”，第 5、6 行是输出变量 curname 的值。在 Makefile 要输出一串字符的话使用“echo”，就和 C 语言中的“printf”一样，第 6 行中的“echo”前面加了个“@”符号，因为 Make 在执行的过程中会自动输出命令执行过程，在命令前面加上“@”的话就不会输出命令执行过程，大家可以测试一下不加“@”的效果。使用命令“make print”来执行上述代码，结果如图 5.4.2.1：

```
zuozhongkai@ubuntu:~/C_Program$ make print
curname: zuozhongkai
zuozhongkai@ubuntu:~/C_Program$
```

图 5.4.2.1 make 执行结果

在图 5.4.2.1 中可以看到 curname 的值不是“zzk”，竟然是“zuozhongkai”，也就是变量“name”最后一次赋值的结果，这就是赋值符“=”的神奇之处！借助另外一个变量，可以将变量的真实值推到后面去定义。也就是变量的真实值取决于它所引用的变量的最后一次有效值。

2、赋值符“:=”

在“示例代码 5.4.2.1”上来测试赋值符“:=”，修改“示例代码 5.4.2.1”中的第 2 行，将其中的“=”改为“:=”，修改完成以后的代码如下：

示例代码 5.4.2.2 ":="的使用

```
1 name = zzk
2 curname := $(name)
3 name = zuozhongkai
4
5 print:
6     @echo curname: $(curname)
```

修改完成以后重新执行一下 Makefile，结果如图 5.4.2.2 所示：

```
zuozhongkai@ubuntu:~/C_Program$ make print
curname: zzk
zuozhongkai@ubuntu:~/C_Program$
```

图 5.4.2.2 make 执行结果

从图 5.4.2.2 中可以看到此时的 curname 是 zzk，不是 zuozhongkai 了。这是因为赋值符“:=”不会使用后面定义的变量，只能使用前面已经定义好的，这就是“=”和“:=”两个的区别。

3、赋值符“?=”

“?=”是一个很有用的赋值符，比如下面这行代码：

```
curname ?= zuozhongkai
```

上述代码的意思就是，如果变量 curname 前面没有被赋值，那么此变量就是“zuozhongkai”，如果前面已经赋过值了，那么就使用前面赋的值。

4、变量追加“+=”

Makefile 中的变量是字符串，有时候我们需要给前面已经定义好的变量添加一些字符串进去，此时就要使用到符号“+=”，比如如下所示代码：

```
objects = main.o inpiut.o
objects += calcu.o
```

一开始变量 objects 的值为“main.o input.o”，后面我们给他追加了一个“calcu.o”，因此变量 objects 变成了“main.o input.o calcu.o”，这个就是变量的追加。

5.4.3 Makefile 模式规则

在 5.3.2 小节中我们编写了一个 Makefile 文件用来编译工程，这个 Makefile 的内容如下：

示例代码 5.4.3.1 Makefile 文件代码

```
1  main: main.o input.o calcu.o
2      gcc -o main  main.o input.o calcu.o
3  main.o: main.c
4      gcc -c main.c
5  input.o: input.c
6      gcc -c input.c
7  calcu.o: calcu.c
8      gcc -c calcu.c
9
10 clean:
```

```
11      rm *.o
12      rm main
```

上述 Makefile 中第 3~8 行是将对应的.c 源文件编译为.o 文件，每一个 C 文件都要写一个对应的规则，如果工程中 C 文件很多的话显然不能这么做。为此，我们可以使用 Makefile 中的模式规则，通过模式规则我们就可以使用一条规则来将所有的.c 文件编译为对应的.o 文件。

模式规则中，至少在规则的目标定定义中要包涵“%”，否则就是一般规则，目标中的“%”表示对文件名的匹配，“%”表示长度任意的非空字符串，比如“%.c”就是所有的以.c 结尾的文件，类似与通配符，a.%.c 就表示以 a.开头，以.c 结束的所有文件。

当“%”出现在目标中的时候，目标中“%”所代表的值决定了依赖中的“%”值，使用方法如下：

```
%.o : %.c
    命令
```

因此“示例代码 5.4.3.1”中的 Makefile 可以改为如下形式：

示例代码 5.4.3.2 模式规则使用

```
1  objects = main.o input.o calcu.o
2  main: $(objects)
3      gcc -o main $(objects)
4
5  %.o : %.c
6      #命令
7
8  clean:
9      rm *.o
10     rm main
```

“示例代码 5.4.3.2”中第 5、6 这两行代码替代了“示例代码 5.4.3.1”中的 3~8 行代码，修改以后的 Makefile 还不能运行，因为第 6 行的命令我们还没写呢，第 6 行的命令我们需要借助另外一种强大的变量—自动化变量。

5.4.4 Makefile 自动化变量

上面讲的模式规则中，目标和依赖都是一系列的文件，每一次对模式规则进行解析的时候都会是不同的目标和依赖文件，而命令只有一行，如何通过一行命令来从不同的依赖文件中生成对应的目标？自动化变量就是完成这个功能的！所谓自动化变量就是这种变量会把模式中所定义的一系列的文件自动的挨个取出，直至所有的符合模式的文件都取完，自动化变量只应该出现在规则的命令中，常用的自动化变量如表 5.4.4.1：

自动化变量	描述
$@	规则中的目标集合，在模式规则中，如果有多个目标的话，“$@”表示匹配模式中定义的目标集合。
$%	当目标是函数库的时候表示规则中的目标成员名，如果目标不是函数库文件，那么其值为空。
$<	依赖文件集合中的第一个文件，如果依赖文件是以模式(即“%”)定义的，那么“$<”就是符合模式的一系列的文件集合。

$?	所有比目标新的依赖目标集合，以空格分开。
$^	所有依赖文件的集合，使用空格分开，如果在依赖文件中有多个重复的文件，“$^”会去除重复的依赖文件，值保留一份。
$+	和“$^”类似，但是当依赖文件存在重复的话不会去除重复的依赖文件。
$*	这个变量表示目标模式中"%"及其之前的部分，如果目标是 test/a.test.c，目标模式为 a.%.c，那么“$*”就是 test/a.test。

表 5.4.4.1 自动化变量

表 5.4.4.1 中的 7 个自动化变量中，常用的三种：$@、$<和$^，我们使用自动化变量来完成“示例代码 5.4.3.2”中的 Makefile，最终的完整代码如下所示：

示例代码 5.4.4.1 自动化变量

```
1  objects = main.o input.o calcu.o
2  main: $(objects)
3      gcc -o main $(objects)
4
5  %.o : %.c
6      gcc -c $<
7
8  clean:
9      rm *.o
10     rm main
```

上述代码代码就是修改后的完成的 Makefile，可以看出相比 5.3.2 小节中的要精简了很多，核心就在于第 5、6 这两行，第 5 行使用了模式规则，第 6 行使用了自动化变量。

5.4.5 Makefile 伪目标

Makefile 有一种特殊的目标——伪目标，一般的目标名都是要生成的文件，而伪目标不代表真正的目标名，在执行 make 命令的时候通过指定这个伪目标来执行其所在规则的定义的命令。

使用伪目标的主要是为了避免 Makefile 中定义的只执行命令的目标和工作目录下的实际文件出现名字冲突，有时候我们需要编写一个规则用来执行一些命令，但是这个规则不是用来创建文件的，比如在前面的“示例代码 5.4.4.1”中有如下代码用来完成清理工程的功能：

```
clean:
rm *.o
    rm main
```

上述规则中并没有创建文件 clean 的命令，因此工作目录下永远都不会存在文件 clean，当我们输入“make clean”以后，后面的“rm *.o”和“rm main”总是会执行。可是如果我们“手贱”，在工作目录下创建一个名为“clean”的文件，那就不一样了，当执行“make clean”的时候，规则因为没有依赖文件，所以目标被认为是最新的，因此后面的 rm 命令也就不会执行，我们预先设想的清理工程的功能也就无法完成。为了避免这个问题，我们可以将 clean 声明为伪目标，声明方式如下：

```
.PHONY : clean
```

我们使用伪目标来更改“示例代码 5.4.4.1”，修改完成以后如下：

示例代码 3.4.5.1 伪目标

```
1  objects = main.o input.o calcu.o
```

```
2  main: $(objects)
3      gcc -o main $(objects)
4
5  .PHONY : clean
6
7  %.o : %.c
8      gcc -c $<
9
10 clean:
11     rm *.o
12     rm main
```

上述代码第 5 行声明 clean 为伪目标，声明 clean 为伪目标以后不管当前目录下是否存在名为“clean”的文件，输入“make clean”的话规则后面的 rm 命令都会执行。

5.4.6 Makefile 条件判断

在 C 语言中我们通过条件判断语句来根据不同的情况来执行不同的分支，Makefile 也支持条件判断，语法有两种如下：

```
<条件关键字>
    <条件为真时执行的语句>
endif
```

以及：

```
<条件关键字>
    <条件为真时执行的语句>
else
    <条件为假时执行的语句>
endif
```

其中条件关键字有 4 个：ifeq、ifneq、ifdef 和 ifndef，这四个关键字其实分为两对、ifeq 与 ifneq、ifdef 与 ifndef，先来看一下 ifeq 和 ifneq，ifeq 用来判断是否相等，ifneq 就是判断是否不相等，ifeq 用法如下：

```
ifeq (<参数 1>,  <参数 2>)
ifeq ‘<参数 1 >’,‘ <参数 2>’
ifeq “<参数 1>”, “<参数 2>”
ifeq “<参数 1>”, ‘<参数 2>’
ifeq ‘<参数 1>’, “<参数 2>”
```

上述用法中都是用来比较“参数 1”和“参数 2”是否相同，如果相同则为真，“参数 1”和“参数 2”可以为函数返回值。ifneq 的用法类似，只不过 ifneq 是用来了比较“参数 1”和“参数 2”是否不相等，如果不相等的话就为真。

ifdef 和 ifndef 的用法如下：

```
ifdef <变量名>
```

如果“变量名”的值非空，那么表示表达式为真，否则表达式为假。“变量名”同样可以是一个函数的返回值。ifndef 用法类似，但是含义用户 ifdef 相反。

5.4.7 Makefile 函数使用

Makefile 支持函数，类似 C 语言一样，Makefile 中的函数是已经定义好的，我们直接使用，不支持我们自定义函数。make 所支持的函数不多，但是绝对够我们使用了，函数的用法如下：

```
$(函数名 参数集合)
```

或者

```
${函数名 参数集合}
```

可以看出，调用函数和调用普通变量一样，使用符号“$”来标识。参数集合是函数的多个参数，参数之间以逗号“,”隔开，函数名和参数之间以“空格”分隔开，函数的调用以“$”开头。接下来我们介绍几个常用的函数，其它的函数大家可以参考《跟我一起写 Makefile》这份文档。

1、函数 subst

函数 subst 用来完成字符串替换，调用形式如下：

```
$(subst <from>,<to>,<text>)
```

此函数的功能是将字符串<text>中的<from>内容替换为<to>，函数返回被替换以后的字符串，比如如下示例：

```
$(subst    zzk,ZZK,my name is zzk)
```

把字符串“my name is zzk”中的“zzk”替换为“ZZK”，替换完成以后的字符串为“my name is ZZK”。

2、函数 patsubst

函数 patsubst 用来完成模式字符串替换，使用方法如下：

```
$(patsubst <pattern>,<replacement>,<text>)
```

此函数查找字符串<text>中的单词是否符合模式<pattern>，如果匹配就用<replacement>来替换掉，<pattern>可以使用包括通配符“%”，表示任意长度的字符串，函数返回值就是替换后的字符串。如果<replacement>中也包涵“%”，那么<replacement>中的“%”将是<pattern>中的那个“%”所代表的字符串，比如：

```
$(patsubst %.c,%.o,a.c b.c c.c)
```

将字符串“a.c b.c c.c”中的所有符合“%.c”的字符串，替换为“%.o”，替换完成以后的字符串为“a.o b.o c.o”。

3、函数 dir

函数 dir 用来获取目录，使用方法如下：

```
$(dir <names…>)
```

此函数用来从文件名序列<names>中提取出目录部分，返回值是文件名序列<names>的目录部分，比如：

```
$(dir </src/a.c>)
```

提取文件“/src/a.c”的目录部分，也就是“/src”。

4、函数 notdir

函数 notdir 看名字就是知道去除文件中的目录部分，也就是提取文件名，用法如下：

```
$(notdir <names…>)
```

此函数用与从文件名序列<names>中提取出文件名非目录部分，比如：

```
$(notdir </src/a.c>)
```

提取文件“/src/a.c”中的非目录部分，也就是文件名“a.c”。

5、函数 foreach

foreach 函数用来完成循环，用法如下：

```
$(foreach <var>, <list>,<text>)
```

此函数的意思就是把参数<list>中的单词逐一取出来放到参数<var>中，然后再执行<text>所包含的表达式。每次<text>都会返回一个字符串，循环的过程中，<text>中所包含的每个字符串会以空格隔开，最后当整个循环结束时，<text>所返回的每个字符串所组成的整个字符串将会是函数 foreach 函数的返回值。

6、函数 wildcard

通配符“%”只能用在规则中，只有在规则中它才会展开，如果在变量定义和函数使用时，通配符不会自动展开，这个时候就要用到函数 wildcard，使用方法如下：

```
$(wildcard PATTERN…)
```

比如：

```
$(wildcard *.c)
```

上面的代码是用来获取当前目录下所有的.c 文件，类似“%”。

关于 Makefile 的相关内容就讲解到这里，本节只是对 Makefile 做了最基本的讲解，确保大家能够完成后续的学习，Makefile 还有大量的知识没有提到，有兴趣的可以自行参考《跟我一起写 Makefile》这份文档来深入学习 Makefile。